Computer Vision · Weeks 2 & 3

Beginner study guide (notebooks with outputs)

01 Neural-network building blocks · 02 LeNet, AlexNet, VGG · 03 ResNet, EfficientNet
04 Object detection · 05 Segmentation & Mask R-CNN · 06 RNN, LSTM, video · 07 Vision Transformers

Plus: answers to the 60-question course question bank and the cheat sheet.
Runnable notebooks are in the **notebooks/** folder next to this PDF.

01 · Neural-network building blocks

From Week 1's hand-made kernels to *learned* kernels

Covers (Week 3 Key Concepts): fully connected vs convolutional layers · ReLU · Dropout · pooling · how a network learns. **Questions answered here:** Easy 5–7 · Hard 1–2 · Week-1 Q "role of kernels in CNNs".

Everything later this week (LeNet, ResNet, YOLO, U-Net, LSTMs, ViT) is built from the few pieces in this notebook. Run each cell with **Shift + Enter** and read the text around it.

> **The one-sentence version:** a CNN is a stack of Week-1 convolutions, but instead of a human choosing the kernel numbers (like Sobel), **the computer learns them from examples**.

0 · Setup

Install once, from a terminal: `pip install torch torchvision scikit-learn matplotlib opencv-python`. On Google Colab everything is already installed.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
import cv2

torch.manual_seed(0)
np.random.seed(0)
print("torch", torch.__version__, "| GPU available:", torch.cuda.is_available())
```

```
torch 2.14.1+cu130 | GPU available: False
```

1 · A convolution layer *is* Week-1 convolution

PyTorch's `nn.Conv2d` slides kernels over an image exactly like `cv2.filter2D`. To prove it, we create a conv layer, **paste the Sobel-X kernel into its weights by hand**, and compare it with OpenCV.

- `nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3, padding=1, bias=False)` means 1 input channel (grayscale) and 1 kernel of size 3×3. `padding=1` keeps the output the same size, like OpenCV's border handling.
- PyTorch images are shaped **(batch, channels, height, width)**, unlike OpenCV's (height, width, channels).

```
# A small test image: dark left half, bright right half -> one vertical edge
img = np.zeros((8, 8), np.float32)
img[:, 4:] = 200
```

```
sobel_x = np.array([[-1, 0, 1],
                    [-2, 0, 2],
                    [-1, 0, 1]], np.float32)

conv = nn.Conv2d(1, 1, kernel_size=3, padding=1, bias=False)
with torch.no_grad():
    conv.weight[:] = torch.from_numpy(sobel_x)          # paste Sobel into the layer

x = torch.from_numpy(img)[None, None]                   # (8,8) -> (1,1,8,8) = (batch, channel,
H, W)
torch_out = conv(x)[0, 0].detach().numpy()
cv_out = cv2.filter2D(img, -1, sobel_x, borderType=cv2.BORDER_CONSTANT)

print("PyTorch conv layer output (row 0):", torch_out[0])
print("OpenCV filter2D output    (row 0):", cv_out[0])
print("Same result?", np.allclose(torch_out, cv_out))
```

```
PyTorch conv layer output (row 0): [   0.    0.    0.  600.  600.    0.    0. -600.]
OpenCV filter2D output    (row 0): [   0.    0.    0.  600.  600.    0.    0. -600.]
Same result? True
```

Same numbers. The only difference in a real CNN is that the 9 kernel numbers start **random** and are adjusted during training until they become useful detectors. A CNN layer also has **many** kernels (e.g. 64), each producing its own **feature map**.

2 · Fully connected (dense) layer vs convolutional layer

	Fully connected (`nn.Linear`)	Convolutional (`nn.Conv2d`)
Connects to	**every** input value	a small **local** patch (receptive field)
Input	a **flat vector** (spatial layout lost)	an **image** (H × W × C kept)
Weights	different for every position	the **same kernel reused everywhere** (weight sharing)
Good at	combining everything for the final decision	finding local patterns: edges, textures, shapes
Parameters	**huge**	**few**

Let's **count the parameters** (the learnable numbers) for a small 32×32 colour image:

```
def n_params(layer):
    return sum(p.numel() for p in layer.parameters())

fc = nn.Linear(32 * 32 * 3, 100)                    # 3072 inputs -> 100 neurons
cv = nn.Conv2d(3, 100, kernel_size=3, padding=1)    # 100 kernels of size 3x3x3

print(f"Fully connected: {n_params(fc):>9,} parameters  (each neuron sees all 3,072 pixel
values)")
print(f"Convolutional  : {n_params(cv):>9,} parameters  (each kernel is just 3x3x3 = 27 weights +
1 bias)")
print(f"-> the FC layer needs {n_params(fc) / n_params(cv):.0f}x more parameters")

big_fc = 224 * 224 * 3 * 1000
print(f"\nFor a 224x224 photo, an FC layer of 1000 neurons would need {big_fc / 1e6:.0f} million
weights!")
```

```
Fully connected:   307,300 parameters  (each neuron sees all 3,072 pixel values)
Convolutional  :     2,800 parameters  (each kernel is just 3x3x3 = 27 weights + 1 bias)
-> the FC layer needs 110x more parameters

For a 224x224 photo, an FC layer of 1000 neurons would need 151 million weights!
```

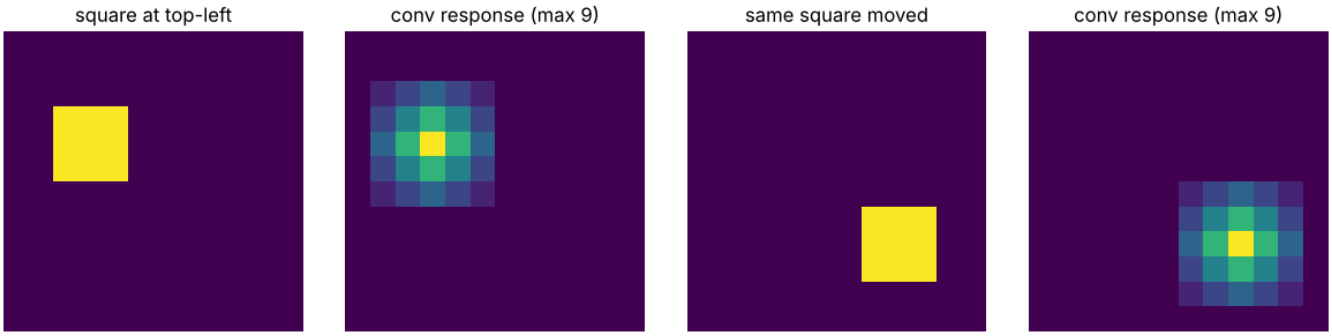
Translation invariance: "find the pattern wherever it is"

Because the **same** kernel slides over every position, a pattern produces the **same response wherever it appears**: move the pattern and the response moves with it. A fully connected layer has separate weights per pixel, so a moved pattern looks like a completely different input.

```
pattern = np.zeros((12, 12), np.float32); pattern[3:6, 2:5] = 1          # a small square, top-left
moved = np.zeros((12, 12), np.float32);   moved[7:10, 7:10] = 1          # same square, bottom-right

with torch.no_grad():
    k = nn.Conv2d(1, 1, 3, padding=1, bias=False)
    k.weight[:] = 1.0                                                     # a simple "blob detector"
    r1 = k(torch.from_numpy(pattern)[None, None])[0, 0].numpy()
    r2 = k(torch.from_numpy(moved)[None, None])[0, 0].numpy()

fig, ax = plt.subplots(1, 4, figsize=(12, 3))
for a, im, t in zip(ax, [pattern, r1, moved, r2],
                    ["square at top-left", f"conv response (max {r1.max():.0f})",
                     "same square moved", f"conv response (max {r2.max():.0f})"]):
    a.imshow(im, cmap="viridis"); a.set_title(t); a.axis("off")
plt.tight_layout(); plt.show()
print("Strongest response is identical:", r1.max() == r2.max(), "- it just moved along with the square.")
```




```
Strongest response is identical: True - it just moved along with the square.
```

In practice CNNs use **convolutional layers for feature extraction** and **fully connected layers at the end for the decision** ("this is a 7"). You'll see that shape in LeNet, AlexNet and VGG in notebook 02.

3 · Activation functions: why ReLU won

Without an activation function, stacking layers is pointless: a linear layer followed by another linear layer is still just one linear layer. **Activations add non-linearity**, so networks can learn curved, complex patterns.

Function	Formula	Output range
Sigmoid	$1 / (1 + e^{-x})$	0 to 1

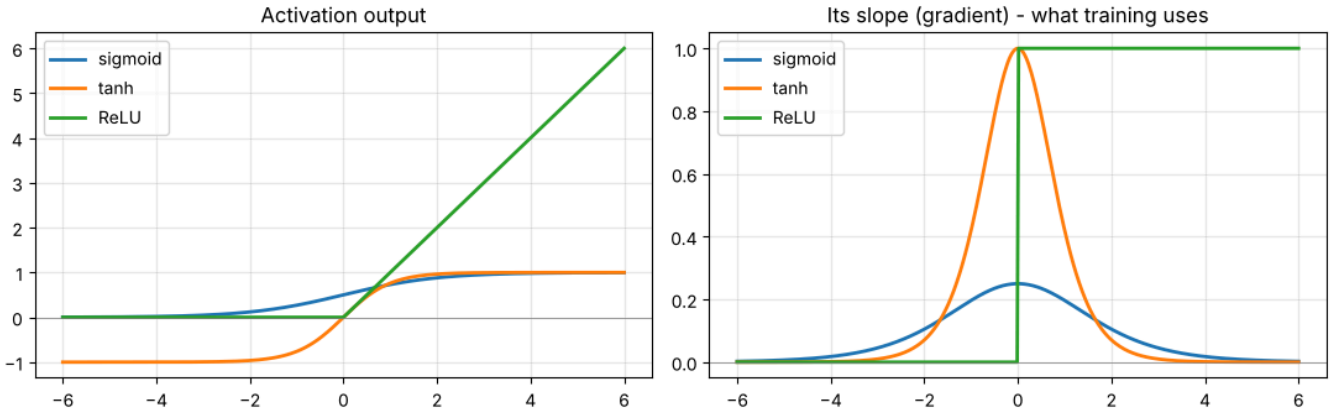
Function	Formula	Output range
Tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	−1 to 1
ReLU	**max(0, x)**	0 to ∞

ReLU: "if the input is positive, keep it; if negative, output zero." It's like a filter that only lets positive signals through.

```
x = torch.linspace(-6, 6, 400, requires_grad=True)
funcs = {"sigmoid": torch.sigmoid, "tanh": torch.tanh, "ReLU": torch.relu}

fig, ax = plt.subplots(1, 2, figsize=(11, 3.6))
for name, f in funcs.items():
    y = f(x)
    grad, = torch.autograd.grad(y.sum(), x)          # the slope (derivative) at every x
    ax[0].plot(x.detach(), y.detach(), label=name, lw=2)
    ax[1].plot(x.detach(), grad, label=name, lw=2)
ax[0].set_title("Activation output"); ax[1].set_title("Its slope (gradient) - what training
uses")
for a in ax:
    a.axhline(0, color="gray", lw=0.5); a.legend(); a.grid(alpha=0.3)
plt.tight_layout(); plt.show()

print("ReLU on [-3, -1, 0, 2, 5] ->", torch.relu(torch.tensor([-3., -1., 0., 2., 5.])).tolist())
```




```
ReLU on [-3, -1, 0, 2, 5] -> [0.0, 0.0, 0.0, 2.0, 5.0]
```

Look at the **right plot**:

- **Sigmoid's slope is never more than 0.25**, and it's almost 0 for big or small inputs ("saturation"). Tanh's slope also dies at the ends.
- **ReLU's slope is exactly 1 for every positive input.**

Why that matters: the vanishing gradient problem

Training works by **backpropagation**. The error signal travels backwards through the layers and gets **multiplied by each layer's slope** on the way. Multiply by ≤ 0.25 again and again and the signal **shrinks to almost nothing**, so the early layers stop learning. With ReLU, positive paths multiply by 1, so the signal survives.

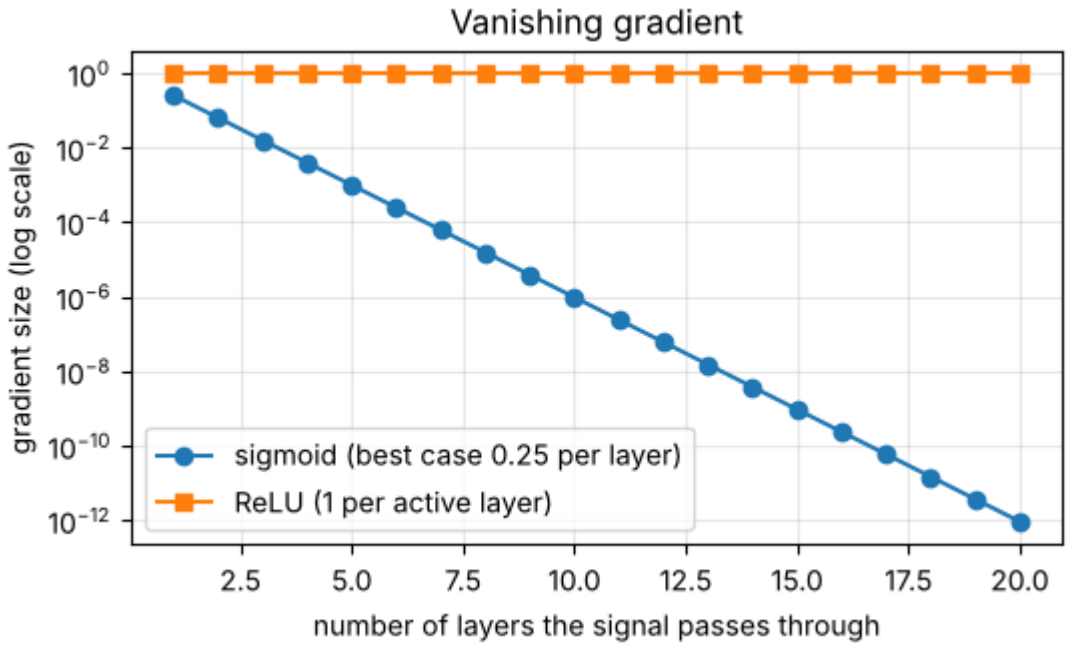
```
layers = np.arange(1, 21)
print("Best-case gradient left after N layers (sigmoid slope <= 0.25, ReLU slope = 1):\n")
for n in [1, 2, 5, 10, 20]:
    print(f"  {n:>2} layers: sigmoid {0.25 ** n:.2e}   ReLU {1.0 ** n:.0f}")

plt.figure(figsize=(6, 3.2))
```

```
plt.semilogy(layers, 0.25 ** layers, "o-", label="sigmoid (best case 0.25 per layer)")
plt.semilogy(layers, np.ones_like(layers, dtype=float), "s-", label="ReLU (1 per active layer)")
plt.xlabel("number of layers the signal passes through"); plt.ylabel("gradient size (log scale)")
plt.title("Vanishing gradient"); plt.legend(); plt.grid(alpha=0.3); plt.show()
```

```
Best-case gradient left after N layers (sigmoid slope <= 0.25, ReLU slope = 1):

   1 layers: sigmoid 2.50e-01   ReLU 1
   2 layers: sigmoid 6.25e-02   ReLU 1
   5 layers: sigmoid 9.77e-04   ReLU 1
  10 layers: sigmoid 9.54e-07   ReLU 1
  20 layers: sigmoid 9.09e-13   ReLU 1
```



After only 10 sigmoid layers the gradient is a **millionth** of its size. This is one of the reasons **AlexNet (2012)** switched to ReLU: it trained about **6× faster** than tanh in the original paper, and deeper networks became practical.

> Exam phrasing: *ReLU introduces non-linearity, is cheap to compute (just max(0, x)), and its gradient is 1 for positive inputs, so it reduces the vanishing-gradient problem.* One caveat: neurons stuck at negative inputs output 0 forever (the "dying ReLU" problem). Variants such as Leaky ReLU fix this.

4 · Dropout: randomly benching neurons during training

Definition: a **regularization** technique (a way to prevent **overfitting**, i.e. memorising the training data instead of learning general rules). **How:** during each **training** step, each neuron is switched off (set to 0) with probability *p*. A different random set is dropped each time. **Analogy:** a team where random members sit out each practice, so everyone has to learn to contribute and nobody becomes a single point of failure.

The Key Concepts example: *a layer has 10 neurons and dropout rate 0.3, so about 3 are ignored in each training pass.*

```
drop = nn.Dropout(p=0.3)
activations = torch.ones(10)                    # 10 neurons, all outputting 1.0

drop.train()                                    # TRAINING mode: dropout is active
```

```
for step in range(3):
    out = drop(activations)
    print(f"training pass {step + 1}: {np.round(out.numpy(), 2)}  -> {int((out == 0).sum())}
dropped")

drop.eval()                                          # EVALUATION / test mode: dropout switched
OFF
print("\ntest time      :", drop(activations).numpy())
```

```
training pass 1: [1.43 1.43 0.   1.43 1.43 1.43 1.43 1.43 1.43 1.43]  -> 1 dropped
training pass 2: [1.43 1.43 0.   1.43 1.43 1.43 1.43 1.43 0.   0.  ]  -> 3 dropped
training pass 3: [1.43 1.43 1.43 0.   0.   1.43 0.   1.43 1.43 1.43]  -> 3 dropped

test time      : [1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

Two details worth knowing:

1. The surviving neurons are **scaled up by 1/(1 − p)** (here 1/0.7 ≈ 1.43). This keeps the *average* signal the same as at test time.
2. Dropout is **only active during training**. `model.train()` turns it on and `model.eval()` turns it off. Forgetting `model.eval()` before testing is a classic bug.

Why it generalises better (Hard Q1): the network can't rely on one specific neuron ("the neuron that detects *this exact* cat photo's ear"), because that neuron may be missing. It has to spread knowledge across many neurons and learn **robust features** that work even with parts missing. That's exactly what's needed on new, unseen images. Below we test it on a deliberately overfitting-prone setup.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

digits = load_digits()                                       # 1,797 handwritten digits, 8x8
pixels, ships with sklearn
X = torch.tensor(digits.data, dtype=torch.float32) / 16.0    # pixel values 0..16 -> 0..1
y = torch.tensor(digits.target)

def train_and_test(p_drop, seed, n_train=60, epochs=500):
    # a BIG network + only 60 training images = a recipe for overfitting
    Xtr, Xte, ytr, yte = train_test_split(X, y, train_size=n_train, random_state=seed,
stratify=y)
    torch.manual_seed(seed)
    model = nn.Sequential(nn.Linear(64, 1024), nn.ReLU(), nn.Dropout(p_drop),
                          nn.Linear(1024, 1024), nn.ReLU(), nn.Dropout(p_drop),
                          nn.Linear(1024, 10))
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    for _ in range(epochs):
        model.train()                                        # dropout ON
        loss = F.cross_entropy(model(Xtr), ytr)
        opt.zero_grad(); loss.backward(); opt.step()
    model.eval()                                             # dropout OFF for testing
    with torch.no_grad():
        train_acc = (model(Xtr).argmax(1) == ytr).float().mean().item()
        test_acc = (model(Xte).argmax(1) == yte).float().mean().item()
    return train_acc, test_acc

# Results vary from run to run, so average 5 runs (different data splits and starting weights)
for p in [0.0, 0.5]:
    runs = [train_and_test(p, seed) for seed in range(5)]
    tr = np.mean([r[0] for r in runs]); te = np.mean([r[1] for r in runs])
    print(f"dropout p={p}: train accuracy {tr:.1%}   test accuracy (unseen) {te:.1%}")
```

```
dropout p=0.0: train accuracy 100.0%   test accuracy (unseen) 86.7%
dropout p=0.5: train accuracy 100.0%   test accuracy (unseen) 87.8%
```

Both versions score **100% on the 60 training images**: they have memorised them. On the ~1,700 **unseen** images, the dropout version does slightly but consistently better. The gap between training and test accuracy *is* overfitting, and dropout narrows it.

The gain here is small (about 1 percentage point) because this problem is tiny. Dropout matters most in **huge** layers. AlexNet's fully connected layers held about 58 million weights, and the paper reports that without dropout the network overfitted badly.

5 · Pooling: shrinking feature maps

CNNs (LeNet, AlexNet, VGG) alternate **conv → ReLU → pool**. **Max pooling 2×2** keeps the biggest value in each 2×2 block:

- it halves the height and width, which means less computation for the next layer,
- it keeps "was the pattern present somewhere around here?", and
- it adds a little tolerance to small shifts.

It has no learnable parameters.

```
fm = torch.tensor([[1., 3, 2, 0],
                   [4, 6, 1, 1],
                   [0, 2, 9, 5],
                   [1, 1, 3, 7]])
print("feature map (4x4):\n", fm.int().numpy())
print("\nafter 2x2 max pooling (2x2):\n", F.max_pool2d(fm[None, None], 2)[0, 0].int().numpy())
print("\nafter 2x2 average pooling:\n", F.avg_pool2d(fm[None, None], 2)[0, 0].numpy())
```

```
feature map (4x4):
 [[1 3 2 0]
 [4 6 1 1]
 [0 2 9 5]
 [1 1 3 7]]

after 2x2 max pooling (2x2):
 [[6 2]
 [2 9]]

after 2x2 average pooling:
 [[3.5 1. ]
 [1.  6. ]]
```

6 · How a network learns: the training loop

Every model in this course is trained with the same 4-step loop, repeated thousands of times:

1. **Forward pass:** push images through the network to get predictions.
2. **Loss:** one number for "how wrong were we?" (cross-entropy for classification).
3. **Backward pass:** `loss.backward()` computes the **gradient**, i.e. which way to nudge every weight to reduce the loss. This is backpropagation.
4. **Step:** the optimizer (SGD or Adam) nudges all the weights a little.

Below is a tiny CNN learning the 8×8 digits. Watch the loss fall, and compare the first layer's kernels before and after training: every number has been adjusted by the loop.

```
Xtr_img, Xte_img, ytr_d, yte_d = train_test_split(X.reshape(-1, 1, 8, 8), y, test_size=0.3,
random_state=0, stratify=y)

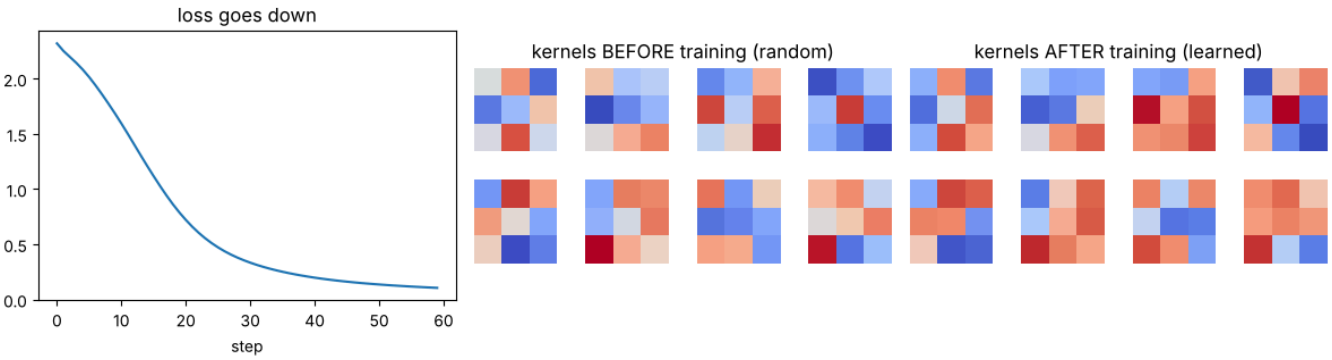
torch.manual_seed(0)
tiny_cnn = nn.Sequential(
    nn.Conv2d(1, 8, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 8 learned 3x3
kernels -> 8 maps of 4x4
    nn.Flatten(), nn.Linear(8 * 4 * 4, 10))                                  # FC layer makes the
decision
first_kernels_before = tiny_cnn[0].weight.detach().clone()

opt = torch.optim.Adam(tiny_cnn.parameters(), lr=0.01)
losses = []
for epoch in range(60):
    logits = tiny_cnn(Xtr_img)                # 1. forward
    loss = F.cross_entropy(logits, ytr_d)     # 2. loss
    opt.zero_grad(); loss.backward()          # 3. backward (gradients)
    opt.step()                                # 4. update weights
    losses.append(loss.item())

with torch.no_grad():
    acc = (tiny_cnn(Xte_img).argmax(1) == yte_d).float().mean().item()
print(f"test accuracy after 60 steps: {acc:.1%}  (random guessing = 10%)")

fig = plt.figure(figsize=(12, 3.4))
ax = fig.add_subplot(1, 3, 1); ax.plot(losses); ax.set_title("loss goes down");
ax.set_xlabel("step")
for row, (w, title) in enumerate([(first_kernels_before, "kernels BEFORE training (random)"),
                                  (tiny_cnn[0].weight.detach(), "kernels AFTER training
(learned)")]):
    sub = fig.add_subplot(1, 3, row + 2); sub.set_title(title); sub.axis("off")
    tiles = np.full((7, 15), np.nan)
    for i in range(8):
        r, c = divmod(i, 4)
        tiles[r * 4:r * 4 + 3, c * 4:c * 4 + 3] = w[i, 0].numpy()
    sub.imshow(tiles, cmap="coolwarm")
plt.tight_layout(); plt.show()
```

```
test accuracy after 60 steps: 96.1%  (random guessing = 10%)
```



The learned kernels changed, but on 8×8 images with only 3×3 kernels they're hard to interpret by eye. Notebook 02 trains LeNet on bigger images, where the learned filters and their feature maps are much easier to read.

That is the whole idea of deep learning in vision: **the same convolution as Week 1, with kernels learned by the 4-step loop.**

✅ Check yourself

1. What does ReLU do to negative inputs? *(sets them to 0; positives pass through unchanged)*

2. Give two reasons ReLU replaced sigmoid/tanh in AlexNet. *(faster and simpler to compute; gradient = 1 for positive inputs, so less vanishing gradient)*
3. What is dropout, and is it used at test time? *(randomly zeroing neurons during training to prevent overfitting; no, it's switched off with* `model.eval()` *)*
4. A layer has 20 neurons and dropout 0.25. How many are dropped per pass, on average? *(5)*
5. Why do conv layers need far fewer parameters than fully connected layers? *(local connections + the same kernel reused across the whole image)*
6. Where are conv layers and FC layers used in a typical CNN? *(conv for feature extraction early; FC at the end for the decision)*

02 · Classic CNN architectures: LeNet → AlexNet → VGG

Covers (Week 3 Key Concepts, "CNN Architectures"): LeNet (1998), AlexNet (2012), VGG (2014). **Questions answered here:** Easy 1–9 · Medium 1–4 · Hard 2, 10.

Year	Model	One-line contribution
1998	**LeNet-5** (Yann LeCun)	**Proof of concept**: CNNs can read handwriting (cheques, zip codes)
2012	**AlexNet** (Krizhevsky, Sutskever, Hinton)	**The breakthrough**: won ImageNet by a huge margin using ReLU, dropout and GPUs
2014	**VGG** (Visual Geometry Group, Oxford)	**Depth + simplicity**: 16–19 layers, only small 3×3 filters
2015	ResNet	made *very* deep networks trainable (notebook 03)
2019	EfficientNet	balanced, efficient scaling (notebook 03)
2020	Vision Transformer	attention instead of convolution (notebook 07)

We'll **build and train LeNet** ourselves (it runs in seconds on a laptop CPU), then **inspect** AlexNet and VGG. Those are too big to train here, but we can build them and look inside.

```
%matplotlib inline
import time
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
print("torch", torch.__version__)
```

```
torch 2.14.1+cu130
```

1 · Data: handwritten digits

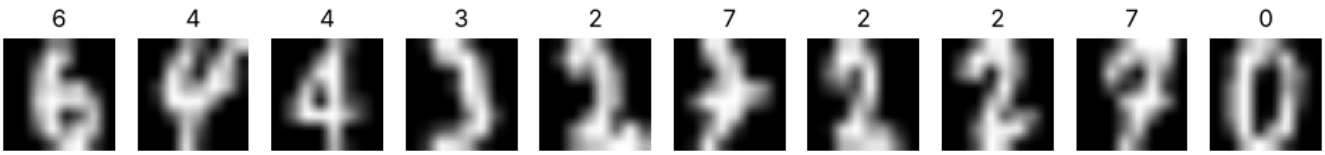
LeNet was trained on **MNIST**: 70,000 handwritten digits, 28×28 pixels, padded to 32×32. MNIST needs a download, so we use scikit-learn's built-in **digits** dataset instead. It has the same idea (handwritten 0–9) but is smaller: 1,797 images of 8×8. We **resize them to 32×32**, LeNet's input size.

```
digits = load_digits()
X = torch.tensor(digits.images, dtype=torch.float32)[:, None] / 16.0      # (1797, 1, 8, 8), values 0..1
X = F.interpolate(X, size=(32, 32), mode="bilinear", align_corners=False)  # -> (1797, 1, 32, 32)
y = torch.tensor(digits.target)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
print("train:", tuple(Xtr.shape), " test:", tuple(Xte.shape))

fig, ax = plt.subplots(1, 10, figsize=(12, 1.6))
```

```
for i in range(10):
    ax[i].imshow(Xtr[i, 0], cmap="gray"); ax[i].set_title(int(ytr[i])); ax[i].axis("off")
plt.show()
```

```
train: (1257, 1, 32, 32)  test: (540, 1, 32, 32)
```



2 · LeNet-5 (1998)

Inventor: Yann LeCun (Bell Labs). **Purpose:** digit recognition. It was used to read **bank cheques and postal codes**. **Architecture: Convolution → Pooling → Convolution → Pooling → Fully Connected → Fully Connected → Output.**

```
input 32×32 — conv 5×5 (6 maps) — pool — conv 5×5 (16 maps) — pool — FC 120 — FC 84 — 10
classes
              "find strokes"            "combine strokes"            "decide which
digit"
```

The original used tanh/sigmoid and average pooling. We use ReLU and max-pooling, the modern defaults, and compare activations later.

```
class LeNet5(nn.Module):
    def __init__(self, act=nn.ReLU):
        super().__init__()
        self.features = nn.Sequential(                # feature extraction (convolutions)
            nn.Conv2d(1, 6, kernel_size=5), act(), nn.MaxPool2d(2),
            nn.Conv2d(6, 16, kernel_size=5), act(), nn.MaxPool2d(2))
        self.classifier = nn.Sequential(              # decision making (fully connected)
            nn.Flatten(),
            nn.Linear(16 * 5 * 5, 120), act(),
            nn.Linear(120, 84), act(),
            nn.Linear(84, 10))

    def forward(self, x):
        return self.classifier(self.features(x))

model = LeNet5()

# Trace one image through the network and print the shape after every layer
x = Xtr[:1]
print(f"{'layer':<34} output shape")
print(f"{'input':<34} {tuple(x.shape)}")
for layer in list(model.features) + list(model.classifier):
    x = layer(x)
    print(f"{str(layer):<34} {tuple(x.shape)}")
print(f"\nTotal parameters: {sum(p.numel() for p in model.parameters()):,}")
```

```
layer                                output shape
input                                (1, 1, 32, 32)
Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1)) (1, 6, 28, 28)
ReLU()                               (1, 6, 28, 28)
MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False) (1, 6, 14, 14)
Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1)) (1, 16, 10, 10)
ReLU()                               (1, 16, 10, 10)
MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False) (1, 16, 5, 5)
Flatten(start_dim=1, end_dim=-1)     (1, 400)
Linear(in_features=400, out_features=120, bias=True) (1, 120)
ReLU()                               (1, 120)
Linear(in_features=120, out_features=84, bias=True) (1, 84)
ReLU()                               (1, 84)
Linear(in_features=84, out_features=10, bias=True) (1, 10)

Total parameters: 61,706
```

Read the shapes:

- **32×32 → 28×28** after a 5×5 convolution with no padding (32 − 5 + 1 = 28, the Week-1 formula).
- **28 → 14** after 2×2 pooling, **14 → 10** after the second 5×5 conv, **10 → 5** after pooling.
- 16 maps × 5 × 5 = **400 numbers** are flattened and sent to the fully connected layers.

The whole of LeNet has only about **62 thousand** parameters. AlexNet has about 61 *million* (see below).

Train it

```python
def train(model, epochs=15, lr=1e-3, batch=64, augment=None, verbose=True):
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    history = []
    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(len(Xtr))
        for i in range(0, len(Xtr), batch):
            idx = perm[i:i + batch]
            xb = Xtr[idx] if augment is None else augment(Xtr[idx], seed=epoch * 10_000 + i)
            loss = F.cross_entropy(model(xb), ytr[idx])
            opt.zero_grad(); loss.backward(); opt.step()
        history.append(accuracy(model, Xte))
        if verbose and (epoch % 5 == 4 or epoch == 0):
            print(f"epoch {epoch + 1:>2}: test accuracy {history[-1]:.1%}")
    return history

def accuracy(model, X_, y_=None):
    y_ = yte if y_ is None else y_
    model.eval()
    with torch.no_grad():
        return (model(X_).argmax(1) == y_).float().mean().item()

torch.manual_seed(0)
lenet = LeNet5()
t = time.time()
hist_relu = train(lenet)
print(f"\nLeNet-5 final test accuracy: {hist_relu[-1]:.1%}  (trained in {time.time() - t:.1f} s
on CPU)")
```

```
epoch  1: test accuracy 57.2%
epoch  5: test accuracy 91.7%
epoch 10: test accuracy 95.4%
epoch 15: test accuracy 95.9%

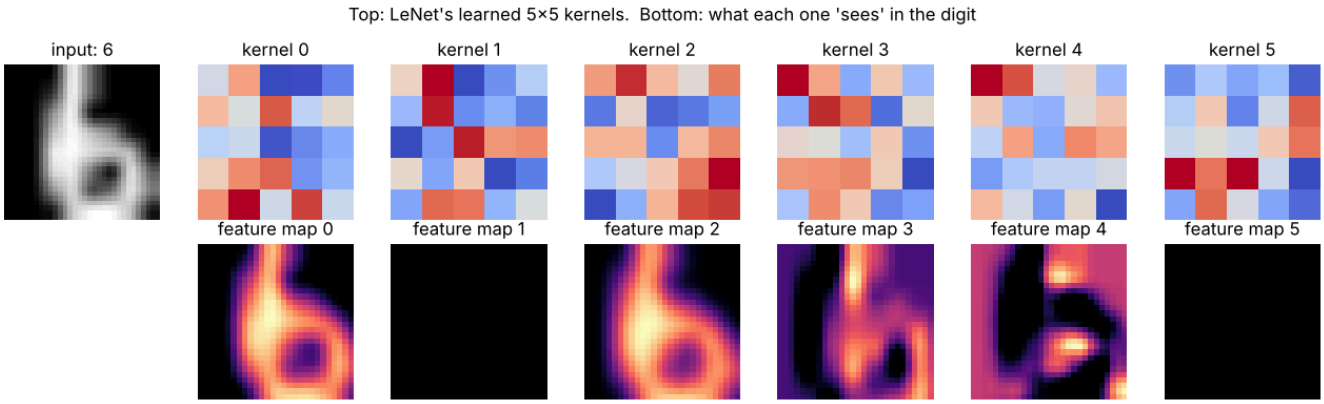
LeNet-5 final test accuracy: 95.9%  (trained in 2.9 s on CPU)
```

What did LeNet learn? Look at its filters and feature maps

The first layer has **6 learned 5×5 kernels**. Applying them to a digit gives 6 **feature maps**. Each one lights up where its pattern (a stroke direction, an edge or a blob) appears.

```
sample = Xte[3:4]
with torch.no_grad():
    maps = F.relu(lenet.features[0](sample))[0]           # output of first conv + ReLU

fig, ax = plt.subplots(2, 7, figsize=(13, 4))
ax[0, 0].imshow(sample[0, 0], cmap="gray"); ax[0, 0].set_title(f"input: {int(yte[3])}")
ax[1, 0].axis("off"); ax[0, 0].axis("off")
for i in range(6):
    ax[0, i + 1].imshow(lenet.features[0].weight[i, 0].detach(), cmap="coolwarm")
    ax[0, i + 1].set_title(f"kernel {i}"); ax[0, i + 1].axis("off")
    ax[1, i + 1].imshow(maps[i], cmap="magma"); ax[1, i + 1].set_title(f"feature map {i}"); ax[1,
i + 1].axis("off")
plt.suptitle("Top: LeNet's learned 5x5 kernels.  Bottom: what each one 'sees' in the digit")
plt.tight_layout(); plt.show()
```



Different kernels respond to different parts of the stroke: some to its edges on one side, some to the whole body of the stroke. A completely black map means that kernel's pattern isn't in this digit at all (ReLU turned every negative response into 0). It may fire on other digits. **Nobody programmed these.** They're Week-1-style edge and stroke detectors that LeNet discovered by itself from the data.

LeNet vs "traditional machine learning" (Medium Q1)

The course's exam answer: *LeNet was the **first proof** that CNNs beat traditional ML methods (like SVMs, support vector machines) on images, which inspired their use in handwriting recognition and banking.*

Let's test it honestly on our small dataset. We'll also test **shifted** digits (moved up to 4 pixels), because CNNs are supposed to be good at "find the pattern wherever it is".

```
from sklearn.svm import SVC

def shift(images, max_shift=4, seed=0):
    # move every image by a random amount (up to max_shift pixels in x and y)
    g = torch.Generator().manual_seed(seed)
    out = torch.empty_like(images)
    for i in range(len(images)):
        dx, dy = torch.randint(-max_shift, max_shift + 1, (2,), generator=g).tolist()
        out[i] = torch.roll(images[i], shifts=(dy, dx), dims=(1, 2))
    return out
```

```
Xte_shifted = shift(Xte)

svm = SVC(gamma="scale").fit(Xtr.reshape(len(Xtr), -1), ytr)      # SVM sees the raw 1,024 pixel
values
svm_clean = svm.score(Xte.reshape(len(Xte), -1), yte)
svm_shift = svm.score(Xte_shifted.reshape(len(Xte), -1), yte)

# Fair comparison: give BOTH models training data with shifted copies (augmentation)
torch.manual_seed(0)
lenet_aug = LeNet5()
train(lenet_aug, epochs=25, augment=shift, verbose=False)          # sees new random shifts every
epoch
X_aug = torch.cat([Xtr] + [shift(Xtr, seed=s) for s in range(1, 4)])   # SVM: original + 3
shifted copies
svm_aug = SVC(gamma="scale").fit(X_aug.reshape(len(X_aug), -1), ytr.repeat(4))

def svm_scores(m):
    return m.score(Xte.reshape(len(Xte), -1), yte), m.score(Xte_shifted.reshape(len(Xte), -1),
yte)

rows = [("SVM (raw pixels)", (svm_clean, svm_shift)),
        ("LeNet-5", (accuracy(lenet, Xte), accuracy(lenet, Xte_shifted))),
        ("SVM + shifted training copies", svm_scores(svm_aug)),
        ("LeNet-5 + shifted training copies", (accuracy(lenet_aug, Xte), accuracy(lenet_aug,
Xte_shifted)))]
print(f"{'model':<36}{'normal test':>12}{'shifted test':>14}")
for name, (a, b) in rows:
    print(f"{name:<36}{a:>12.1%}{b:>14.1%}")
```

```
model                                normal test  shifted test
SVM (raw pixels)                           98.9%         69.3%
LeNet-5                                    95.9%         67.2%
SVM + shifted training copies              99.4%         95.6%
LeNet-5 + shifted training copies          98.3%         93.3%
```

Honest result: on this *tiny* dataset (1,257 training images, originally 8×8 pixels) a modern SVM is **just as good or slightly better** than LeNet. Both collapse on shifted digits (about 70%) until they're trained on shifted examples (augmentation, from Week 1 Day 2), and then both recover to over 90%.

So why do we credit LeNet?

- **In 1998, on MNIST** (60,000 training images), LeNet-5 reached about **0.95% error**. That was on par with the best SVMs of the time, while being **much faster and smaller to run**, which mattered for reading millions of cheques. Boosted LeNet variants did best of all.
- **CNNs learn their own features.** The SVM only works this well because the digits are tiny, centred and clean. As images get bigger and messier (real photos, many classes), hand-made features plus SVMs fall far behind. AlexNet proved that dramatically in 2012.

> **Exam answer (Medium Q1):** LeNet showed that a CNN trained end-to-end on raw pixels could match or beat the best traditional methods (SVMs with hand-crafted features) on handwriting recognition, while learning its own features. That led to real deployments in banking (cheque reading) and postal services.

3 · AlexNet (2012): the breakthrough

Event: won the **ImageNet** competition (ILSVRC 2012: 1.2 million photos, 1,000 classes) by a huge margin.

	Top-5 error	Top-5 accuracy
Best entry *before* deep learning (2011, hand-crafted features)	≈ 26%	≈ 74%
AlexNet (2012)	**≈ 15.3%**	**≈ 85%**

The second-best entry in 2012 scored about 26%, so AlexNet's lead was more than 10 points. The course notes summarise this as "~70% → ~85%+". **Top-5 accuracy** means "the right answer is somewhere in the model's 5 best guesses".

Architecture: 5 convolutional layers + 3 fully connected layers.

```
from torchvision.models import alexnet
anet = alexnet(weights=None)          # build the architecture with random weights (no download)
print(anet)

convs = [m for m in anet.modules() if isinstance(m, nn.Conv2d)]
fcs = [m for m in anet.modules() if isinstance(m, nn.Linear)]
n_conv = sum(p.numel() for m in convs for p in m.parameters())
n_fc = sum(p.numel() for m in fcs for p in m.parameters())
print(f"\n{len(convs)} conv layers + {len(fcs)} fully connected layers")
print(f"parameters: conv {n_conv / 1e6:.1f} M   +   FC {n_fc / 1e6:.1f} M   =   {(n_conv + n_fc)
/ 1e6:.1f} M total")
```

```
AlexNet(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU(inplace=True)
    (10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=9216, out_features=4096, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=4096, out_features=4096, bias=True)
    (5): ReLU(inplace=True)
    (6): Linear(in_features=4096, out_features=1000, bias=True)
  )
)

5 conv layers + 3 fully connected layers
parameters: conv 2.5 M   +   FC 58.6 M   =   61.1 M total
```

Notice: **about 95% of AlexNet's parameters are in the 3 fully connected layers.** That's exactly why it needed **dropout** (you can see `Dropout(p=0.5)` in the classifier above). Those huge layers would otherwise memorise the training set.

AlexNet's three innovations

Innovation	What it did
ReLU activation	much faster training than sigmoid/tanh; less vanishing gradient
Dropout	reduced overfitting in the giant FC layers
GPU training	split across 2 NVIDIA GTX 580 GPUs, it trained in about a week instead of months

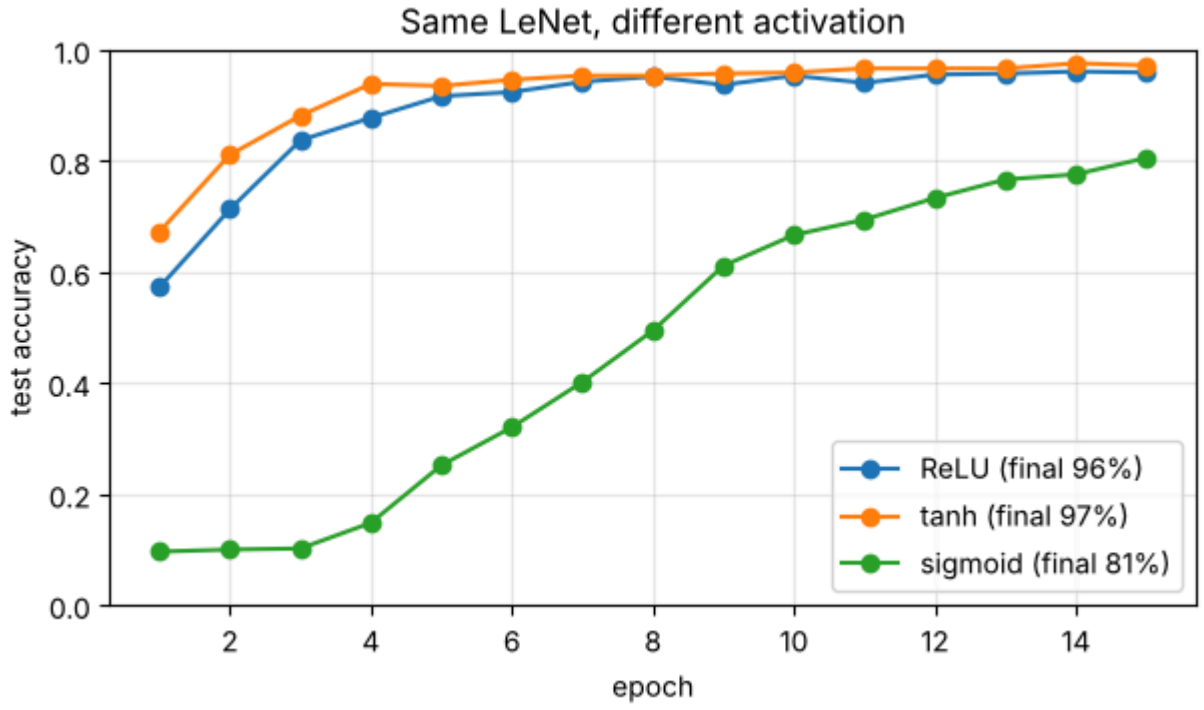
Plus heavy **data augmentation** (random crops and flips, from Day 2) and **overlapping max-pooling**.

ReLU vs sigmoid, measured (Easy Q5, Hard Q2)

Let's train LeNet three times, changing only the activation:

```
results = {"ReLU": hist_relu}
for name, act in [("tanh", nn.Tanh), ("sigmoid", nn.Sigmoid)]:
    torch.manual_seed(0)
    results[name] = train(LeNet5(act=act), verbose=False)

plt.figure(figsize=(7, 3.6))
for name, h in results.items():
    plt.plot(range(1, len(h) + 1), h, "o-", label=f"{name} (final {h[-1]:.0%})")
plt.xlabel("epoch"); plt.ylabel("test accuracy"); plt.ylim(0, 1)
plt.title("Same LeNet, different activation"); plt.legend(); plt.grid(alpha=0.3); plt.show()
```



Sigmoid learns much more slowly. It's stuck near 10% (random guessing) for the first epochs because its small slopes (≤ 0.25) shrink the training signal at every layer. **ReLU and tanh both learn fast here.** This network is shallow (5 weight layers) and the modern Adam optimizer helps, so tanh keeps up, even edging ahead. The gap grows with depth and with the plain SGD optimizer used in 2012. In AlexNet's paper a ReLU network reached the target error **about 6× faster** than the same network with tanh.

> **Why ReLU reduces vanishing gradients (Hard Q2):** the gradient of ReLU is exactly 1 for every positive input, so the error signal passes backwards through many layers without shrinking. Sigmoid and tanh have slopes well below 1 almost everywhere

> and near 0 when saturated, and multiplying many small numbers makes the gradient vanish. ReLU is also cheaper to compute (no exponentials), which made training faster.

Why GPUs mattered (Medium Q2)

Training a CNN is mostly **millions of independent multiply-adds** (every kernel at every position). A CPU has a few powerful cores. A GPU has **thousands of small cores** that do these in parallel. Let's time one AlexNet forward pass on this machine's CPU:

```
batch = torch.randn(16, 3, 224, 224)       # 16 photos of 224x224
anet.eval()
with torch.no_grad():
    t = time.time(); anet(batch); cpu_time = time.time() - t
print(f"AlexNet forward pass, 16 images, CPU: {cpu_time:.2f} s")
print(f"One pass over ImageNet (1.2 M images) would take about {cpu_time / 16 * 1.2e6 / 3600:.1f}
hours on this CPU - "
      "and training needs a backward pass too (about 2-3x more work), repeated ~90 times.")
print("GPU available here:", torch.cuda.is_available(), "- on a GPU the same work is typically
10-50x faster.")
```

```
AlexNet forward pass, 16 images, CPU: 0.08 s
One pass over ImageNet (1.2 M images) would take about 1.8 hours on this CPU - and training needs
a backward pass too (about 2-3x more work), repeated ~90 times.
GPU available here: False - on a GPU the same work is typically 10-50x faster.
```

4 · VGG (2014): deep and simple

VGG (Visual Geometry Group, Oxford). **Architecture:** very deep, **16–19 layers** (VGG-16 and VGG-19). **Design choice:** use **only small 3×3 filters**, stacked. No 11×11 or 5×5 like AlexNet. **Contribution:** showed that **depth improves accuracy**. Its uniform, simple design made it a standard **baseline** in research.

```
from torchvision.models import vgg16, vgg19

for name, fn in [("VGG-16", vgg16), ("VGG-19", vgg19)]:
    net = fn(weights=None)
    n_conv = sum(isinstance(m, nn.Conv2d) for m in net.modules())
    n_fc = sum(isinstance(m, nn.Linear) for m in net.modules())
    kernel_sizes = {m.kernel_size for m in net.modules() if isinstance(m, nn.Conv2d)}
    print(f"{name}: {n_conv} conv + {n_fc} FC = {n_conv + n_fc} weight layers, "
          f"kernel sizes used: {kernel_sizes}, parameters: {sum(p.numel() for p in
net.parameters()) / 1e6:.0f} M")
    del net
```

```
VGG-16: 13 conv + 3 FC = 16 weight layers, kernel sizes used: {(3, 3)}, parameters: 138 M
VGG-19: 16 conv + 3 FC = 19 weight layers, kernel sizes used: {(3, 3)}, parameters: 144 M
```

Why stack small 3×3 filters? (Medium Q4)

1. **Same view, fewer weights.** Two 3×3 layers in a row "see" a 5×5 area of the input, and three of them see 7×7. That's the same **receptive field** as one big 7×7 filter, with fewer parameters.
2. **More non-linearity.** Three layers means three ReLUs instead of one, so the network can learn more complex patterns.

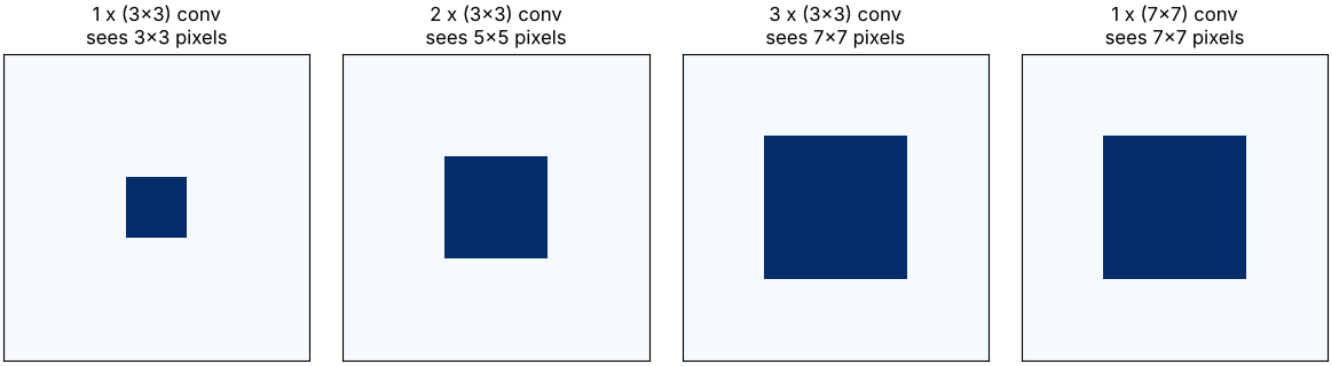
Let's **measure** the receptive field. We ask: "which input pixels can change the centre output pixel?" Backpropagation answers exactly that.

```python
def receptive_field(layers, size=15):
    x = torch.zeros(1, 1, size, size, requires_grad=True)
    out = layers(x)
    out[0, 0, out.shape[2] // 2, out.shape[3] // 2].backward()     # nudge the centre output
    return (x.grad[0, 0] != 0).numpy()                              # which inputs affected it?

def stack(k, n):
    torch.manual_seed(0)
    layers = []
    for _ in range(n):
        conv = nn.Conv2d(1, 1, k, padding=k // 2, bias=False)
        nn.init.constant_(conv.weight, 1.0)
        layers.append(conv)
    return nn.Sequential(*layers)

fig, ax = plt.subplots(1, 4, figsize=(12, 3.2))
for a, (k, n) in zip(ax, [(3, 1), (3, 2), (3, 3), (7, 1)]):
    rf = receptive_field(stack(k, n))
    a.imshow(rf, cmap="Blues"); a.set_xticks([]); a.set_yticks([])
    a.set_title(f"{n} x ({k}x{k}) conv\nsees {rf.sum(1).max()}x{rf.sum(0).max()} pixels")
plt.tight_layout(); plt.show()

C = 64   # channels in and out
print(f"Parameters for C={C} channels in and out:")
print(f"  one 7x7 layer      : 7*7*C*C   = {49 * C * C:,}")
print(f"  three 3x3 layers   : 3*3*3*C*C = {27 * C * C:,}   ({27 / 49:.0%} of the 7x7 cost, with
3 ReLUs instead of 1)")
```




```
Parameters for C=64 channels in and out:
  one 7x7 layer      : 7*7*C*C   = 200,704
  three 3x3 layers   : 3*3*3*C*C = 110,592   (55% of the 7x7 cost, with 3 ReLUs instead of 1)
```

Three stacked 3×3 layers see the same 7×7 window as one 7×7 layer, with **45% fewer parameters** and **three** non-linearities. That's VGG's core idea, and almost every later CNN (ResNet included) uses 3×3 convolutions.

VGG's downside: about 138 million parameters (mostly in the FC layers again). It's slow and memory-hungry. ResNet and EfficientNet (next notebook) fix this.

Key takeaways (from the Key Concepts sheet)

- **LeNet:** proof of concept (conv → pool → FC). Digits, cheques.
- **AlexNet:** breakthrough in scale and performance. ReLU + dropout + GPUs, 5 conv + 3 FC, won ImageNet 2012 by about 10 points.
- **VGG:** depth and simplicity as design principles. 16–19 layers, all 3×3 filters, a research baseline.

- Together they form the **timeline of CNN evolution**, and that trend toward deeper and better-designed networks led to ResNet, EfficientNet and Vision Transformers (Hard Q10, answered in `practice/question_bank_answers.md`).

✅ Check yourself

1. Who invented LeNet, what was it for, and what dataset was it trained on? *(Yann LeCun; handwritten digit recognition for cheques and postal codes; MNIST)*
2. LeNet's layer pattern? *(conv → pool → conv → pool → FC → FC → output)*
3. AlexNet's biggest achievement and three innovations? *(won ImageNet 2012 by a huge margin; ReLU, dropout, GPU training)*
4. How many layers does AlexNet have? *(5 conv + 3 FC)*
5. How many layers does VGG have, and what filter size did it standardise? *(16–19; 3×3)*
6. What receptive field do three stacked 3×3 convs have? *(7×7)*
7. Why is most of AlexNet's/VGG's parameter count in the FC layers? *(every FC neuron connects to every input value; conv kernels are tiny and shared)*

03 · Modern CNNs: ResNet and EfficientNet

Covers (Week 3 Key Concepts, "Why Study Modern CNNs?"): ResNet (2015), EfficientNet (2019), plus MobileNet for "lightweight models for phones". **Questions answered here:** Easy 10–14 · Medium 5–9 · Hard 3–4, 7.

The story so far: VGG showed that deeper means more accurate. So why not 100 layers? People tried, and **very deep plain networks got *worse***, even on their *training* data. Two problems stood in the way:

1. **Depth:** gradients vanish, so the early layers stop learning. **ResNet** fixed this.
2. **Efficiency:** bigger networks are slow and wasteful if you grow them the wrong way. **EfficientNet** fixed this.

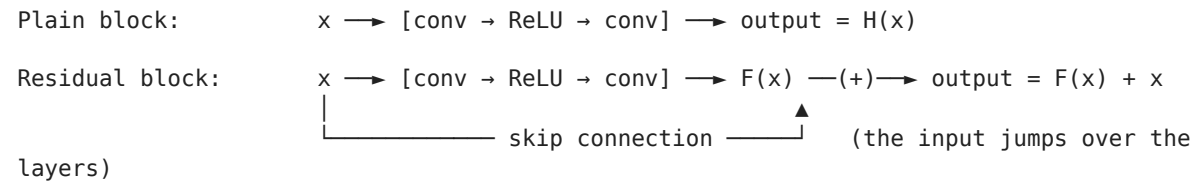
```
%matplotlib inline
import time
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
digits = load_digits()
X = torch.tensor(digits.images, dtype=torch.float32)[:, None] / 16.0
X = F.interpolate(X, size=(16, 16), mode="bilinear", align_corners=False)
y = torch.tensor(digits.target)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
print("data:", tuple(Xtr.shape), tuple(Xte.shape))
```

```
data: (1257, 1, 16, 16) (540, 1, 16, 16)
```

1 · ResNet (2015): the residual connection

The idea in one picture



- A **plain** block must learn the whole desired output **H(x)** (a "direct mapping").
- A **residual** block only learns the **difference** (the *residual*) **F(x) = H(x) − x**, i.e. what to *change* about its input. The input is then added back.

Analogy: instead of redrawing a picture from scratch at every layer, each layer only makes **corrections** to the picture it received. If a layer has nothing useful to add, it can learn F(x) ≈ 0 and simply pass the input through unchanged. That's very easy. For a plain block, learning to "do nothing" (H(x) = x) is surprisingly hard.

Why it fixes vanishing gradients

For output = F(x) + x, the gradient flowing backwards is:

d(output)/dx = F′(x) + 1

That **"+1"** is a direct highway for the gradient. Even if F′(x) is tiny, the signal still passes back through the skip connection unchanged. In a plain network the gradient is a **product** of many F′ terms, and the product of many small numbers vanishes.

Let's build both kinds of block and **prove it**.

```
class Block(nn.Module):
    def __init__(self, channels, residual):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, 3, padding=1)
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)
        self.residual = residual

    def forward(self, x):
        out = self.conv2(F.relu(self.conv1(x)))      # F(x)
        if self.residual:
            out = out + x                            # the skip connection: F(x) + x
        return F.relu(out)

class Net(nn.Module):
    def __init__(self, n_blocks, residual, channels=32):
        super().__init__()
        self.stem = nn.Conv2d(1, channels, 3, padding=1)
        self.blocks = nn.Sequential(*[Block(channels, residual) for _ in range(n_blocks)])
        self.head = nn.Linear(channels, 10)

    def forward(self, x):
        x = F.relu(self.stem(x))
        x = self.blocks(x)
        return self.head(x.mean(dim=(2, 3)))         # global average pooling -> classifier

def first_layer_gradient(model):
    model.zero_grad()
    F.cross_entropy(model(Xtr[:128]), ytr[:128]).backward()
    return model.stem.weight.grad.norm().item()     # how big is the learning signal reaching
layer 1?

print(f"{'layers':>6} | {'plain net: gradient at layer 1':>32} | {'residual net: gradient at
layer 1':>34}")
for n_blocks in [2, 10, 25]:
    torch.manual_seed(0); g_plain = first_layer_gradient(Net(n_blocks, residual=False))
    torch.manual_seed(0); g_res = first_layer_gradient(Net(n_blocks, residual=True))
    print(f"{2 * n_blocks + 2:>6} | {g_plain:>32.2e} | {g_res:>34.2e}")
```

```
layers |   plain net: gradient at layer 1 |  residual net: gradient at layer 1
     6 |                         3.00e-04 |                             1.81e-02
    22 |                         1.92e-10 |                             1.00e-02
    52 |                         0.00e+00 |                             2.04e-02
```

Read the left column: the deeper the plain network, the smaller the learning signal that reaches the first layer. At 22 layers it's around 10^{-10}, and at 52 layers it's **exactly zero**: the first layer gets no information at all about how to improve. The residual network keeps a healthy signal at every depth.

Does it actually change training? (≈ 3 minutes on a laptop CPU)

```
def train_eval(model, epochs=15, lr=3e-3, batch=64):
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    for _ in range(epochs):
        model.train()
        perm = torch.randperm(len(Xtr))
        for i in range(0, len(Xtr), batch):
            idx = perm[i:i + batch]
            loss = F.cross_entropy(model(Xtr[idx]), ytr[idx])
            opt.zero_grad(); loss.backward(); opt.step()
    model.eval()
    with torch.no_grad():
        return F.cross_entropy(model(Xtr), ytr).item(), (model(Xte).argmax(1) ==
yte).float().mean().item()

results = {}
t = time.time()
for n_blocks in [2, 10, 25]:
    for residual in [False, True]:
        torch.manual_seed(0)
        results[(2 * n_blocks + 2, residual)] = train_eval(Net(n_blocks, residual))
print(f"done in {time.time() - t:.0f} s\n")
print(f"{'layers':>6} | {'plain: train loss / test acc':>30} | {'residual: train loss / test
acc':>32}")
for L in [6, 22, 52]:
    (lp, ap), (lr_, ar) = results[(L, False)], results[(L, True)]
    print(f"{L:>6} | {lp:>15.3f} / {ap:>9.1%} | {lr_:>17.3f} / {ar:>9.1%}")
```

```
done in 214 s

layers |   plain: train loss / test acc |  residual: train loss / test acc
     6 |           0.687 /     78.1% |            0.291 /     88.0%
    22 |           2.303 /     10.2% |            0.587 /     76.1%
    52 |           2.303 /     10.2% |            0.816 /     66.1%
```

- At **6 layers** both train, and the residual one is already better.
- At **22 and 52 layers** the plain network is **stuck at 10%**, which is random guessing among 10 digits. Its training loss stays at 2.30 = ln(10), the loss of a model that knows nothing. **It cannot even fit its training data.** This is the *degradation problem* the ResNet paper described.
- The **residual** networks of the same depth train fine.

Our deep residual nets score lower than the 6-layer one only because they need more epochs and BatchNorm. Real ResNets add BatchNorm after every convolution, and that plus skip connections let them train with **50, 101, even 152 layers**.

> **Exam answer (Hard Q4):** In a plain deep network the gradient reaching early layers is a product of many per-layer factors, so it shrinks toward zero (vanishing gradients) and adding layers makes even *training* accuracy worse (degradation). A residual block outputs F(x) + x, so the gradient is F′(x) + 1. The identity path carries the gradient back unchanged, and a block that isn't useful can learn F(x) ≈ 0 and pass its input through. Extra depth therefore never hurts, and networks with 100+ layers become trainable.

Real ResNets

```
from torchvision.models import resnet18, resnet34, resnet50, resnet101, resnet152

print(f"{'model':<10}{'weight layers':>15}{'parameters':>14}")
for fn in [resnet18, resnet34, resnet50, resnet101, resnet152]:
    net = fn(weights=None)
```

```
    layers = sum(isinstance(m, (nn.Conv2d, nn.Linear)) for m in net.modules())
    downsample = sum(isinstance(m, nn.Conv2d) and m.kernel_size == (1, 1) and "downsample" in
name
                     for name, m in net.named_modules())
    params = sum(p.numel() for p in net.parameters())
    print(f"{fn.__name__:<10}{layers - downsample:>15}{params / 1e6:>12.1f} M")
    del net
```

```
model       weight layers    parameters
resnet18               18        11.7 M
resnet34               34        21.8 M
resnet50               50        25.6 M
resnet101             101        44.5 M
resnet152             152        60.2 M
```

The number in the name is the count of weight layers on the main path. The four 1×1 "downsample" convolutions on skip connections, which match sizes when the shape changes, are not counted. **ResNet-152 has 152 layers but fewer parameters (60 M) than VGG-16 (138 M)**, thanks to "bottleneck" blocks (1×1 → 3×3 → 1×1 convs) and **no giant fully connected layers**, just global average pooling and one FC layer.

Achievement: ResNet won ImageNet 2015 with **3.57% top-5 error**, better than the estimated human error of about 5%. **Real-world use (Medium Q8):** ResNet is the default backbone in **medical imaging**, e.g. classifying chest X-rays or skin lesions, often **pretrained on ImageNet and fine-tuned** on a smaller medical dataset. It's also the backbone inside Faster R-CNN and Mask R-CNN (notebooks 04–05).

Skip connections in ResNet vs U-Net (preview of Hard Q7)

	ResNet	U-Net (notebook 05)
Connects	a block's input to its output (short jumps)	encoder layer → matching decoder layer (long jumps)
Combines by	**adding** (F(x) + x)	**concatenating** feature maps
Main purpose	**gradient flow**: train very deep networks	**restore fine spatial detail** lost by downsampling

2 · EfficientNet (2019): scaling the smart way

Problem solved: to get a more accurate CNN, people scaled it up in **one** direction:

- **depth**: more layers (ResNet-18 → ResNet-152),
- **width**: more channels/filters per layer,
- **resolution**: bigger input images (224 → 600 pixels).

Scaling just one of these quickly hits **diminishing returns**: lots more computation for a tiny accuracy gain. It's like a bigger bike with only a bigger front wheel.

Innovation: compound scaling. Scale **all three together** with one formula, controlled by a single number φ ("phi", how much more compute you can afford):

depth × **α^φ**, width × **β^φ**, resolution × **γ^φ**, with **$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$**

EfficientNet uses **α = 1.2, β = 1.1, γ = 1.15**. Each +1 in φ then roughly **doubles the computation** (FLOPs). Width and resolution are squared because doubling either one makes the work per layer about 4× bigger.

```
alpha, beta, gamma = 1.2, 1.1, 1.15
print(f"alpha * beta^2 * gamma^2 = {alpha * beta**2 * gamma**2:.3f}  (~2 -> each step doubles compute)\n")
print(f"{'phi':>3} | {'depth x':>8} | {'width x':>8} | {'resolution':>10} | {'compute x':>9}")
for phi in range(0, 8):
    d, w, r = alpha ** phi, beta ** phi, gamma ** phi
    print(f"{phi:>3} | {d:>8.2f} | {w:>8.2f} | {224 * r:>8.0f}px | {d * w**2 * r**2:>9.1f}")
```

```
alpha * beta^2 * gamma^2 = 1.920  (~2 -> each step doubles compute)

phi |  depth x |  width x | resolution | compute x
  0 |     1.00 |     1.00 |      224px |       1.0
  1 |     1.20 |     1.10 |      258px |       1.9
  2 |     1.44 |     1.21 |      296px |       3.7
  3 |     1.73 |     1.33 |      341px |       7.1
  4 |     2.07 |     1.46 |      392px |      13.6
  5 |     2.49 |     1.61 |      451px |      26.1
  6 |     2.99 |     1.77 |      518px |      50.1
  7 |     3.58 |     1.95 |      596px |      96.3
```

Every step grows depth, width *and* resolution a bit. The released models **B0 (φ = 0) to B7** follow this recipe, with input sizes from 224 up to 600 pixels.

Why is this better than just going deeper or wider? (Medium Q7, Hard Q3) The three dimensions **depend on each other**. A higher-resolution image has more detail, so the network needs **more layers** (a larger receptive field to cover bigger objects) **and more channels** (to capture the extra fine-grained patterns). Growing only one dimension leaves the others as the bottleneck. Balanced growth uses each extra FLOP where it helps most. The constants α, β and γ were found by a small grid search on the base model, then reused for all sizes.

The payoff: accuracy per parameter

```
from torchvision.models import efficientnet_b0, efficientnet_b7, resnet50, mobilenet_v3_large

def count(fn):
    net = fn(weights=None)
    n = sum(p.numel() for p in net.parameters())
    del net
    return n / 1e6

# ImageNet top-1 accuracies as reported in the papers (we can't train these here)
table = [("ResNet-50", count(resnet50), 76.0),
         ("EfficientNet-B0", count(efficientnet_b0), 77.1),
         ("EfficientNet-B7", count(efficientnet_b7), 84.3),
         ("MobileNetV3-Large", count(mobilenet_v3_large), 75.2)]
print(f"{'model':<20}{'parameters':>12}{'ImageNet top-1':>16}")
for name, p, acc in table:
    print(f"{name:<20}{p:>10.1f} M{acc:>15.1f}%")
```

```
model                parameters  ImageNet top-1
ResNet-50              25.6 M           76.0%
EfficientNet-B0         5.3 M           77.1%
EfficientNet-B7        66.3 M           84.3%
MobileNetV3-Large       5.5 M           75.2%
```

EfficientNet-B0 beats ResNet-50 with about 5× fewer parameters. In the paper, EfficientNet-B7 matched the best model of the time (84.3%) while being **8.4× smaller** and much faster.

Real-world use (Medium Q9): on-device **mobile apps**, e.g. a phone app that recognises plant diseases or food from the camera, and anything else needing good accuracy on limited hardware (drones, embedded cameras).

3 · Lightweight models for phones: MobileNet (Easy Q14)

The course's answer for "lightweight yet accurate for mobile vision apps" is **EfficientNet**. Its smallest versions (B0) are tiny. Its building block comes from **MobileNet**, the other classic mobile architecture, which uses **depthwise-separable convolutions**:

- **Normal conv:** every filter looks at all input channels at once, giving K × K × C_in × C_out weights.
- **Depthwise-separable:** ① one K×K filter **per channel** (depthwise), then ② a **1×1 conv** to mix the channels (pointwise), giving K × K × C_in + C_in × C_out weights.

```
C_in, C_out, K = 128, 128, 3
normal = nn.Conv2d(C_in, C_out, K, padding=1, bias=False)
separable = nn.Sequential(nn.Conv2d(C_in, C_in, K, padding=1, groups=C_in, bias=False),   # depthwise
                          nn.Conv2d(C_in, C_out, 1, bias=False))                          # pointwise 1x1
n1 = sum(p.numel() for p in normal.parameters())
n2 = sum(p.numel() for p in separable.parameters())
x = torch.randn(1, C_in, 56, 56)
print("same output shape:", normal(x).shape == separable(x).shape)
print(f"normal 3x3 conv     : {n1:>7,} weights")
print(f"depthwise-separable : {n2:>7,} weights  -> {n1 / n2:.1f}x fewer (and about that much less computation)")
```

```
same output shape: True
normal 3x3 conv     : 147,456 weights
depthwise-separable :  17,536 weights  -> 8.4x fewer (and about that much less computation)
```

Key takeaways (Key Concepts sheet)

- **ResNet:** made very deep networks trainable. **Learn residuals instead of direct mappings**; skip connections let gradients flow. 50/101/152 layers, state of the art on ImageNet 2015. Used in **medical imaging**.
- **EfficientNet:** made scaling efficient and practical. **Compound scaling** of depth, width and resolution with one formula: high accuracy with fewer parameters. Used in **mobile vision apps**.
- Together they represent the **modern era of CNNs** (solving depth and efficiency), and their ideas (skip connections, careful scaling) carried straight into **Vision Transformers**, which use residual connections in every block (notebook 07).

✅ Check yourself

1. What problem does ResNet solve? *(vanishing gradients / degradation when training very deep networks)*

2. ResNet's key innovation? *(residual (skip) connections: output = F(x) + x)*
3. Direct mapping vs residual? *(a plain block learns H(x) itself; a residual block learns only the change F(x) = H(x) − x)*
4. Why does the gradient survive in ResNet? *(d/dx [F(x) + x] = F′(x) + 1; the +1 path carries it backwards)*
5. Which architecture made 152-layer networks trainable? *(ResNet)*
6. What does EfficientNet's compound scaling balance? *(depth, width and resolution, all together via φ)*
7. Lightweight but accurate for phones? *(EfficientNet (B0) / MobileNet)*

04 · Object detection: R-CNN, YOLO, SSD

Covers (Week 3 Key Concepts, "Object Detection Architectures"): R-CNN (2014), YOLO (2016), SSD (2016) and the **accuracy vs speed** trade-off. **Questions answered here:** Easy 15–18 · Medium 10–14 · Hard 5–6.

> **Object detection = identifying *what* each object is AND *where* it is**: a **class label + a bounding box** for every object in the image.

Task	Output	Example
Classification	one label for the whole image	"cat"
Localization	one label + one box	"cat, at (40, 30, 120, 110)"
Detection	**a label + box for EVERY object**	"cat here, dog there, person there"
Segmentation (notebook 05)	a label for every **pixel**	exact outline of each object

In this notebook we **build tiny versions of R-CNN and YOLO ourselves** on a synthetic "shapes" dataset and race them. Both train on a laptop CPU in about 2 minutes.

```
%matplotlib inline
import time
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import cv2
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision.ops import nms, box_iou, roi_align

torch.manual_seed(0)
print("torch", torch.__version__)
```

```
torch 2.14.1+cu130
```

1 · Two tools every detector uses: IoU and NMS

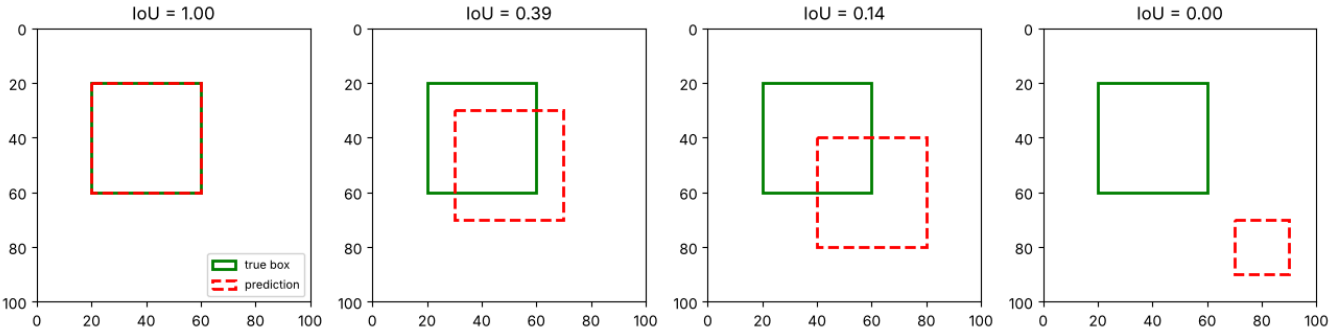
IoU (Intersection over Union): "how much do two boxes overlap?"

IoU = area of overlap / area of union: 1 = identical boxes, 0 = no overlap.

It's used to (a) decide whether a predicted box **counts as correct** (usually IoU ≥ 0.5 with the true box) and (b) remove duplicate boxes (NMS, below). Boxes are written as **(x1, y1, x2, y2)**: top-left and bottom-right corners.

```
def iou(a, b):
    x1, y1 = max(a[0], b[0]), max(a[1], b[1])          # overlap rectangle: the inner corners
    x2, y2 = min(a[2], b[2]), min(a[3], b[3])
    inter = max(0, x2 - x1) * max(0, y2 - y1)
    area_a = (a[2] - a[0]) * (a[3] - a[1])
    area_b = (b[2] - b[0]) * (b[3] - b[1])
    return inter / (area_a + area_b - inter)            # union = A + B - overlap (don't count it twice)
```

```
truth = (20, 20, 60, 60)
examples = [(20, 20, 60, 60), (30, 30, 70, 70), (40, 40, 80, 80), (70, 70, 90, 90)]
fig, ax = plt.subplots(1, 4, figsize=(13, 3.3))
for a, pred in zip(ax, examples):
    a.add_patch(mpatches.Rectangle(truth[:2], 40, 40, fill=False, ec="green", lw=2, label="true
box"))
    a.add_patch(mpatches.Rectangle(pred[:2], pred[2] - pred[0], pred[3] - pred[1], fill=False,
ec="red", lw=2, ls="--", label="prediction"))
    a.set_xlim(0, 100); a.set_ylim(100, 0); a.set_aspect("equal")
    a.set_title(f"IoU = {iou(truth, pred):.2f}")
ax[0].legend(loc="lower right", fontsize=8)
plt.tight_layout(); plt.show()
print("check with torchvision:", box_iou(torch.tensor([truth], dtype=torch.float32),
torch.tensor(examples, dtype=torch.float32)).numpy().round(2))
```




```
check with torchvision: [[1.   0.39 0.14 0.  ]]
```

NMS (Non-Maximum Suppression): "keep the best box, delete its duplicates"

Detectors usually fire **several overlapping boxes on the same object**. NMS cleans this up:

1. Sort boxes by confidence score.
2. Keep the highest-scoring box and **delete every other box that overlaps it** with IoU > threshold (e.g. 0.5).
3. Repeat with the next remaining box.

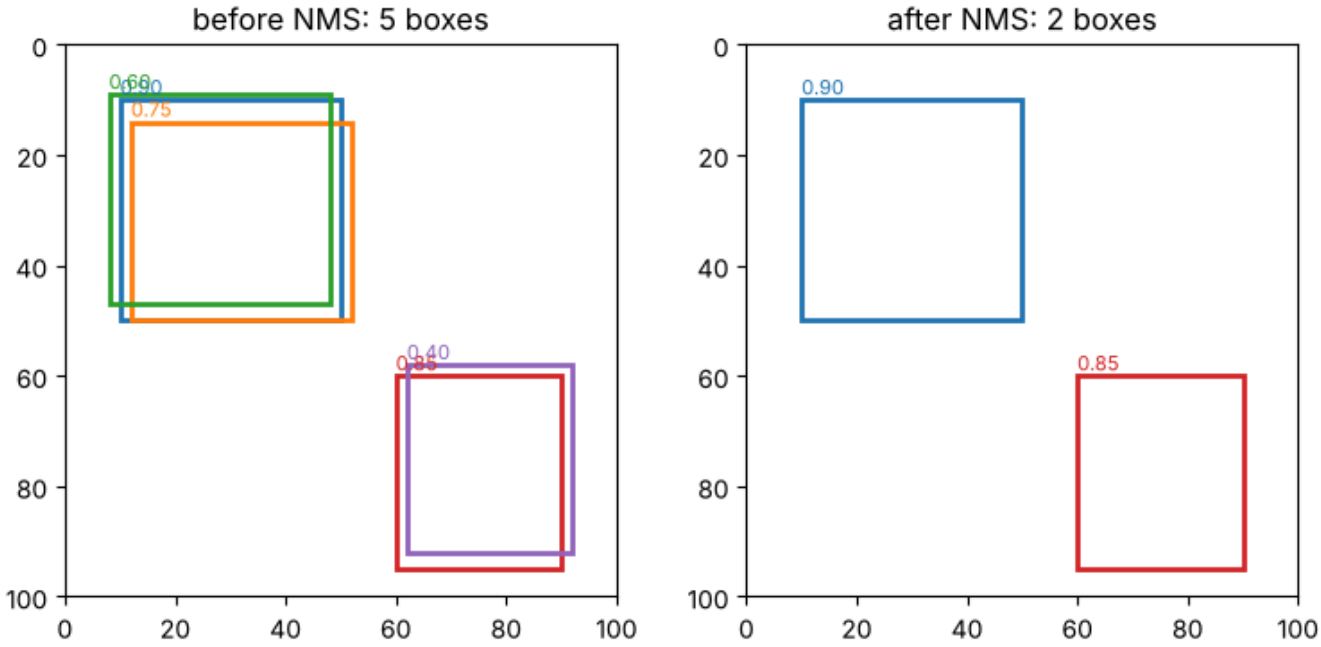
```
def my_nms(boxes, scores, thresh=0.5):
    order = list(np.argsort(scores)[::-1])          # best first
    keep = []
    while order:
        best = order.pop(0)
        keep.append(best)
        order = [i for i in order if iou(boxes[best], boxes[i]) <= thresh]   # drop the
duplicates of `best`
    return keep

boxes = [(10, 10, 50, 50), (12, 14, 52, 50), (8, 9, 48, 47), (60, 60, 90, 95), (62, 58, 92, 92)]
scores = [0.90, 0.75, 0.60, 0.85, 0.40]
kept = my_nms(boxes, scores)
print("kept boxes (mine)       :", sorted(kept))
print("kept boxes (torchvision):", sorted(nms(torch.tensor(boxes, dtype=torch.float32),
torch.tensor(scores), 0.5).tolist()))

fig, ax = plt.subplots(1, 2, figsize=(9, 4))
for a, idxs, title in [(ax[0], range(len(boxes)), "before NMS: 5 boxes"), (ax[1], kept, f"after
NMS: {len(kept)} boxes")]:
    for i in idxs:
        b = boxes[i]
        a.add_patch(mpatches.Rectangle(b[:2], b[2] - b[0], b[3] - b[1], fill=False, ec="C%d" % i,
lw=2))
        a.text(b[0], b[1] - 1, f"{scores[i]:.2f}", color="C%d" % i, fontsize=8)
```

```
    a.set_xlim(0, 100); a.set_ylim(100, 0); a.set_aspect("equal"); a.set_title(title)
plt.show()
```

```
kept boxes (mine)       : [np.int64(0), np.int64(3)]
kept boxes (torchvision): [0, 3]
```



2 · A dataset we can train on in minutes

Real detectors train on COCO (118,000 photos, 80 classes) for days on GPUs. We make our own: **96×96 scenes with 1–3 objects** (circle, square or triangle) of different sizes on a noisy background. We know the exact boxes because we drew them.

```
CLASSES = ["circle", "square", "triangle"]
IMG = 96

def draw_shape(img, cls, cx, cy, r, brightness):
    if cls == 0:
        cv2.circle(img, (cx, cy), r, brightness, -1)
    elif cls == 1:
        cv2.rectangle(img, (cx - r, cy - r), (cx + r, cy + r), brightness, -1)
    else:
        cv2.fillPoly(img, [np.array([[cx, cy - r], [cx - r, cy + r], [cx + r, cy + r]],
np.int32)], brightness)

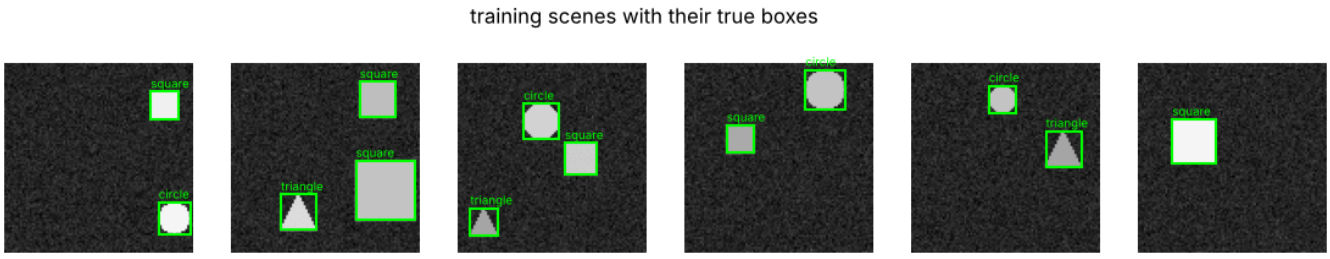
def make_scene(rng, max_objects=3):
    img = rng.normal(40, 12, (IMG, IMG)).clip(0, 255).astype(np.uint8)       # noisy dark
background
    boxes, labels = [], []
    for _ in range(rng.integers(1, max_objects + 1)):
        for _attempt in range(20):                                           # find a spot that
doesn't overlap
            r = int(rng.integers(6, 16)); cx = int(rng.integers(r + 1, IMG - r - 1)); cy =
int(rng.integers(r + 1, IMG - r - 1))
            box = [cx - r, cy - r, cx + r, cy + r]
            if all(iou(box, b) == 0 for b in boxes):
                cls = int(rng.integers(0, 3))
                draw_shape(img, cls, cx, cy, r, int(rng.integers(150, 255)))
                boxes.append(box); labels.append(cls)
                break
    return img, np.array(boxes, np.float32), np.array(labels)

rng = np.random.default_rng(0)
train_scenes = [make_scene(rng) for _ in range(3000)]
```

```
test_scenes = [make_scene(rng) for _ in range(200)]

def show_scene(ax, img, boxes, labels, scores=None, color="lime"):
    ax.imshow(img, cmap="gray", vmin=0, vmax=255); ax.axis("off")
    for k, ((x1, y1, x2, y2), c) in enumerate(zip(boxes, labels)):
        ax.add_patch(mpatches.Rectangle((x1, y1), x2 - x1, y2 - y1, fill=False, ec=color,
lw=1.5))
        txt = CLASSES[int(c)] + (f" {scores[k]:.2f}" if scores is not None else "")
        ax.text(x1, y1 - 2, txt, color=color, fontsize=7)

fig, ax = plt.subplots(1, 6, figsize=(14, 2.6))
for a, (img, b, l) in zip(ax, train_scenes[:6]):
    show_scene(a, img, b, l)
plt.suptitle("training scenes with their true boxes"); plt.show()
```




```
def evaluate(all_predictions, scenes=test_scenes):
    # A prediction is correct if IoU >= 0.5 with a true box of the SAME class (each true box used
once)
    tp = fp = fn = 0
    for preds, (img, boxes, labels) in zip(all_predictions, scenes):
        gt = torch.tensor(boxes); used = set()
        for box, score, cls in preds:
            ious = box_iou(torch.tensor([box], dtype=torch.float32), gt)[0]
            k = int(ious.argmax())
            if ious[k] >= 0.5 and k not in used and labels[k] == cls:
                tp += 1; used.add(k)
            else:
                fp += 1
        fn += len(boxes) - len(used)
    precision = tp / max(tp + fp, 1)      # of the boxes we drew, how many were right?
    recall = tp / max(tp + fn, 1)         # of the real objects, how many did we find?
    return precision, recall
```

3 · R-CNN (2014): "propose regions, then classify each one"

Full name: Region-based Convolutional Neural Network (Girshick et al.). **Process:**

1. **Generate region proposals**: about 2,000 "maybe there's an object here" boxes per image (R-CNN used the *selective search* algorithm).
2. **Run a CNN on each region** separately: crop, resize, classify.
3. **Classify and refine** the bounding boxes, then remove duplicates with NMS.

Strength: high accuracy. **Weakness:** very slow, because the CNN runs **thousands of times per image**. **Key contribution:** the first major CNN-based detector. It proved deep learning works for detection.

Our version: proposals = **sliding windows** of 6 sizes (simpler than selective search), and the CNN is a small classifier for 4 classes: *background, circle, square, triangle*.

```python
def make_proposals():
    out = []
    for size in [12, 16, 20, 24, 28, 32]:                 # windows of several sizes...
        for y in range(0, IMG - size + 1, 4):             # ...slid across the image every 4
pixels
            for x in range(0, IMG - size + 1, 4):
                out.append([x, y, x + size, y + size])
    return torch.tensor(out, dtype=torch.float32)

PROPOSALS = make_proposals()
print("region proposals per image:", len(PROPOSALS))

CROP = 24
def crop_regions(img, boxes):
    # cut out every box and resize it to 24x24 (roi_align does crop+resize for many boxes at
once)
    t = torch.tensor(img, dtype=torch.float32)[None, None] / 255
    return roi_align(t, [boxes], output_size=(CROP, CROP), aligned=True)

# Training crops: true boxes (slightly jittered) = object classes 1..3;
# background = random windows (IoU < 0.3) + "hard negatives" that cover only PART of an object
(IoU 0.1-0.45)
rng2 = np.random.default_rng(1)
crops, crop_labels = [], []
for img, boxes, labels in train_scenes[:1500]:
    gt = torch.tensor(boxes)
    jittered = gt + torch.tensor(rng2.integers(-2, 3, gt.shape), dtype=torch.float32)
    crops.append(crop_regions(img, jittered)); crop_labels += [int(l) + 1 for l in labels]
    best_iou = box_iou(PROPOSALS, gt).max(1).values
    background = PROPOSALS[best_iou < 0.3][torch.tensor(rng2.choice(int((best_iou < 0.3).sum()),
3, replace=False))]
    partial = PROPOSALS[(best_iou > 0.1) & (best_iou < 0.45)]
    if len(partial) >= 3:
        background = torch.cat([background, partial[torch.tensor(rng2.choice(len(partial), 3,
replace=False))]])
    crops.append(crop_regions(img, background)); crop_labels += [0] * len(background)
crops = torch.cat(crops); crop_labels = torch.tensor(crop_labels)
print("training crops:", tuple(crops.shape), " class counts (bg, circle, square, triangle):",
torch.bincount(crop_labels).tolist())

torch.manual_seed(0)
region_classifier = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Flatten(), nn.Linear(32 * 6 * 6, 64), nn.ReLU(), nn.Linear(64, 4))
opt = torch.optim.Adam(region_classifier.parameters(), lr=2e-3)
t = time.time()
for epoch in range(8):
    perm = torch.randperm(len(crops))
    for i in range(0, len(crops), 128):
        idx = perm[i:i + 128]
        loss = F.cross_entropy(region_classifier(crops[idx]), crop_labels[idx])
        opt.zero_grad(); loss.backward(); opt.step()
region_classifier.eval()
print(f"region classifier trained in {time.time() - t:.0f} s")
```

```
region proposals per image: 2299
training crops: (11993, 1, 24, 24)  class counts (bg, circle, square, triangle): [9000, 1015, 98
5, 993]
region classifier trained in 9 s
```

```python
def rcnn_detect(img, score_threshold=0.9):
    with torch.no_grad():
        probs = F.softmax(region_classifier(crop_regions(img, PROPOSALS)), dim=1)   # CNN on
EVERY proposal
    score, cls = probs[:, 1:].max(1)                      # best non-background class per proposal
    keep = score > score_threshold
    boxes, score, cls = PROPOSALS[keep], score[keep], cls[keep]
    final = nms(boxes, score, 0.3)                        # remove duplicates
```

```
    return [(boxes[i].tolist(), score[i].item(), cls[i].item()) for i in final]

t = time.time()
rcnn_preds = [rcnn_detect(img) for img, _, _ in test_scenes]
rcnn_ms = (time.time() - t) / len(test_scenes) * 1000
rcnn_p, rcnn_r = evaluate(rcnn_preds)
print(f"R-CNN style: precision {rcnn_p:.1%}, recall {rcnn_r:.1%}, {rcnn_ms:.0f} ms per image "
      f"({len(PROPOSALS)} CNN runs per image)")
```

```
R-CNN style: precision 72.2%, recall 91.9%, 196 ms per image (2299 CNN runs per image)
```

4 · YOLO (2016): "You Only Look Once"

Process: treat detection as **one regression problem**. Split the image into an **S×S grid** (we use 6×6, so each cell is 16×16 pixels). In **one forward pass**, every cell predicts:

- **objectness**: "is the centre of an object in me?" (0–1),
- **box**: x and y offset inside the cell, width, height,
- **class probabilities**: circle / square / triangle.

So the network's output is a 6 × 6 × (1 + 4 + 3) block of numbers, the whole answer at once.

Strength: real-time speed. **Weakness:** less accurate for **small objects**, especially small objects close together, because each cell predicts only a limited number of objects. **Key contribution:** made detection practical for real-time uses like self-driving cars.

```
S = 6
CELL = IMG // S

def encode_target(boxes, labels):
    # turn the true boxes into the 6x6x8 grid of numbers the network should output
    t = torch.zeros(S, S, 5 + len(CLASSES))
    for (x1, y1, x2, y2), c in zip(boxes.tolist(), labels):
        cx, cy, w, h = (x1 + x2) / 2, (y1 + y2) / 2, x2 - x1, y2 - y1
        i, j = int(cy // CELL), int(cx // CELL)          # the cell that contains the object's
CENTRE
        t[i, j, 0] = 1                                   # objectness
        t[i, j, 1:5] = torch.tensor([cx / CELL - j, cy / CELL - i, w / IMG, h / IMG])
        t[i, j, 5 + c] = 1                               # one-hot class
    return t

Xtr = torch.tensor(np.stack([s[0] for s in train_scenes]), dtype=torch.float32)[:, None] / 255
Ytr = torch.stack([encode_target(s[1], s[2]) for s in train_scenes])
Xte = torch.tensor(np.stack([s[0] for s in test_scenes]), dtype=torch.float32)[:, None] / 255

class TinyYOLO(nn.Module):
    def __init__(self):
        super().__init__()
        def block(i, o):
            return nn.Sequential(nn.Conv2d(i, o, 3, padding=1), nn.BatchNorm2d(o),
nn.LeakyReLU(0.1))
        self.net = nn.Sequential(
            block(1, 16), nn.MaxPool2d(2),       # 96 -> 48
            block(16, 32), nn.MaxPool2d(2),      # 48 -> 24
            block(32, 64), nn.MaxPool2d(2),      # 24 -> 12
            block(64, 64), nn.MaxPool2d(2),      # 12 -> 6   (one value per grid cell)
            block(64, 128), nn.Conv2d(128, 5 + len(CLASSES), 1))
    def forward(self, x):
        return self.net(x).permute(0, 2, 3, 1)  # (batch, 6, 6, 8)

def yolo_loss(pred, target):
    has_obj = target[..., 0] == 1
    objectness = F.binary_cross_entropy_with_logits(pred[..., 0], target[..., 0])      # every
```

```
cell
    box = F.mse_loss(torch.sigmoid(pred[..., 1:5][has_obj]), target[..., 1:5][has_obj])  # only
cells with objects
    cls = F.cross_entropy(pred[..., 5:][has_obj], target[..., 5:][has_obj].argmax(-1))
    return 5 * objectness + 5 * box + cls

torch.manual_seed(0)
yolo = TinyYOLO()
opt = torch.optim.Adam(yolo.parameters(), lr=2e-3)
t = time.time()
for epoch in range(12):
    yolo.train()
    perm = torch.randperm(len(Xtr))
    for i in range(0, len(Xtr), 64):
        idx = perm[i:i + 64]
        loss = yolo_loss(yolo(Xtr[idx]), Ytr[idx])
        opt.zero_grad(); loss.backward(); opt.step()
yolo.eval()
print(f"TinyYOLO trained in {time.time() - t:.0f} s")
```

```
TinyYOLO trained in 83 s
```

```
def yolo_decode(pred, threshold=0.5):
    conf = torch.sigmoid(pred[..., 0])
    out = []
    for i in range(S):
        for j in range(S):
            if conf[i, j] > threshold:
                ox, oy, w, h = torch.sigmoid(pred[i, j, 1:5]).tolist()
                cx, cy, w, h = (j + ox) * CELL, (i + oy) * CELL, w * IMG, h * IMG
                out.append(([cx - w / 2, cy - h / 2, cx + w / 2, cy + h / 2], conf[i, j].item(),
pred[i, j, 5:].argmax().item()))
    return out

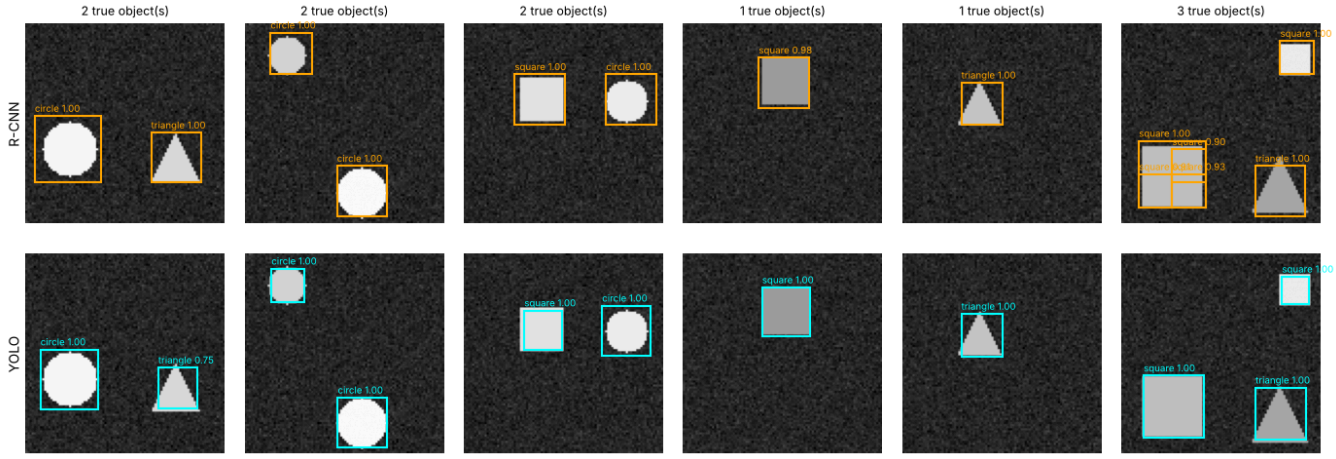
t = time.time()
with torch.no_grad():
    raw = yolo(Xte)                                          # ONE forward pass for all 200 images
yolo_ms = (time.time() - t) / len(Xte) * 1000
yolo_preds = [yolo_decode(raw[k]) for k in range(len(Xte))]
yolo_p, yolo_r = evaluate(yolo_preds)

print(f"{'':<14}{'precision':>10}{'recall':>9}{'ms / image':>12}")
print(f"{'R-CNN style':<14}{rcnn_p:>10.1%}{rcnn_r:>9.1%}{rcnn_ms:>12.1f}")
print(f"{'YOLO style':<14}{yolo_p:>10.1%}{yolo_r:>9.1%}{yolo_ms:>12.1f}")
print(f"\nYOLO is about {rcnn_ms / yolo_ms:.0f}x faster: 1 network run per image instead of
{len(PROPOSALS)}.")

fig, ax = plt.subplots(2, 6, figsize=(14, 5.2))
for k in range(6):
    img, b, l = test_scenes[k]
    for row, preds, name in [(0, rcnn_preds[k], "R-CNN style"), (1, yolo_preds[k], "YOLO
style")]:
        show_scene(ax[row, k], img, [p[0] for p in preds], [p[2] for p in preds], [p[1] for p in
preds],
                   color="orange" if row == 0 else "cyan")
    ax[0, k].set_title(f"{len(b)} true object(s)", fontsize=9)
ax[0, 0].text(-10, 48, "R-CNN", rotation=90, va="center"); ax[1, 0].text(-10, 48, "YOLO",
rotation=90, va="center")
plt.tight_layout(); plt.show()
```

```
               precision    recall  ms / image
R-CNN style        72.2%     91.9%       195.9
YOLO style         99.5%     99.0%         1.6

YOLO is about 126x faster: 1 network run per image instead of 2299.
```



The speed gap is the whole point. Our R-CNN runs its CNN about 2,300 times per image, while YOLO runs once. That's the **"why is R-CNN slow compared to YOLO?" (Medium Q10)** answer in one number.

Careful with the accuracy column: our *toy* R-CNN looks *less* accurate than toy-YOLO only because it's crude. Its boxes are stuck on a 4-pixel sliding grid and it has **no box-refinement step**. The real R-CNN used much better proposals (selective search), a large ImageNet-pretrained CNN and a **bounding-box regressor**. For years, R-CNN-family models (Fast/Faster R-CNN) were **more accurate** than YOLO, especially on small objects and cluttered scenes. The trade-off the course wants you to know:

	R-CNN	YOLO	SSD
Full name	Region-based CNN	You Only Look Once	Single Shot MultiBox Detector
Idea	propose regions → CNN on each	one pass, grid predicts boxes + classes	one pass, predictions from **several feature-map scales**
Speed	very slow (thousands of regions)	**fastest**, real-time	fast
Accuracy	**highest** (esp. small/complex)	lower on **small objects**	competitive; good across sizes
Best for	offline, accuracy-critical (**medical**)	real-time (**driving**, video)	balanced, **mobile**

How the R-CNN family got faster

- **Fast R-CNN (2015):** run the CNN **once** on the whole image, then crop each proposal from the *feature map* (RoI Pooling) instead of from the pixels.
- **Faster R-CNN (2015):** replace selective search with a small **Region Proposal Network (RPN)** that shares the same features. Mask R-CNN (notebook 05) builds on this.

5 · SSD (2016): Single Shot MultiBox Detector

Process: one pass, like YOLO, but predictions come from **multiple feature maps at different scales**:

- **early, large** feature maps (fine grid, small cells) → detect **small** objects,
- **late, small** feature maps (coarse grid, big cells) → detect **large** objects.

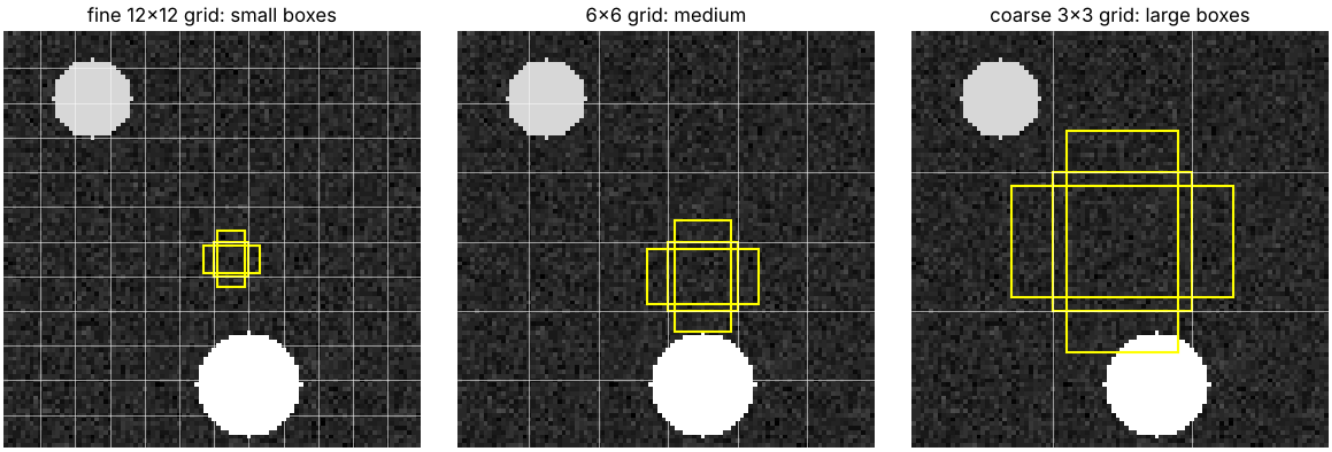
At every cell of every map, SSD checks a few **default boxes** ("anchors") of different shapes and predicts a class and a box correction for each.

Strengths: faster than R-CNN, competitive accuracy, handles **small and large objects in the same image** (Medium Q12). **Weakness:** slightly less accurate than R-CNN on very complex scenes.

```
# The six feature maps of SSD300 (input 300x300) and how many default boxes each contributes
ssd_maps = [(38, 4), (19, 6), (10, 6), (5, 6), (3, 4), (1, 4)]   # (grid size, boxes per cell)
total = 0
print(f"{'feature map':>12} {'cell size':>10} {'boxes':>7}   detects")
for size, k in ssd_maps:
    n = size * size * k; total += n
    kind = "small objects" if size >= 19 else ("medium objects" if size >= 5 else "large
objects")
    print(f"{size:>8}x{size:<3} {300 / size:>8.0f}px {n:>7}   {kind}")
print(f"{'total':>12} {'':>10} {total:>7} default boxes checked in ONE pass")

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
img, b, l = test_scenes[1]
for a, (grid, title) in zip(ax, [(12, "fine 12x12 grid: small boxes"), (6, "6x6 grid: medium"),
(3, "coarse 3x3 grid: large boxes")]):
    a.imshow(img, cmap="gray"); a.set_title(title); a.axis("off")
    cell = IMG / grid
    for g in range(1, grid):
        a.axhline(g * cell, color="white", lw=0.4); a.axvline(g * cell, color="white", lw=0.4)
    cx = cy = (grid // 2 + 0.5) * cell                        # draw the default boxes of one cell
    for w, h in [(cell, cell), (cell * 1.6, cell * 0.8), (cell * 0.8, cell * 1.6)]:
        a.add_patch(mpatches.Rectangle((cx - w / 2, cy - h / 2), w, h, fill=False, ec="yellow",
lw=1.5))
plt.tight_layout(); plt.show()
```

```
  feature map   cell size    boxes   detects
       38x38         8px     5776   small objects
       19x19        16px     2166   small objects
       10x10        30px      600   medium objects
        5x5         60px      150   medium objects
        3x3        100px       36   large objects
        1x1        300px        4   large objects
        total                8732 default boxes checked in ONE pass
```



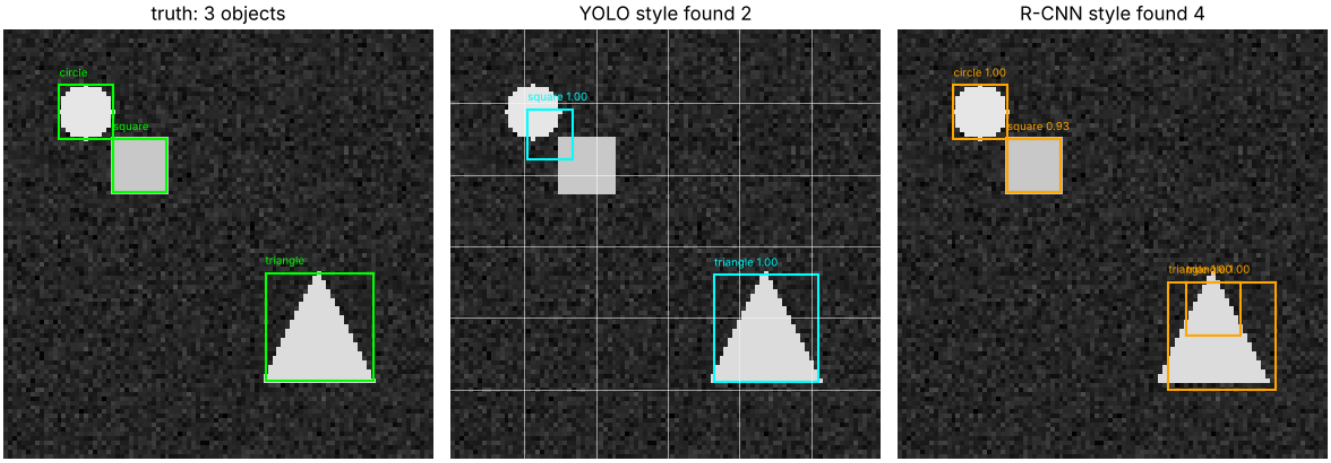
Our TinyYOLO uses a single 6×6 grid, and each cell predicts **one** object (the one whose centre falls in it). Our random training scenes never put two centres in the same cell. But in real photos, small objects in a crowd do: a flock of birds, people far away, distant cars. Let's build such a

scene by hand. A small circle and a small square are placed so that **both centres fall in the same 16×16 cell** (row 1, column 1), with a normal triangle elsewhere for comparison.

```
crowded = np.random.default_rng(5).normal(40, 12, (IMG, IMG)).clip(0, 255).astype(np.uint8)
draw_shape(crowded, 0, 18, 18, 6, 230)      # small circle, centre (18, 18) -> cell (1, 1)
draw_shape(crowded, 1, 30, 30, 6, 200)      # small square, centre (30, 30) -> ALSO cell (1, 1)
draw_shape(crowded, 2, 70, 66, 12, 220)     # a normal triangle elsewhere
crowded_boxes = np.array([[12, 12, 24, 24], [24, 24, 36, 36], [58, 54, 82, 78]], np.float32)
crowded_labels = np.array([0, 1, 2])

with torch.no_grad():
    yolo_found = yolo_decode(yolo(torch.tensor(crowded, dtype=torch.float32)[None, None] / 255)
[0])
rcnn_found = rcnn_detect(crowded)

fig, ax = plt.subplots(1, 3, figsize=(12, 4.2))
show_scene(ax[0], crowded, crowded_boxes, crowded_labels); ax[0].set_title("truth: 3 objects")
show_scene(ax[1], crowded, [p[0] for p in yolo_found], [p[2] for p in yolo_found], [p[1] for p in
yolo_found], color="cyan")
ax[1].set_title(f"YOLO style found {len(yolo_found)}")
show_scene(ax[2], crowded, [p[0] for p in rcnn_found], [p[2] for p in rcnn_found], [p[1] for p in
rcnn_found], color="orange")
ax[2].set_title(f"R-CNN style found {len(rcnn_found)}")
for g in range(1, S):                                  # draw YOLO's grid on the middle panel
    ax[1].axhline(g * CELL, color="white", lw=0.4); ax[1].axvline(g * CELL, color="white",
lw=0.4)
plt.tight_layout(); plt.show()
print("YOLO-style precision/recall on this scene  :", [round(v, 2) for v in
evaluate([yolo_found], [(crowded, crowded_boxes, crowded_labels)])])
print("R-CNN-style precision/recall on this scene :", [round(v, 2) for v in
evaluate([rcnn_found], [(crowded, crowded_boxes, crowded_labels)])])
```




```
YOLO-style precision/recall on this scene  : [0.5, 0.33]
R-CNN-style precision/recall on this scene : [0.75, 1.0]
```

What happened: both small shapes have their centres in the **same grid cell**, and that cell can output only one box. TinyYOLO produces a single, badly placed box somewhere between the circle and the square, and misses one object completely. The R-CNN style detector checks thousands of independent windows, so it finds **both** small shapes, at over 100× the cost. (It also leaves a duplicate box on the triangle, a reminder that its partial-window problem never fully goes away.)

This is YOLO's "**less accurate for small objects**" weakness in miniature. The original YOLO used a 7×7 grid with only 2 boxes per cell, so flocks of birds or distant crowds lost objects exactly like this.

How would you adapt YOLO for very small objects? (Hard Q6)

- **Higher input resolution** and/or a **finer grid** (more, smaller cells).
- **Multi-scale prediction heads**, as SSD does (later YOLO versions, v3 onwards, predict at 3 scales using a Feature Pyramid Network).
- **More boxes per cell / anchors sized for small objects** (choose anchor sizes by clustering the training boxes).
- **Tiling**: cut a large image into overlapping tiles, detect in each and merge (common for drone and satellite images).
- **Data augmentation** (mosaic, copy-paste of small objects) and giving small objects more weight in the loss.

6 · Which architecture for which job? (Medium Q13–14)

	Medical imaging	Autonomous driving
Priority	**Accuracy > speed**	**Speed > accuracy**
Choice	**R-CNN family** (Faster R-CNN; Mask R-CNN for exact outlines)	**YOLO** (or SSD)
Why	missing a tumour (a false negative) is unacceptable; small lesions need precise localization; analysis is offline, so a few seconds per scan is fine	must detect pedestrians, vehicles and signs **in real time** (30+ frames/second); a slightly less precise box *now* beats a perfect box too late
Example	finding small tumours in MRI scans	spotting a pedestrian stepping onto the road

Key insight: the choice depends on the application's **tolerance for errors** versus its **need for real-time performance**.

7 · Optional: try a real pretrained detector on your own photo

Our toy models only know circles, squares and triangles. Real detectors trained on COCO know 80 classes (person, car, dog…). This cell downloads a pretrained **Faster R-CNN** (~160 MB) the first time, so set `RUN = True` on your own machine with internet.

```
RUN = False          # set to True to try it (downloads weights on first run)
IMAGE_PATH = "../../week-1/code/sample.jpg"

if RUN:
    from torchvision.models.detection import fasterrcnn_resnet50_fpn,
FasterRCNN_ResNet50_FPN_Weights
    weights = FasterRCNN_ResNet50_FPN_Weights.DEFAULT
    detector = fasterrcnn_resnet50_fpn(weights=weights).eval()
    photo = cv2.cvtColor(cv2.imread(IMAGE_PATH), cv2.COLOR_BGR2RGB)
    with torch.no_grad():
        out = detector([torch.tensor(photo).permute(2, 0, 1).float() / 255])[0]
    plt.figure(figsize=(10, 6)); plt.imshow(photo); plt.axis("off")
    for box, label, score in zip(out["boxes"], out["labels"], out["scores"]):
        if score > 0.6:
            x1, y1, x2, y2 = box.tolist()
            plt.gca().add_patch(mpatches.Rectangle((x1, y1), x2 - x1, y2 - y1, fill=False,
```

```
ec="red", lw=2))
            plt.text(x1, y1, f"{weights.meta['categories'][label]} {score:.2f}", color="white",
backgroundcolor="red", fontsize=8)
    plt.show()
else:
    print("Skipped. Set RUN = True to download and run a pretrained Faster R-CNN on your own
image.")
```

```
Skipped. Set RUN = True to download and run a pretrained Faster R-CNN on your own image.
```

✅ Check yourself

1. What does object detection identify? *(what each object is (class) and where it is (bounding box))*
2. What do R-CNN, YOLO and SSD stand for? *(Region-based CNN; You Only Look Once; Single Shot MultiBox Detector)*
3. Why is R-CNN slow? *(it runs a CNN separately on ~2,000 region proposals per image)*
4. How does YOLO treat detection differently? *(as one regression problem: a grid predicts boxes and classes in a single forward pass)*
5. Why is SSD good with different object sizes? *(it predicts from several feature maps: fine maps for small objects, coarse maps for large ones)*
6. What do IoU and NMS do? *(IoU measures box overlap; NMS keeps the best box and removes overlapping duplicates)*
7. Medical imaging vs self-driving: which detector and why? *(R-CNN family for accuracy; YOLO/SSD for real-time speed)*

05 · Segmentation: FCN, U-Net and Mask R-CNN

Covers (Week 3 Key Concepts): "Semantic Segmentation Architectures" (FCN, U-Net, encoder, decoder, skip connections) and "Mask R-CNN" (RPN, RoI Align, three outputs). **Questions answered here:** Easy 19–20 · Medium 15–20 · Hard 7–9 · Mask R-CNN 1–8.

> **Semantic segmentation = give a class label to *every pixel* in an image.**

	Output	Two touching cats →
Detection (notebook 04)	boxes + labels	two boxes
Semantic segmentation	a class for every pixel	one "cat" region (it doesn't separate individuals)
Instance segmentation (Mask R-CNN)	a separate mask for every object	cat #1 mask and cat #2 mask

The central challenge: segmentation needs both **context** (*what* is this? which needs a big view of the image) and **precision** (*exactly where* is the boundary? which needs fine pixel detail). Classification only needs the first.

```
%matplotlib inline
import time
import numpy as np
import matplotlib.pyplot as plt
import cv2
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision.ops import roi_pool, roi_align

torch.manual_seed(0)
print("torch", torch.__version__)
```

```
torch 2.14.1+cu130
```

1 · A segmentation dataset: every pixel has a label

64×64 noisy images with circles, squares and triangles. The **mask** labels each pixel: 0 = background, 1 = circle, 2 = square, 3 = triangle. We get it for free by drawing each shape twice: once into the image and once (with its class number) into the mask.

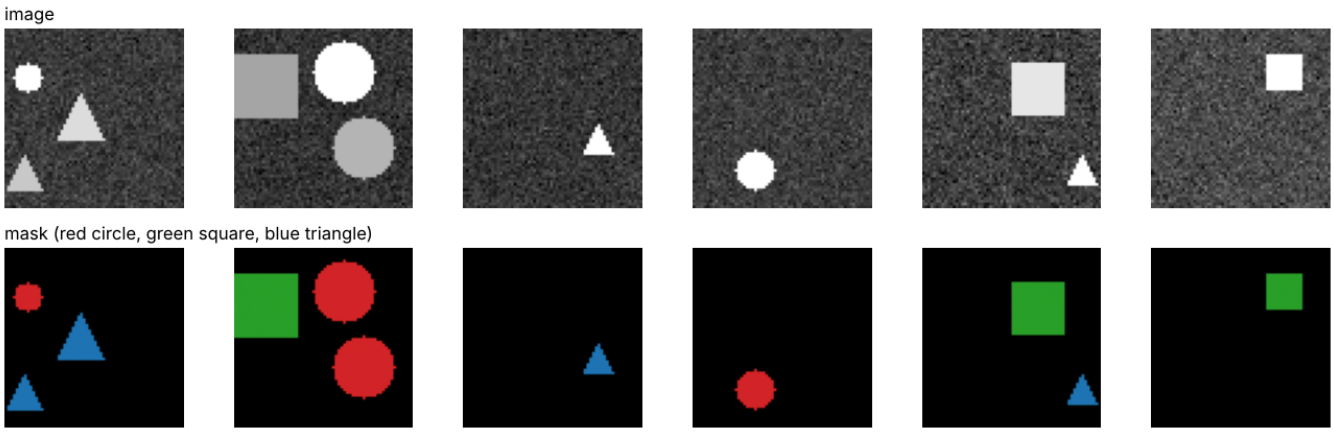
```
IMG = 64
NAMES = ["background", "circle", "square", "triangle"]

def draw(img, mask, cls, cx, cy, r, brightness):
    for target, value in [(img, brightness), (mask, cls + 1)]:
        if cls == 0:
            cv2.circle(target, (cx, cy), r, value, -1)
        elif cls == 1:
            cv2.rectangle(target, (cx - r, cy - r), (cx + r, cy + r), value, -1)
        else:
            cv2.fillPoly(target, [np.array([[cx, cy - r], [cx - r, cy + r], [cx + r, cy + r]],
np.int32)], value)
```

```
def make_sample(rng):
    img = rng.normal(60, 15, (IMG, IMG)).clip(0, 255).astype(np.uint8)
    mask = np.zeros((IMG, IMG), np.uint8)
    for _ in range(rng.integers(1, 4)):
        r = int(rng.integers(5, 14)); cx = int(rng.integers(r, IMG - r)); cy =
int(rng.integers(r, IMG - r))
        draw(img, mask, int(rng.integers(0, 3)), cx, cy, r, int(rng.integers(140, 255)))
    return img, mask

rng = np.random.default_rng(0)
train = [make_sample(rng) for _ in range(2000)]
test = [make_sample(rng) for _ in range(200)]
Xtr = torch.tensor(np.stack([a for a, _ in train]), dtype=torch.float32)[:, None] / 255
Ytr = torch.tensor(np.stack([b for _, b in train])).long()
Xte = torch.tensor(np.stack([a for a, _ in test]), dtype=torch.float32)[:, None] / 255
Yte = torch.tensor(np.stack([b for _, b in test])).long()

mask_cmap = plt.matplotlib.colors.ListedColormap(["black", "tab:red", "tab:green", "tab:blue"])
fig, ax = plt.subplots(2, 6, figsize=(13, 4.4))
for k in range(6):
    ax[0, k].imshow(train[k][0], cmap="gray"); ax[0, k].axis("off")
    ax[1, k].imshow(train[k][1], cmap=mask_cmap, vmin=0, vmax=3); ax[1, k].axis("off")
ax[0, 0].set_title("image", loc="left"); ax[1, 0].set_title("mask (red circle, green square, blue
triangle)", loc="left")
plt.tight_layout(); plt.show()
```



2 · FCN: Fully Convolutional Network (2015)

Problem solved: classic CNNs end with **fully connected layers**. These flatten the feature maps into a vector, which **throws away *where* things are**, and they only accept one fixed input size. **Innovation: replace the fully connected layers with convolutional layers.** Everything stays a 2-D map, so the output is a **dense, pixel-wise prediction map** (one class score per pixel), for any input size. **Key idea:** end-to-end segmentation using convolutional layers only.

```
def conv_block(c_in, c_out):
    return nn.Sequential(nn.Conv2d(c_in, c_out, 3, padding=1), nn.BatchNorm2d(c_out), nn.ReLU(),
                         nn.Conv2d(c_out, c_out, 3, padding=1), nn.BatchNorm2d(c_out), nn.ReLU())

class FCN(nn.Module):
    # encoder shrinks 64 -> 8 (x1/8); a 1x1 conv gives class scores; then simply stretch back x8
    def __init__(self, n_classes=4):
        super().__init__()
        self.enc1, self.enc2, self.enc3, self.bottom = conv_block(1, 16), conv_block(16, 32),
conv_block(32, 64), conv_block(64, 64)
        self.classify = nn.Conv2d(64, n_classes, kernel_size=1)          # a conv instead of an
FC layer
    def forward(self, x):
        x = self.enc1(x)
```

```
        x = self.enc2(F.max_pool2d(x, 2))
        x = self.enc3(F.max_pool2d(x, 2))
        x = self.bottom(F.max_pool2d(x, 2))                                  # 8x8 map of rich "what"
features
        return F.interpolate(self.classify(x), scale_factor=8, mode="bilinear",
align_corners=False)

fcn = FCN()
for size in [64, 128]:
    out = fcn(torch.zeros(1, 1, size, size))
    print(f"input {size}x{size}  ->  output {tuple(out.shape)}   (4 class scores for every
pixel)")
```

```
input 64x64  ->  output (1, 4, 64, 64)   (4 class scores for every pixel)
input 128x128  ->  output (1, 4, 128, 128)   (4 class scores for every pixel)
```

No `Flatten`, no `nn.Linear`. The same network accepts 64×64 *or* 128×128 images and returns a map of the same size. Use sizes divisible by 8 here, because each pooling step rounds odd sizes down. **Weakness:** the prediction is made on an **8×8 map** and then blown up 8×. Each predicted value covers an 8×8-pixel block, so **predictions are coarse and boundaries blurry** (Medium Q17). Let's see that by training it.

```
def train_seg(model, epochs=6, lr=2e-3, batch=32):
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    for _ in range(epochs):
        model.train()
        perm = torch.randperm(len(Xtr))
        for i in range(0, len(Xtr), batch):
            idx = perm[i:i + batch]
            loss = F.cross_entropy(model(Xtr[idx]), Ytr[idx])      # cross-entropy computed at
EVERY pixel
            opt.zero_grad(); loss.backward(); opt.step()
    model.eval()
    with torch.no_grad():
        return model(Xte).argmax(1)

def mean_iou(pred, truth):
    ious = []
    for c in range(1, 4):
        inter = ((pred == c) & (truth == c)).sum().item(); union = ((pred == c) | (truth ==
c)).sum().item()
        if union:
            ious.append(inter / union)
    return float(np.mean(ious))

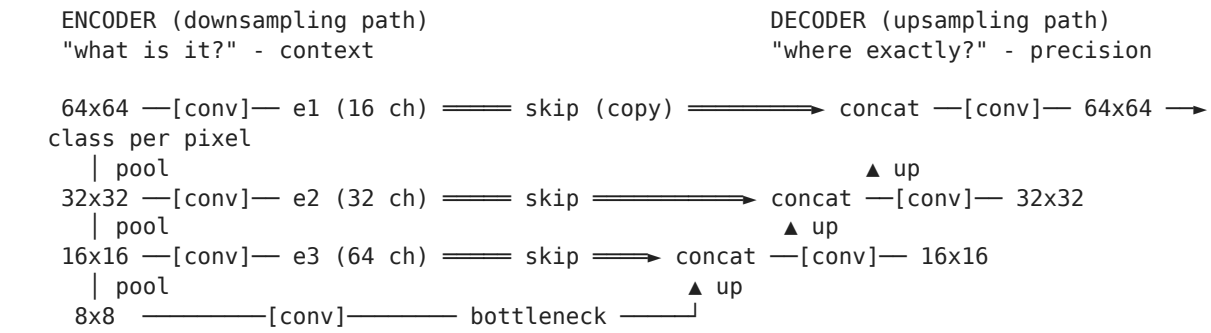
def boundary_accuracy(pred, truth):
    # accuracy measured ONLY on pixels lying on object edges - the hard part of segmentation
    edges = np.stack([cv2.morphologyEx((m > 0).astype(np.uint8), cv2.MORPH_GRADIENT, np.ones((3,
3), np.uint8))
                      for m in truth.numpy()]) > 0
    return (pred.numpy()[edges] == truth.numpy()[edges]).mean()

torch.manual_seed(0)
t = time.time()
fcn_pred = train_seg(FCN())
print(f"FCN trained in {time.time() - t:.0f} s:  mean IoU {mean_iou(fcn_pred, Yte):.3f},  "
      f"pixel accuracy {(fcn_pred == Yte).float().mean():.1%},  accuracy on boundary pixels
{boundary_accuracy(fcn_pred, Yte):.1%}")
```

```
FCN trained in 24 s:  mean IoU 0.824,  pixel accuracy 97.9%,  accuracy on boundary pixels 77.2%
```

3 · U-Net (2015): encoder + decoder + skip connections

Designed for **biomedical images** (cells under a microscope), U-Net is now the standard in medical segmentation.



- **Encoder (downsampling path):** conv + pooling **reduce the resolution** while building **context and high-level features** ("this region looks like a tumour"). It's like summarising a photo: you keep the big picture and lose fine detail.
- **Decoder (upsampling path): transposed convolutions** (learned upsampling) **recover the spatial resolution** to produce pixel-level predictions. It's like expanding the summary back to full size, but now with a label on every pixel.
- **Skip connections: copy** each encoder level's feature maps across and **concatenate** them with the decoder at the same resolution. This brings back the **fine edge details** that pooling threw away and combines **low-level detail** (edges, from early layers) with **high-level context** (meaning, from deep layers).

> **In short: the encoder compresses for context, the decoder reconstructs for precision, and skip connections bridge the two so no important detail is lost.**

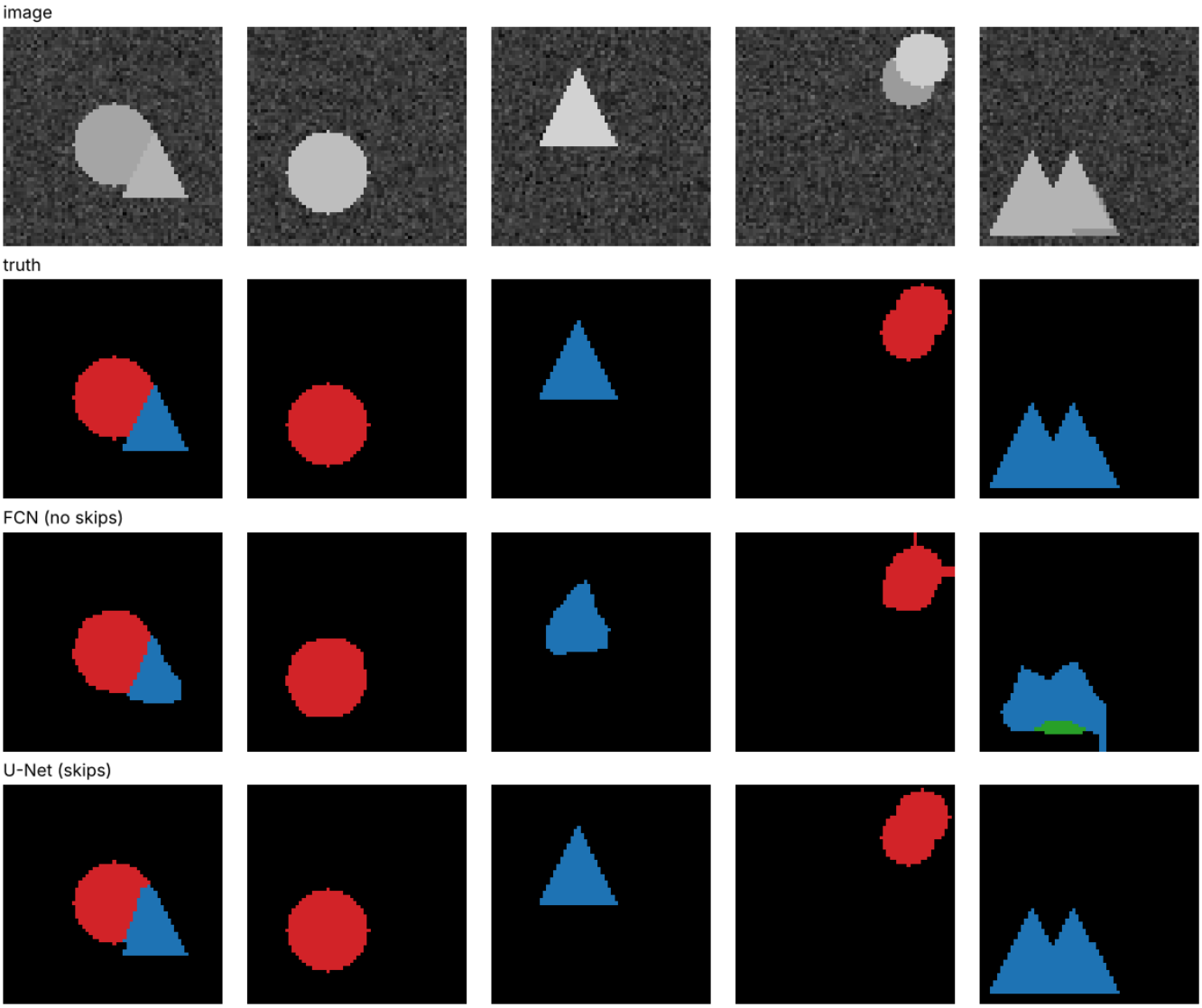
```
class UNet(nn.Module):
    def __init__(self, n_classes=4):
        super().__init__()
        self.enc1, self.enc2, self.enc3, self.bottom = conv_block(1, 16), conv_block(16, 32),
conv_block(32, 64), conv_block(64, 64)
        self.up3, self.dec3 = nn.ConvTranspose2d(64, 64, 2, stride=2), conv_block(64 + 64, 64)
        self.up2, self.dec2 = nn.ConvTranspose2d(64, 32, 2, stride=2), conv_block(32 + 32, 32)
        self.up1, self.dec1 = nn.ConvTranspose2d(32, 16, 2, stride=2), conv_block(16 + 16, 16)
        self.classify = nn.Conv2d(16, n_classes, 1)
    def forward(self, x):
        e1 = self.enc1(x)                                   # 64x64, fine detail
        e2 = self.enc2(F.max_pool2d(e1, 2))                 # 32x32
        e3 = self.enc3(F.max_pool2d(e2, 2))                 # 16x16
        b = self.bottom(F.max_pool2d(e3, 2))                # 8x8, most context
        d3 = self.dec3(torch.cat([self.up3(b), e3], dim=1))     # upsample + SKIP from e3
        d2 = self.dec2(torch.cat([self.up2(d3), e2], dim=1))    # upsample + SKIP from e2
        d1 = self.dec1(torch.cat([self.up1(d2), e1], dim=1))    # upsample + SKIP from e1
        return self.classify(d1)

# Transposed convolution = learned upsampling (doubles height and width here)
print("transposed conv:", tuple(nn.ConvTranspose2d(64, 32, 2, stride=2)(torch.zeros(1, 64, 8,
8)).shape), "<- from (1, 64, 8, 8)")

torch.manual_seed(0)
t = time.time()
unet_pred = train_seg(UNet())
print(f"U-Net trained in {time.time() - t:.0f} s:  mean IoU {mean_iou(unet_pred, Yte):.3f},  "
      f"pixel accuracy {(unet_pred == Yte).float().mean():.1%},  accuracy on boundary pixels
{boundary_accuracy(unet_pred, Yte):.1%}")
```

```
transposed conv: (1, 32, 16, 16) <- from (1, 64, 8, 8)
U-Net trained in 63 s:  mean IoU 0.941,  pixel accuracy 99.6%,  accuracy on boundary pixels 99.2%
```

```
fig, ax = plt.subplots(4, 5, figsize=(12, 10))
for col, k in enumerate([0, 3, 7, 11, 15]):
    rows = [(Xte[k, 0], "gray", "image"), (Yte[k], mask_cmap, "truth"),
            (fcn_pred[k], mask_cmap, "FCN (no skips)"), (unet_pred[k], mask_cmap, "U-Net
(skips)")]
    for row, (im, cmap, title) in enumerate(rows):
        ax[row, col].imshow(im, cmap=cmap, vmin=0, vmax=(1 if cmap == "gray" else 3))
        ax[row, col].axis("off")
        if col == 0:
            ax[row, col].set_title(title, loc="left")
plt.tight_layout(); plt.show()
```



Look at the edges. The FCN's shapes are blobby: corners are rounded off and triangles lose their points, because its decision was made at 1/8 resolution. The U-Net reproduces sharp corners. The *boundary-pixel accuracy* numbers above measure exactly this.

Why skip connections help (Medium Q18, Hard Q8): pooling discards *where* edges are, so the deepest layers know *what* but not *exactly where*. The skip connections hand the decoder the high-resolution feature maps from the encoder, so it can place each boundary precisely. The encoder supplies context, the skips supply precision, and the decoder combines both.

Why U-Net is everywhere in medical imaging (Medium Q20):

1. **Precise boundaries**, which you need to measure tumour size and plan surgery or radiotherapy.

2. **Small structures** (cells, lesions, vessels) survive thanks to the skip connections.
3. It **works with small datasets** (medical labels are expensive), helped by heavy augmentation.

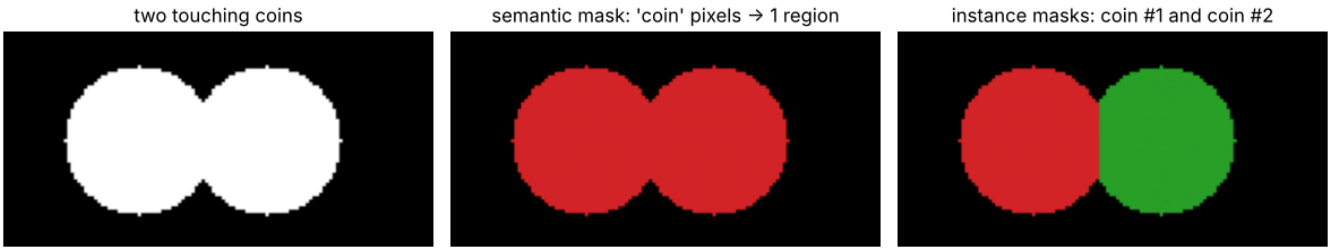
Satellite imagery (Hard Q9): choose a **U-Net** (often with a pretrained ResNet or EfficientNet encoder). Roads, rivers and building outlines are thin structures with sharp boundaries that need pixel precision, and the encoder's context separates look-alike classes (roof vs road). Very large images are processed in overlapping tiles. Use **Mask R-CNN** instead if you need to count *individual* buildings or cars (instance segmentation).

4 · Mask R-CNN (2017): detection + a mask for every object

Limitation it addresses (Q1): detectors output only **boxes + labels**. Many applications need the **exact shape of each individual object**, i.e. **instance segmentation**. Semantic segmentation can't separate touching objects of the same class. Let's see that:

```
touching = np.zeros((64, 128), np.uint8)
cv2.circle(touching, (40, 32), 22, 1, -1)
cv2.circle(touching, (78, 32), 22, 1, -1)        # two circles that overlap/touch

n_regions, regions = cv2.connectedComponents(touching)
fig, ax = plt.subplots(1, 3, figsize=(12, 2.8))
ax[0].imshow(touching, cmap="gray"); ax[0].set_title("two touching coins")
ax[1].imshow(touching, cmap=mask_cmap, vmin=0, vmax=3); ax[1].set_title(f"semantic mask: 'coin'
pixels -> {n_regions - 1} region")
inst = np.zeros_like(touching)
yy, xx = np.mgrid[:64, :128]
inst[(touching > 0) & ((xx - 40) ** 2 + (yy - 32) ** 2 <= (xx - 78) ** 2 + (yy - 32) ** 2)] = 1
inst[(touching > 0) & (inst == 0)] = 2
ax[2].imshow(inst, cmap=mask_cmap, vmin=0, vmax=3); ax[2].set_title("instance masks: coin #1 and
coin #2")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```



Architecture: Faster R-CNN + a mask branch

Base: Faster R-CNN (a detector). **Extension:** a **segmentation mask branch** for every Region of Interest (RoI).

How it works, step by step:

1. **Backbone** (e.g. ResNet-50 + FPN): computes feature maps of the whole image once.
2. **Region Proposal Network (RPN)** (Q3): a small network that slides over the feature map and **suggests candidate object regions** ("something might be here"), with objectness scores. It replaces the slow selective search of R-CNN.

3. **RoI Align** (Q4): cuts out a fixed-size feature patch (e.g. 7×7) for each proposal, **precisely**, without rounding coordinates (see the demo below).
4. **Classification + bounding-box regression:** what object is it, plus a refined box.
5. **Mask branch:** a small FCN that predicts a **binary mask** (e.g. 28×28) inside each box. It runs in parallel with step 4.

Three outputs per object (Q5): ① a bounding box ② a class label ③ a pixel-level mask.

Why RoI Align? (Q4)

Proposal boxes have **fractional** coordinates on the feature map (a box from x = 1.3 to 3.7). The older **RoI Pooling rounds** them to whole cells (1 to 4), so the extracted features are slightly **misaligned** with the object. For boxes that hardly matters, but for **pixel-accurate masks** a shift of half a feature cell becomes many pixels in the image (each cell might cover 16 pixels). **RoI Align** removes the rounding: it **samples the feature map exactly at the fractional positions using bilinear interpolation**. Let's see the difference:

```
# A feature map whose value at every position = its x coordinate (0, 1, 2, ... 7), so
misalignment is easy to read
fmap = torch.arange(8, dtype=torch.float32).repeat(8, 1)[None, None]   # (1, 1, 8, 8)
box = torch.tensor([[0, 1.3, 1.3, 3.7, 3.7]])                           # (batch index, x1, y1,
x2, y2) - fractional!

pooled = roi_pool(fmap, box, output_size=(1, 4), spatial_scale=1.0)
# aligned=False: our toy map stores each value AT its whole-number coordinate (x=0,1,2,...)
aligned = roi_align(fmap, box, output_size=(1, 4), spatial_scale=1.0, sampling_ratio=2,
aligned=False)

# What the 4 output columns SHOULD contain: the x positions at the centres of 4 equal slices of
1.3 .. 3.7
exact = [1.3 + (i + 0.5) * (3.7 - 1.3) / 4 for i in range(4)]
print("ideal values            :", np.round(exact, 2))
print("RoI Pool  (rounds box)  :", pooled[0, 0, 0].numpy(), "<- snapped to whole cells")
print("RoI Align (bilinear)    :", aligned[0, 0, 0].numpy().round(2), "<- follows the true box")
```

```
ideal values            : [1.6 2.2 2.8 3.4]
RoI Pool  (rounds box)  : [1. 2. 3. 4.] <- snapped to whole cells
RoI Align (bilinear)    : [1.6 2.2 2.8 3.4] <- follows the true box
```

RoI Pool's values are snapped to whole feature cells (and taking the max inside each bin biases them further). RoI Align reproduces the ideal values exactly, because it samples where the true box is. For segmentation masks this misalignment translates directly into shifted mask edges. RoI Align improved mask accuracy noticeably in the paper.

Applications (Q7)

- **Medical imaging:** tumour segmentation, organ boundaries.
- **Autonomous driving:** pedestrians, vehicles and lane markings with precise shapes.
- **Crowd analysis:** separating overlapping people in dense scenes.
- **Others:** robotics grasping (exact object outline), photo editing (cut out a person), retail shelf analysis, agriculture (individual fruit or leaves).

Overlapping objects in crowded scenes (Q8)

Mask R-CNN treats each object as its own **proposal (RoI)**. Each RoI gets its own class and **its own mask**, predicted only inside its box. Two overlapping people become two RoIs with two separate masks, even where they touch. NMS removes duplicate proposals. Each mask is a binary *this object / not this object* prediction per RoI, so masks of different objects don't compete with each other.

Why pixel masks beat boxes in medical imaging (Q6)

1. **Precise localization:** tumours and organs have irregular shapes, and a box includes lots of healthy tissue.
2. **Quantitative analysis:** masks give the true **size, area and volume**, while boxes overestimate them (e.g. tumour volume for chemotherapy dosing).
3. **Clinical decisions:** surgery and radiotherapy need the exact border and how it spreads.
4. **Overlaps:** a tumour inside an organ can't be separated by boxes, but masks can.

> **Key insight:** boxes say *"there's a tumour here"*; masks say *exactly which pixels are tumour*.

5 · Optional: a real pretrained Mask R-CNN on your photo

Downloads about 170 MB of weights the first time. Set `RUN = True` on your own machine.

```
RUN = False
IMAGE_PATH = "../../week-1/code/sample.jpg"

if RUN:
    from torchvision.models.detection import maskrcnn_resnet50_fpn, MaskRCNN_ResNet50_FPN_Weights
    weights = MaskRCNN_ResNet50_FPN_Weights.DEFAULT
    model = maskrcnn_resnet50_fpn(weights=weights).eval()
    photo = cv2.cvtColor(cv2.imread(IMAGE_PATH), cv2.COLOR_BGR2RGB)
    with torch.no_grad():
        out = model([torch.tensor(photo).permute(2, 0, 1).float() / 255])[0]
    overlay = photo.copy()
    for mask, label, score in zip(out["masks"], out["labels"], out["scores"]):
        if score > 0.6:
            colour = np.random.randint(0, 255, 3)
            overlay[mask[0].numpy() > 0.5] = (0.5 * overlay[mask[0].numpy() > 0.5] + 0.5 *
colour).astype(np.uint8)
            print(f"{weights.meta['categories'][label]:<12} score {score:.2f}")
    plt.figure(figsize=(10, 6)); plt.imshow(overlay); plt.axis("off"); plt.show()
else:
    print("Skipped. Set RUN = True to download and run a pretrained Mask R-CNN.")
```

```
Skipped. Set RUN = True to download and run a pretrained Mask R-CNN.
```

Key takeaways

- **FCN:** the first step. Replacing FC layers with conv layers proved CNNs can do segmentation, but predictions are **coarse**.
- **U-Net:** improved precision with an **encoder–decoder + skip connections**, and became the standard in medical imaging.

- **Mask R-CNN:** extended detection to **instance segmentation**. Faster R-CNN + mask branch + **RoI Align**: box, class and mask per object.

✅ Check yourself

1. What is semantic segmentation? *(assigning a class label to every pixel)*
2. How does an FCN's output differ from a normal CNN's? *(a dense per-pixel map instead of one label; FC layers replaced by convolutions)*
3. Why are FCN predictions coarse? *(they're made at low resolution after pooling, then upsampled; fine detail is lost)*
4. Roles of encoder and decoder in U-Net? *(encoder: downsample to capture context/what; decoder: upsample to recover resolution/where)*
5. What do U-Net's skip connections do? *(pass high-resolution encoder features to the decoder to restore fine boundaries)*
6. Mask R-CNN's three outputs? *(bounding box, class label, pixel mask)*
7. Why RoI Align instead of RoI Pooling? *(no coordinate rounding; bilinear sampling keeps features aligned, giving accurate masks)*

06 · Sequence modeling for video: RNN, LSTM, CNN + LSTM

Covers (Week 3 Key Concepts): "Sequence Modeling for Video", "RNNs and LSTMs", "RNN vs LSTM", "What features would CNN / LSTM capture?". **Questions answered here:** Sequence 9–14 · RNN vs LSTM 15–18.

> **Sequence modeling** = building models for data that comes **in order over time**, where the current element **depends on what came before** (*temporal dependencies*).

A **video is a sequence of frames**. To understand it you need two things:

- **spatial features**: *what* is in each frame (a hand, a ball, a car). A **CNN** handles this.
- **temporal dependencies**: *how* things change across frames (the hand moves left → right). An **RNN / LSTM** handles this.

Analogy from the notes: the **CNN is a photographer** who takes and describes snapshots; the **LSTM is a storyteller** who connects the snapshots into a narrative.

```
%matplotlib inline
import time
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(0)
print("torch", torch.__version__)
```

```
torch 2.14.1+cu130
```

1 · Why a CNN alone can't understand motion (Q9)

Below are two short videos of a square. One moves **right**, the other moves **left**. Each frame on its own, which is all a CNN sees, looks the same in both: "a square". Even if you have **all** the frames but **shuffle their order**, you can't tell the direction. **The information is in the order.**

```
T, H = 8, 32                        # 8 frames of 32x32 pixels
DIRS = ["right", "left", "down", "up"]

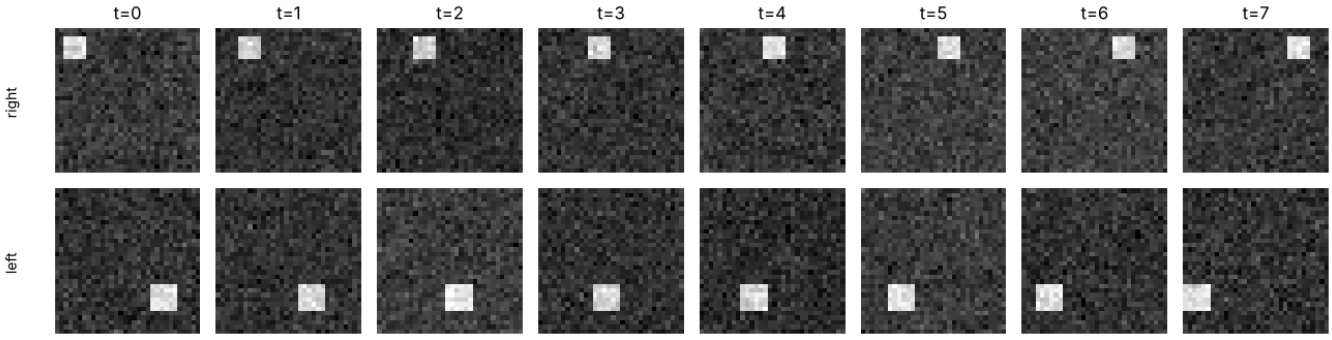
def make_video(rng):
    d = int(rng.integers(0, 4)); speed = int(rng.integers(2, 4)); size = int(rng.integers(5, 9))
    span = speed * (T - 1)
    if d in (0, 1):
        x0, y0 = int(rng.integers(0, H - size - span)), int(rng.integers(0, H - size))
        xs = [x0 + speed * t for t in range(T)] if d == 0 else [x0 + span - speed * t for t in
range(T)]
        ys = [y0] * T
    else:
        y0, x0 = int(rng.integers(0, H - size - span)), int(rng.integers(0, H - size))
        ys = [y0 + speed * t for t in range(T)] if d == 2 else [y0 + span - speed * t for t in
range(T)]
```

```
        xs = [x0] * T
    video = np.zeros((T, 1, H, H), np.float32)
    for t in range(T):
        video[t, 0, ys[t]:ys[t] + size, xs[t]:xs[t] + size] = 1
    video += rng.normal(0, 0.1, video.shape).astype(np.float32)        # a little camera noise
    return video, d

rng = np.random.default_rng(0)
train = [make_video(rng) for _ in range(2000)]
test = [make_video(rng) for _ in range(400)]
Xtr = torch.tensor(np.stack([v for v, _ in train])); ytr = torch.tensor([d for _, d in train])
Xte = torch.tensor(np.stack([v for v, _ in test])); yte = torch.tensor([d for _, d in test])
print("videos:", tuple(Xtr.shape), "= (clips, frames, channels, height, width)")

right = next(v for v, d in train if d == 0); left = next(v for v, d in train if d == 1)
fig, ax = plt.subplots(2, T, figsize=(13, 3.6))
for t in range(T):
    ax[0, t].imshow(right[t, 0], cmap="gray"); ax[0, t].set_title(f"t={t}"); ax[0, t].axis("off")
    ax[1, t].imshow(left[t, 0], cmap="gray"); ax[1, t].axis("off")
ax[0, 0].text(-12, 16, "right", rotation=90, va="center"); ax[1, 0].text(-12, 16, "left",
rotation=90, va="center")
plt.tight_layout(); plt.show()
```

```
videos: (2000, 8, 1, 32, 32) = (clips, frames, channels, height, width)
```



2 · RNN: a network with memory

A **Recurrent Neural Network** processes a sequence **one step at a time** and keeps a **hidden state** *h*, a small vector that carries information forward:

$h_t = \tanh(W \cdot x_t + U \cdot h_{t-1} + b)$

The new memory mixes the **current input** x_t with the **previous memory** h_{t-1}. The same weights W and U are reused at every step.

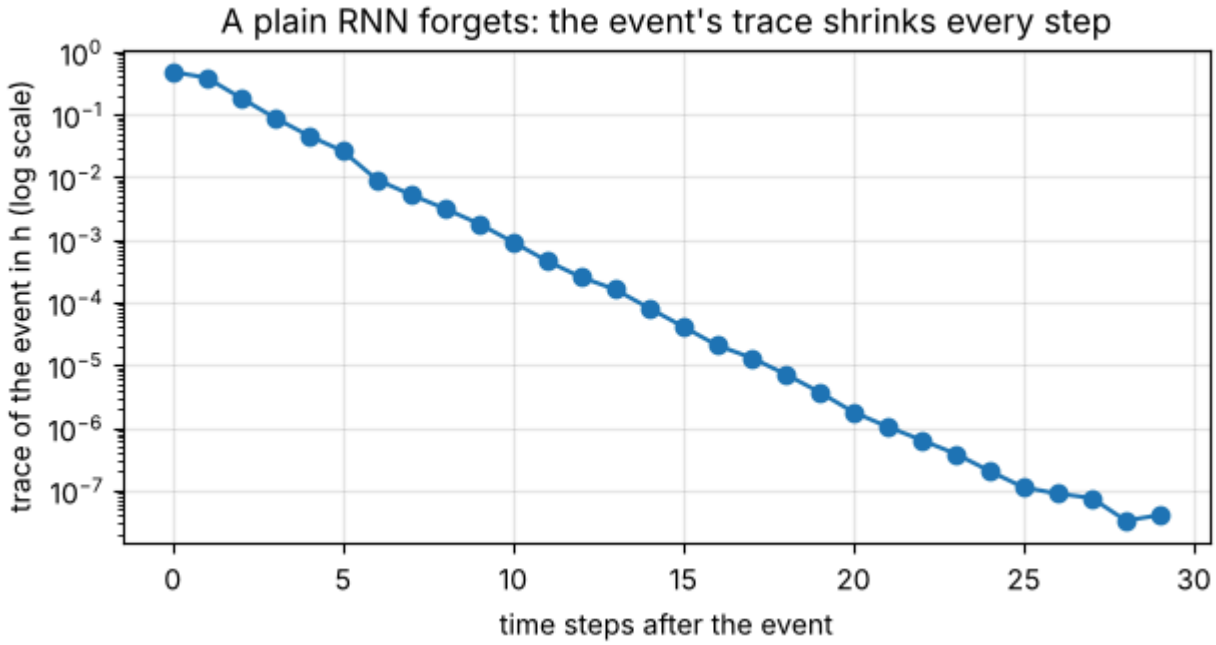
Strengths: captures order and context (e.g. "bank" means river-bank or money-bank depending on earlier words). **Limitation: short-term memory.** Information from early steps **fades** as it is squashed through tanh and multiplied by U again and again. That's the **vanishing gradient problem** again, this time across *time steps* instead of layers. Watch a single "memory" fade:

```
torch.manual_seed(0)
rnn_cell = nn.RNNCell(input_size=1, hidden_size=16)
h = torch.zeros(1, 16)
inputs = [1.0] + [0.0] * 29                          # a single "event" at t=0, then 29 empty
steps
with_event = []
for x in inputs:
    h = rnn_cell(torch.tensor([[x]]), h)
    with_event.append(h)

# Run again with NO event, then compare the two hidden states step by step:
```

```python
# the difference is the "trace" of the event that the RNN still remembers
h0 = torch.zeros(1, 16); diff = []
for t, x in enumerate([0.0] * 30):
    h0 = rnn_cell(torch.tensor([[x]]), h0)
    diff.append((with_event[t] - h0).norm().item())

plt.figure(figsize=(7, 3.2))
plt.semilogy(diff, "o-")
plt.xlabel("time steps after the event"); plt.ylabel("trace of the event in h (log scale)")
plt.title("A plain RNN forgets: the event's trace shrinks every step"); plt.grid(alpha=0.3);
plt.show()
print(f"trace after 1 step: {diff[0]:.3f}   after 10 steps: {diff[10]:.1e}   after 29 steps:
{diff[29]:.1e}")
```




```
trace after 1 step: 0.479   after 10 steps: 9.2e-04   after 29 steps: 4.0e-08
```

3 · LSTM: Long Short-Term Memory

An **LSTM** is a special RNN designed for **long-range dependencies**. It keeps a separate **cell state** c_t, a "conveyor belt" of memory, and three **gates** (each a number between 0 and 1, computed by a small sigmoid layer) decide what happens to it:

Gate	Question it answers	Formula (σ = sigmoid)
Forget gate f_t	what past information should I **discard**?	$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
Input gate i_t	what **new** information should I **store**?	$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
Output gate o_t	what should I **pass forward** right now?	$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$

$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ (keep some old memory + add some new candidate memory $\tilde{c}_t$) **$h_t = o_t \odot \tanh(c_t)$**

Why this beats vanishing gradients (Q15): the cell state is updated by **multiplying by f_t and *adding*** new information, not squashed through tanh at every step. When the forget gate is about 1 ("keep this"), the memory and its gradient pass through many steps almost unchanged. It's the same "additive highway" trick as ResNet's skip connection, but across time.

One LSTM step by hand

```
torch.manual_seed(1)
cell = nn.LSTMCell(input_size=2, hidden_size=3)
x = torch.tensor([[1.0, -0.5]]); h_prev = torch.zeros(1, 3); c_prev = torch.tensor([[0.8, -0.3,
0.5]])

# PyTorch stores the 4 parts stacked as [input gate, forget gate, candidate, output gate]
z = x @ cell.weight_ih.T + cell.bias_ih + h_prev @ cell.weight_hh.T + cell.bias_hh
i_gate, f_gate, g_cand, o_gate = z.chunk(4, dim=1)
i_gate, f_gate, o_gate, g_cand = torch.sigmoid(i_gate), torch.sigmoid(f_gate),
torch.sigmoid(o_gate), torch.tanh(g_cand)
c_new = f_gate * c_prev + i_gate * g_cand
h_new = o_gate * torch.tanh(c_new)

np.set_printoptions(precision=3, suppress=True)
print("forget gate f:", f_gate.detach().numpy()[0], " (1 = keep old memory, 0 = erase)")
print("input gate  i:", i_gate.detach().numpy()[0], " (how much new info to write)")
print("candidate  c~:", g_cand.detach().numpy()[0])
print("output gate o:", o_gate.detach().numpy()[0], " (how much to reveal)")
print("old cell c   :", c_prev.numpy()[0], "-> new cell c:", c_new.detach().numpy()[0])
h_torch, c_torch = cell(x, (h_prev, c_prev))
print("matches PyTorch's LSTMCell:", torch.allclose(h_new, h_torch) and torch.allclose(c_new,
c_torch))
```

```
forget gate f: [0.592 0.5   0.583]  (1 = keep old memory, 0 = erase)
input gate  i: [0.583 0.54  0.456]  (how much new info to write)
candidate  c~: [-0.219  0.344 -0.641]
output gate o: [0.394 0.577 0.449]  (how much to reveal)
old cell c   : [ 0.8 -0.3  0.5] -> new cell c: [ 0.346  0.036 -0.   ]
matches PyTorch's LSTMCell: True
```

4 · Experiment: who can remember the beginning of a long sequence?

Task: each sequence is T random noise vectors, except that **the first step secretly encodes a class (0–3)**. The model must output that class **after reading the whole sequence**. To succeed it has to carry information from step 1 to the end.

We train a plain RNN and an LSTM on T = 5, 20 and 50 steps. A common LSTM trick: start the **forget-gate bias at 1** so it begins by "keeping" its memory. We also measure how much gradient reaches the first step.

```
def memory_data(n, T, seed):
    g = torch.Generator().manual_seed(seed)
    y = torch.randint(0, 4, (n,), generator=g)
    x = torch.randn(n, T, 8, generator=g)
    x[:, 0, :4] = 0
    x[torch.arange(n), 0, y] = 3.0               # the answer is hidden in the FIRST step only
    return x, y

class SequenceClassifier(nn.Module):
    def __init__(self, kind, hidden=32):
        super().__init__()
        self.rnn = (nn.RNN if kind == "RNN" else nn.LSTM)(8, hidden, batch_first=True)
        self.fc = nn.Linear(hidden, 4)
        if kind == "LSTM":                          # forget-gate bias = 1 (PyTorch order: input,
FORGET, cell, output)
            with torch.no_grad():
                self.rnn.bias_ih_l0[hidden:2 * hidden].fill_(1.0)
                self.rnn.bias_hh_l0[hidden:2 * hidden].fill_(0.0)
    def forward(self, x):
        out, _ = self.rnn(x)
```

```
        return self.fc(out[:, -1])                # decide using the LAST hidden state

print(f"{'steps T':>7} | {'RNN accuracy':>12} | {'LSTM accuracy':>13} | {'gradient reaching step
1 (RNN / LSTM)':>38}")
t0 = time.time()
for T_len in [5, 20, 50]:
    Xs, ys = memory_data(2000, T_len, 0); Xv, yv = memory_data(500, T_len, 1)
    row = {}
    for kind in ["RNN", "LSTM"]:
        torch.manual_seed(0)
        fresh = SequenceClassifier(kind)
        xg = Xv[:100].clone().requires_grad_(True)
        F.cross_entropy(fresh(xg), yv[:100]).backward()
        grad_ratio = (xg.grad[:, 0].norm() / xg.grad[:, -1].norm()).item()     # first step vs
last step
        model = fresh
        opt = torch.optim.Adam(model.parameters(), lr=3e-3)
        for _ in range(40):
            perm = torch.randperm(2000)
            for i in range(0, 2000, 100):
                idx = perm[i:i + 100]
                loss = F.cross_entropy(model(Xs[idx]), ys[idx])
                opt.zero_grad(); loss.backward(); opt.step()
        with torch.no_grad():
            row[kind] = ((model(Xv).argmax(1) == yv).float().mean().item(), grad_ratio)
    print(f"{T_len:>7} | {row['RNN'][0]:>12.1%} | {row['LSTM'][0]:>13.1%} | {row['RNN']
[1]:>17.1e} / {row['LSTM'][1]:.1e}")
print(f"(random guessing = 25%; took {time.time() - t0:.0f} s)")
```

```
steps T | RNN accuracy | LSTM accuracy |  gradient reaching step 1 (RNN / LSTM)
      5 |       100.0% |         99.8% |           1.3e-01 / 3.0e-01
     20 |        50.2% |        100.0% |           5.3e-06 / 1.2e-02
     50 |        24.6% |         99.8% |           3.7e-14 / 3.9e-05
(random guessing = 25%; took 17 s)
```

- **T = 5:** both remember. Short sequences are easy, so a plain RNN is enough.
- **T = 20:** the RNN drops to about 50%. **T = 50:** the RNN is at **25%, pure guessing**. The gradient reaching step 1 has shrunk to about 10^{-14} of the last step's, so it never learns that step 1 matters.
- The **LSTM stays near 100%**. Its gradient at step 1 is about a billion times larger.

	RNN	LSTM
Memory	short-term	long-term (long dependencies)
Problem	vanishing gradients	solves (greatly reduces) vanishing gradients
Best for	simple, short sequences	complex, long sequences
Example	predict the next word in a short sentence	understand context across a long paragraph or video

Variable length (Q17): both process one step at a time with the same weights, so any length works. The LSTM's gated memory keeps working when a clip is long (a 10-second gesture vs a 2-second one), while an RNN only remembers the last few frames.

When is a plain RNN still enough? (Q18) Short sequences where only recent steps matter: next-character prediction in short words, smoothing a few recent sensor readings, predicting the next note from the last few notes. RNNs are also smaller and faster. (**GRU** is a lighter LSTM variant with 2 gates, often just as good.)

5 · The CNN + LSTM pipeline for video (Q12)

1. **CNN** extracts **spatial features** from **each frame**: one feature vector per frame (*what*).
2. **LSTM** processes the **sequence of feature vectors** across frames (*how it changes*).
3. An **output layer** predicts the action or gesture class.

We train it on our moving-square videos and compare it with two models that ignore order:

- **single frame:** a CNN that sees only the middle frame,
- **frame average:** a CNN on every frame, with the features averaged (all frames, order thrown away).

```
def frame_cnn():
    # turns one 32x32 frame into a 64-number feature vector
    return nn.Sequential(nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
                         nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
                         nn.Conv2d(32, 32, 3, padding=1), nn.ReLU(), nn.AdaptiveAvgPool2d(4),
                         nn.Flatten(), nn.Linear(32 * 16, 64), nn.ReLU())

class SingleFrame(nn.Module):
    def __init__(self):
        super().__init__(); self.cnn = frame_cnn(); self.fc = nn.Linear(64, 4)
    def forward(self, v):
        return self.fc(self.cnn(v[:, T // 2]))                          # middle frame only

class FrameAverage(nn.Module):
    def __init__(self):
        super().__init__(); self.cnn = frame_cnn(); self.fc = nn.Linear(64, 4)
    def forward(self, v):
        B = v.shape[0]
        feats = self.cnn(v.reshape(B * T, 1, H, H)).reshape(B, T, 64)
        return self.fc(feats.mean(dim=1))                               # all frames, ORDER LOST

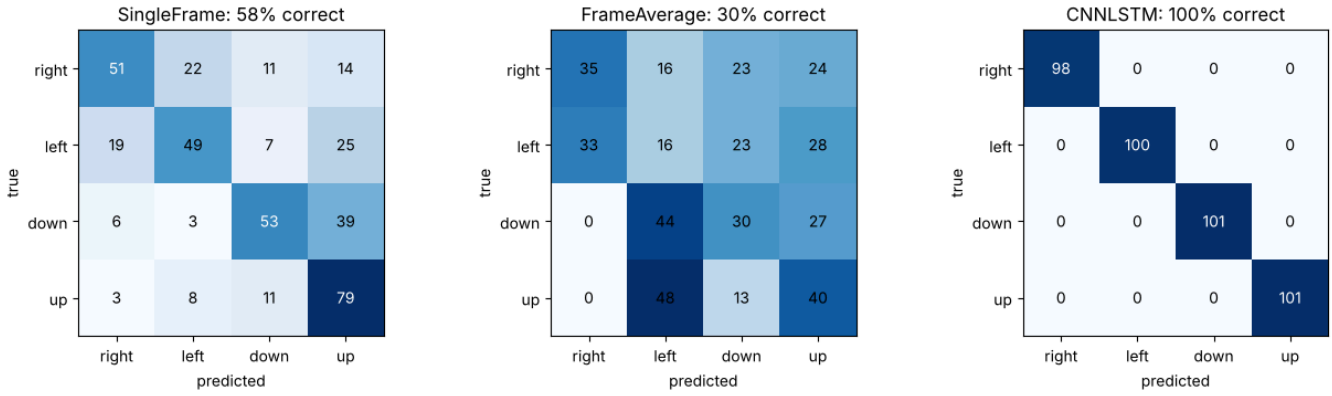
class CNNLSTM(nn.Module):
    def __init__(self):
        super().__init__(); self.cnn = frame_cnn(); self.lstm = nn.LSTM(64, 64,
batch_first=True); self.fc = nn.Linear(64, 4)
    def forward(self, v):
        B = v.shape[0]
        feats = self.cnn(v.reshape(B * T, 1, H, H)).reshape(B, T, 64)   # step 1: CNN on every
frame
        out, _ = self.lstm(feats)                                        # step 2: LSTM over time
        return self.fc(out[:, -1])                                       # step 3: classify

def fit(model, epochs=10):
    opt = torch.optim.Adam(model.parameters(), lr=2e-3)
    for _ in range(epochs):
        model.train(); perm = torch.randperm(len(Xtr))
        for i in range(0, len(Xtr), 64):
            idx = perm[i:i + 64]
            loss = F.cross_entropy(model(Xtr[idx]), ytr[idx])
            opt.zero_grad(); loss.backward(); opt.step()
    model.eval()
    with torch.no_grad():
        return model(Xte).argmax(1)

preds = {}
t0 = time.time()
for M in [SingleFrame, FrameAverage, CNNLSTM]:
    torch.manual_seed(0)
    preds[M.__name__] = fit(M())
print(f"trained 3 models in {time.time() - t0:.0f} s\n")
```

```
fig, ax = plt.subplots(1, 3, figsize=(13, 3.8))
for a, (name, p) in zip(ax, preds.items()):
    cm = np.zeros((4, 4), int)
    for t_, p_ in zip(yte.tolist(), p.tolist()):
        cm[t_, p_] += 1
    a.imshow(cm, cmap="Blues")
    for i in range(4):
        for j in range(4):
            a.text(j, i, cm[i, j], ha="center", va="center", color="white" if cm[i, j] > 50 else
"black")
    a.set_xticks(range(4)); a.set_xticklabels(DIRS); a.set_yticks(range(4));
a.set_yticklabels(DIRS)
    a.set_xlabel("predicted"); a.set_ylabel("true")
    a.set_title(f"{name}: {(p == yte).float().mean():.0%} correct")
plt.tight_layout(); plt.show()
```

```
trained 3 models in 41 s
```



- **CNN + LSTM: about 100%.** It knows *what* (a square, at this position) in each frame **and** *how* that changes over time.
- **Frame average: far below.** Averaging throws away the order, so "right" and "left" (the same frames in reverse) look identical. At best it could tell horizontal from vertical, and in practice it does even worse.
- **Single frame: well below.** One frame can't show motion. It does a little better than guessing only because our 8-frame clips have no exact middle frame, so the chosen frame's position leaks a small hint.

In short: CNNs capture the *what* (spatial features), LSTMs capture the *when/how* (temporal sequence). Together they make video understanding possible.

What each part captures (Key Concepts)

CNN (per frame)	LSTM (across frames)
objects: a hand, a ball, a car	**motion**: the hand moves left → right
shapes & edges: outline of hand or body	**order**: first → next → last
textures & patterns: clothing, background	**dynamics**: speed, rhythm, continuity

Where CNN + LSTM is essential (Q13, Q14)

- **Gesture recognition:** sign language, touch-free controls. A "wave" is a *sequence* of hand positions; any single frame is just "a raised hand". The CNN detects the hand shape per

frame, and the LSTM recognises the movement pattern ("wave", "swipe left"). A static image classifier only sees a pose, so it can't tell "waving" from "raising a hand".

- **Action recognition:** sports analysis, surveillance ("throwing a ball" = person + ball + arm motion over time).
- **Video captioning:** describing what happens in a clip.
- **Autonomous driving:** predicting where a pedestrian is heading from their recent motion.

✅ Check yourself

1. Why can't CNNs alone capture temporal dependencies? *(they process each frame independently; nothing links frames or keeps their order)*
2. What problem do LSTMs solve compared with vanilla RNNs? *(vanishing gradients, giving long-term memory)*
3. Roles of the input, forget and output gates? *(store new info / discard old info / decide what to pass forward)*
4. What does each part of a CNN + LSTM pipeline contribute? *(CNN: spatial features per frame; LSTM: temporal pattern; output layer: the class)*
5. Why do LSTMs suit variable-length video? *(step-by-step processing handles any length; gated memory spans long clips)*
6. When might a plain RNN still be enough? *(short sequences where only recent context matters)*

07 · Vision Transformers (ViT) and Attention

Covers (Week 3 Key Concepts, "Vision Transformers (ViT) & Attention") plus the course's two ViT notebooks (`CV-Week3-vit1/vit2.ipynb` , copies in `course-files/`). This notebook includes and extends all their exercises. **Questions answered here:** ViT 19–20 · Hard 10.

Why something new? CNNs are excellent at **local** patterns (a kernel only sees its small neighbourhood) but struggle with **long-range dependencies**: two related things far apart in an image. **Transformers** were invented for language (2017, "Attention Is All You Need") and use **attention** to connect *any* element with *any other*. The **Vision Transformer (2020)** applies this directly to images.

	CNN	ViT
Looks at	local neighbourhoods first, grows the view layer by layer	**all patches at once**, from layer 1
Built-in assumption	"nearby pixels belong together" (locality)	almost none, learns relationships from data
Data needed	works well on small datasets	needs **large** datasets (or pretraining) to shine

```
%matplotlib inline
import os
import time
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import torch
import torch.nn as nn
import torch.nn.functional as F
from PIL import Image

torch.manual_seed(0)
np.set_printoptions(precision=2, suppress=True)
print("torch", torch.__version__)
```

```
torch 2.14.1+cu130
```

1 · Step 1: split the image into patches ("tokens")

A sentence is a sequence of **words**. ViT turns an image into a sequence of **patches**. With a 128×128 image and 16×16 patches you get 8 × 8 = **64 patches**. Each patch is **flattened** into a vector and treated like a word, called a **token**.

The course notebooks load `cat.jpg` , which isn't in the zip. We use the Week-1 photo and fall back to a drawn image if it's missing.

```
path = "../../week-1/code/sample.jpg"
if os.path.exists(path):
    img = Image.open(path).convert("RGB").resize((128, 128))
else:
    arr = np.ones((128, 128, 3)); arr[32:96, 32:96] = [0.2, 0.6, 0.9]
    img = Image.fromarray((arr * 255).astype(np.uint8))
```

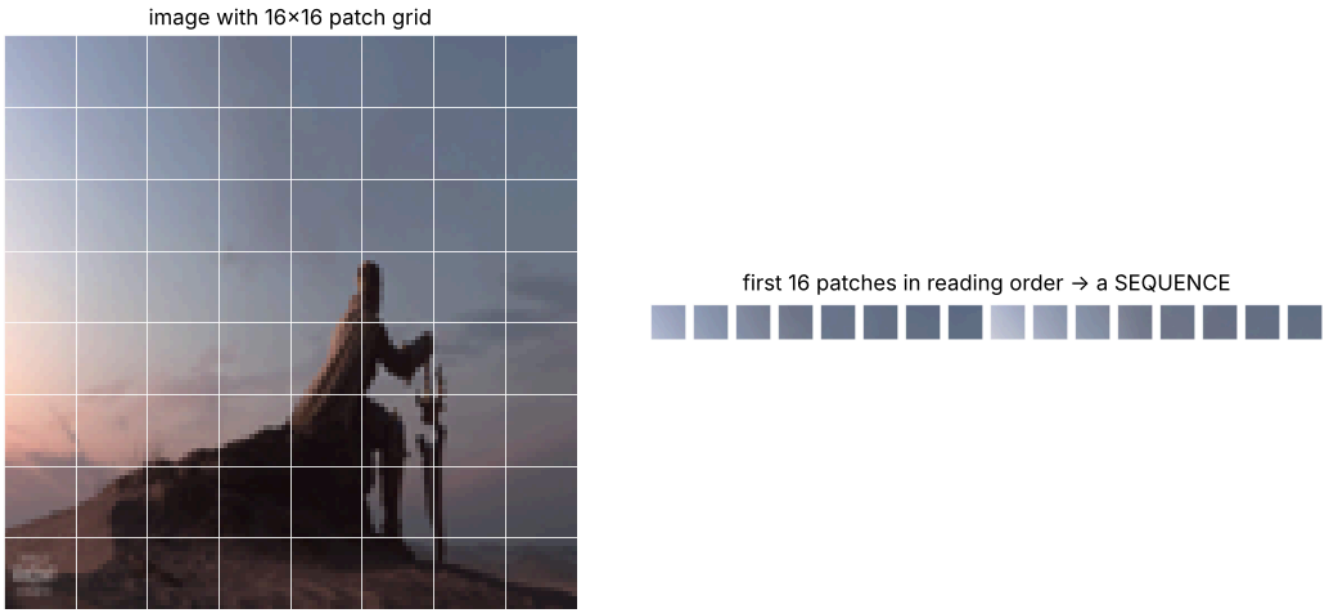
```
img_array = np.array(img)

def split_into_patches(image, patch_size):
    # same function as the course notebook: walk the image in patch_size steps
    patches = []
    for i in range(0, image.shape[0], patch_size):
        for j in range(0, image.shape[1], patch_size):
            patches.append(image[i:i + patch_size, j:j + patch_size])
    return np.array(patches)

patch_size = 16
patches = split_into_patches(img_array, patch_size)
tokens = patches.reshape(patches.shape[0], -1)              # flatten every patch into one long
vector
print("image shape:", img_array.shape)
print(f"{len(patches)} patches of {patch_size}x{patch_size}x3  ->  token matrix {tokens.shape}
(64 tokens, each 768 numbers)")
print("first token (first 12 values):", tokens[0][:12])

fig, ax = plt.subplots(1, 2, figsize=(11, 5))
ax[0].imshow(img_array); ax[0].set_title("image with 16x16 patch grid"); ax[0].axis("off")
for k in range(0, 129, patch_size):
    ax[0].axhline(k - 0.5, color="white", lw=0.6); ax[0].axvline(k - 0.5, color="white", lw=0.6)
strip = np.concatenate([np.pad(p, ((2, 2), (2, 2), (0, 0)), constant_values=255) for p in
patches[:16]], axis=1)
ax[1].imshow(strip); ax[1].set_title("first 16 patches in reading order -> a SEQUENCE");
ax[1].axis("off")
plt.tight_layout(); plt.show()
```

```
image shape: (128, 128, 3)
64 patches of 16x16x3  ->  token matrix (64, 768)  (64 tokens, each 768 numbers)
first token (first 12 values): [165 174 200 164 171 199 163 170 198 161 168 196]
```



Patch size controls the sequence length (course vit2 exercise)

```
fig, ax = plt.subplots(1, 3, figsize=(12, 4))
for a, ps in zip(ax, [8, 16, 32]):
    n = len(split_into_patches(img_array, ps))
    a.imshow(img_array); a.axis("off")
    for k in range(0, 129, ps):
        a.axhline(k - 0.5, color="white", lw=0.5); a.axvline(k - 0.5, color="white", lw=0.5)
    a.set_title(f"patch {ps}x{ps} -> {n} tokens")
    print(f"patch {ps:>2}x{ps:<2} -> {n:>3} tokens -> attention compares {n}x{n} = {n * n:>6,}
pairs")
plt.tight_layout(); plt.show()
```

```
patch  8x8  -> 256 tokens -> attention compares 256x256 = 65,536 pairs
patch 16x16 ->  64 tokens -> attention compares 64x64 =  4,096 pairs
patch 32x32 ->  16 tokens -> attention compares 16x16 =    256 pairs
```



Trade-off: smaller patches keep more detail but create more tokens, and attention compares **every token with every other**, so the cost grows with the **square** of the token count. Working pixel by pixel (16,384 "tokens" for 128×128) would mean 268 million pairs. **Patches make transformers feasible on images.**

2 · Step 2: embeddings, position and the [CLS] token

1. **Patch embedding:** a learned linear layer maps each flattened patch (768 numbers) to a smaller vector (e.g. 64), the "word embedding" of the patch.
2. **Position embedding:** attention itself has **no idea of order**. Shuffle the patches and it gives the same result. So a learned vector per **position** is added, telling the model *where* each patch came from.
3. **[CLS] token:** an extra learned token placed at the start of the sequence. It owns no patch. Through attention it **gathers a summary of the whole image**, and its final value is used for classification ("cat" vs "dog").

3 · Step 3: attention, Query, Key and Value (Q19)

Every token produces three vectors through three learned linear layers:

	Plain meaning	Library analogy
Query (Q)	what am *I* looking for?	your search request
Key (K)	what do *I* contain or offer?	the label on each book
Value (V)	the information I'll hand over if picked	the book's content

Process:

1. Compare each Query with every Key (dot product) to get **similarity scores**.
2. Divide by √d (keeps the numbers in a sensible range) and apply **softmax**, so each token's scores become **weights that sum to 1**.
3. Output = the **weighted sum of the Values**. Each token now carries information **mixed in from the tokens it found relevant**.

Attention(Q, K, V) = softmax(Q·Kᵀ / √d) · V

The course's toy demo (vit2, part 3), explained

The course notebook uses the tokens themselves as Q, K and V (no learned projections), which is the simplest possible attention:

```
torch.manual_seed(0)
tokens = torch.randn(4, 3)                          # 4 patches, each a 3-number embedding
print("tokens (patch embeddings):\n", tokens.numpy())

scores = tokens @ tokens.T                          # every patch compared with every patch (dot
product)
print("\nraw attention scores (4x4):\n", scores.numpy())

weights = torch.softmax(scores, dim=-1)             # each ROW sums to 1
print("\nattention weights (rows sum to 1):\n", weights.numpy(), "\nrow sums:",
weights.sum(1).numpy())

attended = weights @ tokens                         # weighted mix of the tokens
print("\nattended representations (each patch now mixes info from the others):\n",
attended.numpy())
```

```
tokens (patch embeddings):
 [[ 1.54 -0.29 -2.18]
 [ 0.57 -1.08 -1.4 ]
 [ 0.4   0.84 -0.72]
 [-0.4  -0.6   0.18]]

raw attention scores (4x4):
 [[ 7.21  4.24  1.94 -0.84]
 [ 4.24  3.46  0.33  0.16]
 [ 1.94  0.33  1.38 -0.79]
 [-0.84  0.16 -0.79  0.55]]

attention weights (rows sum to 1):
 [[0.95 0.05 0.   0.  ]
 [0.67 0.31 0.01 0.01]
 [0.55 0.11 0.31 0.04]
 [0.11 0.31 0.12 0.46]]
row sums: [1. 1. 1. 1.]

attended representations (each patch now mixes info from the others):
 [[ 1.49 -0.33 -2.13]
 [ 1.21 -0.52 -1.89]
 [ 1.01 -0.04 -1.56]
 [ 0.21 -0.54 -0.68]]
```

Read row *i* of the weights as "how much patch *i* listens to each patch". Notice row 2: patch 2 listens more to patch 0 (0.55) than to itself (0.31). A dot product grows with vector **length** as well as direction, so a long vector like token 0 dominates everyone's attention. **Real attention uses three separate learned projections** (W_Q, W_K, W_V), so a token can *ask for* something different from what it *offers*, and the network **learns** which relationships matter.

The "bank" example: context changes meaning

In language, "**bank**" means different things in "*river bank*" and "*money bank*". Attention lets the "bank" token pull in information from the context word. Below we hand-make tiny 2-D embeddings (dimension 1 = "nature-ness", dimension 2 = "finance-ness") and watch the output for "bank" shift toward its context:

```
vocab = {"river": [1.0, 0.0], "money": [0.0, 1.0], "the": [0.1, 0.1], "bank": [0.5, 0.5]}
W_q = torch.eye(2) * 3; W_k = torch.eye(2) * 3; W_v = torch.eye(2)       # simple "learned"
projections

def attend(sentence):
    X = torch.tensor([vocab[w] for w in sentence])
    Q, K, V = X @ W_q, X @ W_k, X @ W_v
    A = torch.softmax(Q @ K.T / np.sqrt(2), dim=-1)
    return A, A @ V

for sentence in [["the", "river", "bank"], ["the", "money", "bank"]]:
    A, out = attend(sentence)
    b = sentence.index("bank")
    weights = {w: round(float(v), 2) for w, v in zip(sentence, A[b])}
    print(f"{' '.join(sentence):<16} 'bank' attends to {weights}"
          f" -> new 'bank' vector {out[b].numpy()}  (nature, finance)")
```

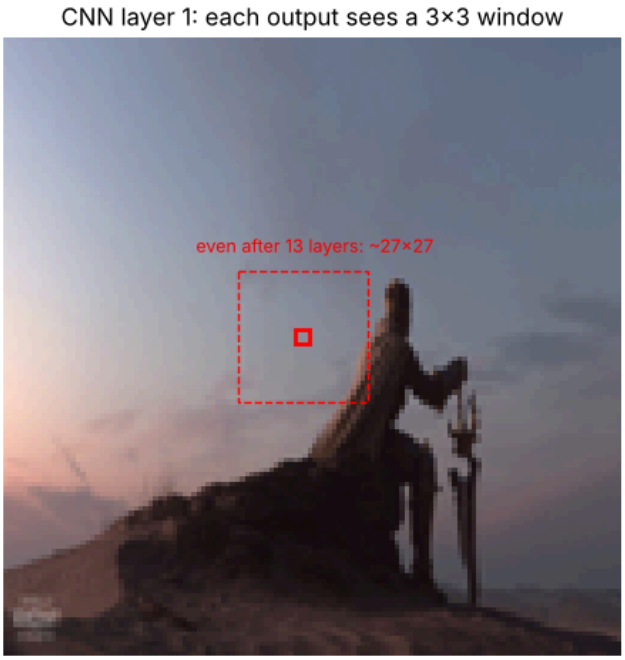
```
the river bank   'bank' attends to {'the': 0.04, 'river': 0.48, 'bank': 0.48} -> new 'bank' vecto
r [0.73 0.24]  (nature, finance)
the money bank   'bank' attends to {'the': 0.04, 'money': 0.48, 'bank': 0.48} -> new 'bank' vecto
r [0.24 0.73]  (nature, finance)
```

Same word, same starting vector [0.5, 0.5], but after attention "bank" leans toward **nature** in one sentence and **finance** in the other. In a ViT the same thing happens with patches: a patch of stripes can attend to other stripe patches far away and "realise" it's part of a zebra.

4 · Why patches + attention capture global context (Q20)

1. **The image becomes a sequence:** patches act like words, so the transformer can relate any two regions.
2. **Long-range dependencies from layer 1:** a CNN neuron in the first layer sees 3×3 pixels, and it takes many layers before distant regions meet. With attention, the top-left patch can directly attend to the bottom-right patch **in the very first layer**.
3. **Reduced complexity:** patches shorten the sequence enough to make attention affordable.
4. **Local and global together:** each patch embedding keeps **local** detail (edges, texture inside the patch), and attention across patches builds the **global** picture (object shape, scene layout).

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4.8))
ax[0].imshow(img_array); ax[0].set_title("CNN layer 1: each output sees a 3x3 window");
ax[0].axis("off")
ax[0].add_patch(mpatches.Rectangle((60, 60), 3, 3, fill=False, ec="red", lw=2))
ax[0].add_patch(mpatches.Rectangle((48, 48), 27, 27, fill=False, ec="red", lw=1, ls="--"))
ax[0].text(64, 44, "even after 13 layers: ~27x27", color="red", ha="center", fontsize=9)
ax[1].imshow(img_array); ax[1].set_title("ViT layer 1: every patch can attend to every patch");
ax[1].axis("off")
centre = (72, 72)
for (r, c) in [(0, 0), (0, 7), (7, 0), (7, 7), (2, 5), (5, 1)]:
    ax[1].annotate("", xy=(c * 16 + 8, r * 16 + 8), xytext=centre, arrowprops=dict(arrowstyle="-
>", color="cyan", lw=1.5))
ax[1].add_patch(mpatches.Rectangle((64, 64), 16, 16, fill=False, ec="cyan", lw=2))
plt.tight_layout(); plt.show()
```





5 · Build and train a tiny Vision Transformer

Our ViT, sized for a laptop: 16×16 digit images → **4×4 patches = 16 tokens** + [CLS], embedding size 64, **2 transformer blocks** with 4 attention heads. Each block is:

```
x → LayerNorm → multi-head attention —(+ skip)→ LayerNorm → MLP —(+ skip)→
```

Notice the **skip (residual) connections**: they're straight from ResNet (notebook 03). **Multi-head** means several attention "heads" run in parallel, each free to learn a different kind of relationship.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

digits = load_digits()
X = F.interpolate(torch.tensor(digits.images, dtype=torch.float32)[:, None] / 16, size=(16, 16),
mode="bilinear", align_corners=False)
y = torch.tensor(digits.target)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)

class Block(nn.Module):
    def __init__(self, dim, heads):
        super().__init__()
        self.norm1, self.norm2 = nn.LayerNorm(dim), nn.LayerNorm(dim)
        self.attn = nn.MultiheadAttention(dim, heads, dropout=0.1, batch_first=True)
        self.mlp = nn.Sequential(nn.Linear(dim, dim * 2), nn.GELU(), nn.Dropout(0.1),
nn.Linear(dim * 2, dim))
        self.last_attention = None
    def forward(self, x):
        h = self.norm1(x)
        a, w = self.attn(h, h, h, need_weights=True)        # Q, K and V all come from the tokens
        self.last_attention = w.detach()                    # (batch, tokens, tokens), averaged
over heads
        x = x + a                                           # residual connection
        return x + self.mlp(self.norm2(x))                  # residual connection

class TinyViT(nn.Module):
    def __init__(self, patch=4, dim=64, depth=2, heads=4, n_classes=10):
        super().__init__()
        self.patch = patch
        n_patches = (16 // patch) ** 2
        self.embed = nn.Linear(patch * patch, dim)                          # patch embedding
```

```
        self.cls = nn.Parameter(torch.zeros(1, 1, dim))                    # [CLS] token
        self.pos = nn.Parameter(torch.randn(1, n_patches + 1, dim) * 0.02)  # position embeddings
        self.blocks = nn.ModuleList([Block(dim, heads) for _ in range(depth)])
        self.norm = nn.LayerNorm(dim)
        self.head = nn.Linear(dim, n_classes)
    def forward(self, x):
        B = x.shape[0]
        p = x.unfold(2, self.patch, self.patch).unfold(3, self.patch, self.patch)   # cut into
4x4 patches
        p = p.reshape(B, -1, self.patch * self.patch)                               # (B, 16
tokens, 16 values)
        t = torch.cat([self.cls.expand(B, -1, -1), self.embed(p)], dim=1) + self.pos
        for blk in self.blocks:
            t = blk(t)
        return self.head(self.norm(t)[:, 0])                                        # classify
from [CLS]

def small_cnn():
    return nn.Sequential(nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
                         nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
                         nn.Flatten(), nn.Linear(64 * 16, 64), nn.ReLU(), nn.Linear(64, 10))

def fit(model, epochs=40):
    opt = torch.optim.AdamW(model.parameters(), lr=2e-3, weight_decay=0.05)
    for _ in range(epochs):
        model.train(); perm = torch.randperm(len(Xtr))
        for i in range(0, len(Xtr), 64):
            idx = perm[i:i + 64]
            loss = F.cross_entropy(model(Xtr[idx]), ytr[idx])
            opt.zero_grad(); loss.backward(); opt.step()
    model.eval()
    with torch.no_grad():
        return (model(Xte).argmax(1) == yte).float().mean().item()

results = {}
for name, make in [("Tiny ViT", TinyViT), ("Small CNN", small_cnn)]:
    torch.manual_seed(0)
    model = make()
    t = time.time(); acc = fit(model)
    results[name] = model
    print(f"{name:<10} parameters {sum(p.numel() for p in model.parameters()):>7,}   test
accuracy {acc:.1%}   ({time.time() - t:.0f} s)")
vit = results["Tiny ViT"]
```

```
Tiny ViT   parameters  69,962   test accuracy 96.9%   (13 s)
Small CNN  parameters  85,066   test accuracy 98.1%   (5 s)
```

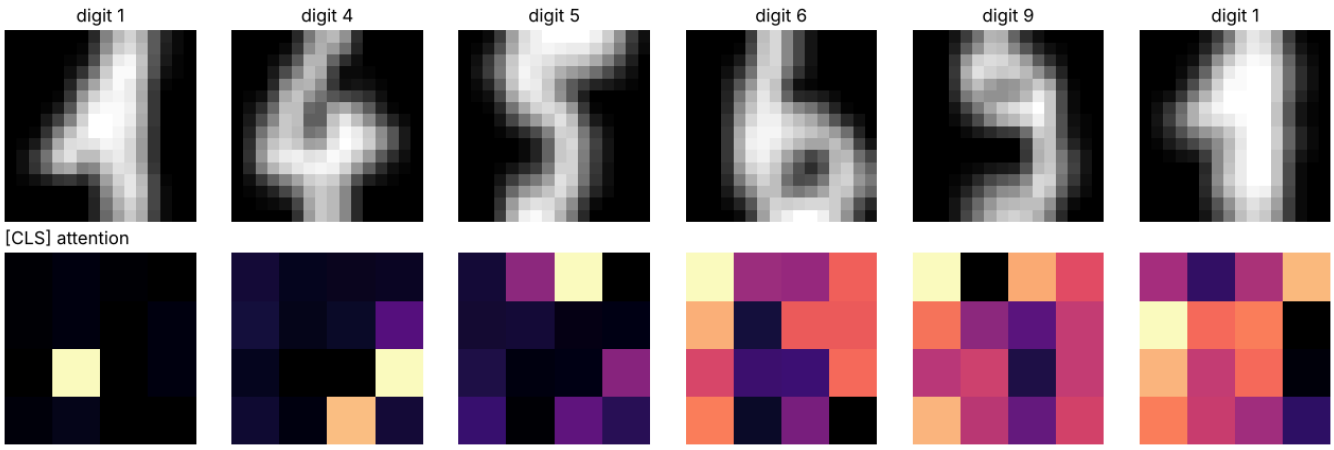
The ViT learns, but the similar-sized CNN wins on this small dataset. That's expected and worth remembering. A CNN has a built-in assumption ("nearby pixels belong together, and patterns can appear anywhere"), which is exactly right for images and saves it from learning that from data. A ViT has to **learn** such relationships, so it needs **much more data**. The original ViT only beat the best CNNs after pretraining on hundreds of millions of images. That's the "**scales well with large datasets**" benefit in the notes.

What does the [CLS] token look at?

The attention weights from [CLS] to the 16 patches, in the last block, show which parts of the digit the model consulted:

```
with torch.no_grad():
    vit(Xte[:6])
cls_attn = vit.blocks[-1].last_attention[:, 0, 1:]          # row 0 = [CLS]; skip its attention
to itself
fig, ax = plt.subplots(2, 6, figsize=(13, 4.4))
```

```
for k in range(6):
    ax[0, k].imshow(Xte[k, 0], cmap="gray"); ax[0, k].set_title(f"digit {int(yte[k])}"); ax[0,
k].axis("off")
    ax[1, k].imshow(cls_attn[k].reshape(4, 4), cmap="magma"); ax[1, k].axis("off")
ax[1, 0].set_title("[CLS] attention", loc="left")
plt.tight_layout(); plt.show()
```



Bright cells are the patches [CLS] drew information from in the last block. For some digits the attention is sharply focused on one or two patches of the stroke. For others it's spread widely, background included, because the earlier block has already mixed information between patches, so a "background" token may by now carry stroke information. Attention maps of real ViTs are similarly suggestive but not a perfect explanation of the model. The key point: the model chooses **where to look** by itself, with no convolution anywhere.

6 · Benefits, applications and the big picture

Benefits: captures **global context** better than CNNs · **scales well** with large datasets · flexible (classification, detection, segmentation) · inspired **hybrid** CNN + Transformer models. **Applications:** image classification (ImageNet benchmarks) · medical imaging (global patterns across a scan) · video understanding (adding attention across time) · multimodal vision + language (image captioning, CLIP-style search). **Impact:** shifted computer vision from **CNN-centric** to **transformer-centric** approaches.

From LeNet to ViT (Hard Q10)

Step	What it contributed to ViT
LeNet	learning features end to end from pixels (no hand-made features)
AlexNet	scale: big data (ImageNet) + GPUs + ReLU/dropout made big models trainable
VGG	depth and simple, repeated building blocks
ResNet	**residual connections** (in every transformer block) and very deep training; normalization
EfficientNet	systematic thinking about **scaling** model and data size
ViT	keeps the deep, residual, scaled-up recipe but swaps local convolution for **global attention** over patches

✅ Check yourself

1. Purpose of Query, Key and Value? *(Query = what a token looks for; Key = what each token offers; Value = information passed on; output = weights from Q·K, applied to V)*
2. Why split an image into patches? *(turns the image into a token sequence, keeps the sequence short enough for attention, and lets distant regions interact directly, giving global context)*
3. What is the [CLS] token for? *(a learned summary token whose final value is used for classification)*
4. Why add position embeddings? *(attention ignores order; positions tell the model where each patch was)*
5. Why might a CNN beat a ViT on a small dataset? *(the CNN's built-in locality assumption; ViTs must learn it from lots of data)*
6. Patch size 16 on a 224×224 image gives how many tokens? *($(224/16)^2 = 196$, plus [CLS] = 197)*

Week 2 & 3 Question Bank: Model Answers

Answers to every question in **CV-Week2-3-Questions.pdf** (original in `course-files/`). Each answer is eval-length and points to the notebook (`notebooks/0X_…`) where you can **see it happen**.

> Cover the answer, say yours out loud, then check. For "Hard" questions, aim for 5–8 lines with an example.

Easy

1. Who invented LeNet and what was its primary application?
Yann LeCun (Bell Labs, LeNet-5 in 1998). Primary application: **handwritten digit recognition**, used for reading **bank cheques and postal (zip) codes**. → nb 02

2. What dataset was LeNet trained on?
MNIST: 60,000 training + 10,000 test images of handwritten digits, 28×28 grayscale (padded to 32×32 for LeNet).

3. What was AlexNet's biggest achievement in 2012?
It **won the ImageNet competition (ILSVRC 2012) by a huge margin**: about 15.3% top-5 error vs about 26% for the runner-up. That **sparked the deep-learning revolution** in computer vision.

4. Name two innovations introduced in AlexNet.
Any two of: **ReLU activation** (faster training), **dropout** (less overfitting), **GPU training** (massive speed-up). Also heavy data augmentation and overlapping max-pooling.

5. What activation function replaced sigmoid/tanh in AlexNet?
ReLU (Rectified Linear Unit), f(x) = max(0, x). → nb 01, 02

6. What does ReLU do to negative input values?
Sets them to **zero**. Positive values pass through unchanged.

7. What is the purpose of dropout?
To **prevent overfitting** (a regularization technique). During training it randomly switches off (zeros) a fraction of neurons each pass, so the network can't rely on a few specific neurons and must learn robust, distributed features. It's switched off at test time. → nb 01

8. How many layers does VGG typically have?
16 to 19 weight layers (VGG-16: 13 conv + 3 FC; VGG-19: 16 conv + 3 FC).

9. What filter size did VGG standardize?
Small **3×3** convolution filters, stacked one after another.

10. What problem does ResNet solve?
Training **very deep networks**: the **vanishing gradient / degradation** problem. Plain networks got *worse* (even on training data) as layers were added, because gradients faded before reaching the early layers. → nb 03

11. What is the key innovation in ResNet?
Residual (skip) connections: each block outputs **F(x) + x**, so it learns only the *residual* F(x) = H(x) − x, and gradients flow through the identity path.

12. What does EfficientNet's compound scaling balance?
The network's **depth** (layers), **width** (channels) and input **resolution**, scaled **together** with one coefficient φ: depth α^φ, width β^φ, resolution γ^φ.

13. Which CNN architecture enabled training of networks with 152 layers?
ResNet (ResNet-152).

14. Which architecture is lightweight yet accurate for mobile vision apps?
EfficientNet (e.g. EfficientNet-B0: about 5.3 M parameters, more accurate than ResNet-50). **MobileNet** is the other classic answer; EfficientNet is built from its depthwise-separable blocks.

15. What does object detection aim to identify in an image?
What each object is (class label) and **where** it is (bounding box), for **every** object in the image.

16. What does R-CNN stand for?
Region-based Convolutional Neural Network.

17. Why is YOLO considered fast?
It looks at the image **once**: a single forward pass of one network predicts all bounding boxes and class probabilities together (detection as one regression problem). There's no separate stage of thousands of region proposals. → nb 04 (our toy YOLO was over 100× faster than our toy R-CNN)

18. What does SSD stand for?
Single Shot MultiBox Detector.

19. What is semantic segmentation?
Assigning a class label to every pixel in an image (e.g. road / car / person / sky), giving a dense label map. It doesn't separate individual objects of the same class. → nb 05

20. What is the role of the encoder in U-Net?
The **downsampling path**: convolution + pooling progressively **reduce resolution** while building **context and high-level features** ("what is in the image"). Like compressing a photo into a summary.

Medium

1. How did LeNet prove CNNs were better than traditional ML methods?
On MNIST, LeNet-5, trained **end to end on raw pixels**, matched or beat the best traditional methods of the time (k-NN, SVMs on hand-crafted features) at about 0.95% error. It **learned its own features** (strokes, edges) instead of relying on human-designed ones, and was far faster to run. This led to real deployments reading cheques and post. *(Nuance: the best SVMs were close on MNIST. The decisive gap appeared with large, complex images, e.g. AlexNet on ImageNet. See nb 02.)*

2. Why was GPU training critical for AlexNet's success?
Training a CNN is **millions of independent multiply-adds** (every kernel at every position, for 1.2 M images, over many epochs). GPUs have **thousands of parallel cores**, so they ran this **many times faster** than CPUs. AlexNet (about 61 M parameters) trained in about a week on two GTX 580 GPUs. On CPUs it would have taken months, making the model impractical. → nb 02 (timing demo)

3. Compare accuracy before and after AlexNet on ImageNet.
Before (2011, hand-crafted features): about **74% top-5 accuracy** (about 26% error). **AlexNet (2012): about 85%** (15.3% error), a jump of more than 10 points in one year. The course notes summarise it as "~70% → ~85%+".

4. Why does stacking small filters (like 3×3 in VGG) improve performance?
1. **Same receptive field, fewer parameters:** two 3×3 layers see 5×5, and three see 7×7, using $27C^2$ weights instead of $49C^2$ (about 45% fewer). 2. **More non-linearity:** each layer adds a ReLU, so three layers give three non-linearities instead of one, and the network can learn more complex features. 3. The **simple, uniform** design makes deeper networks easier to build. → nb 02 (receptive-field demo)

5. How do residual connections help gradient flow in ResNet?
Output = F(x) + x, so the gradient is $\partial out/\partial x = F'(x) + 1$. The "+1" from the identity path passes the gradient straight back through every block, even when F′(x) is tiny. In a plain net the gradient is a **product** of many small factors and vanishes. → nb 03 (plain 52-layer net: gradient at layer 1 = 0; residual: about 0.02)

6. What is the difference between direct mappings and residuals?
- A **direct mapping**: the layers must learn the full desired output **H(x)** themselves. - A **residual**: the layers learn only the **difference** F(x) = H(x) − x, and the input is added back.

Learning "change nothing" is easy for a residual block (F = 0) but hard for a plain one (it must learn the identity). So adding residual layers never makes the network worse.

7. Why is EfficientNet more efficient than simply making networks deeper or wider?
Depth, width and resolution **depend on each other**. Higher resolution needs more depth (a larger receptive field) and more width (more fine-grained features). Scaling just one hits **diminishing returns**: lots of extra computation for little accuracy. **Compound scaling** grows all three in a fixed, balanced ratio ($\alpha = 1.2$, $\beta = 1.1$, $\gamma = 1.15$; each φ step about doubles FLOPs), so extra compute goes where it helps most. EfficientNet-B7 matched the best accuracy of its time with about 8× fewer parameters.

8. Give one real-world application of ResNet.
Medical imaging, e.g. classifying chest X-rays (pneumonia), skin-lesion images or retinal scans, usually an ImageNet-pretrained ResNet fine-tuned on medical data. ResNet is also the **backbone** of Faster/Mask R-CNN and of face-recognition models.

9. Give one real-world application of EfficientNet.
On-device mobile vision, e.g. a phone app that identifies plant diseases, food or products from the camera in real time, without a server. Also drones and embedded cameras.

10. Why is R-CNN slow compared to YOLO?
R-CNN generates about **2,000 region proposals** per image and runs the CNN **separately on**

each (plus a separate classification/regression stage), so it's thousands of forward passes per image. YOLO does **one** forward pass for the whole image. → nb 04

11. How does YOLO treat detection differently from R-CNN?
R-CNN is **two-stage**: propose regions, then classify each. YOLO is **one-stage**: it treats detection as a **single regression problem**. The image is divided into an S×S grid, and each cell directly predicts boxes, confidence and class probabilities in one pass, using **global context** from the whole image.

12. Why is SSD better at detecting objects of different sizes?
It makes predictions from **multiple feature maps at different scales**. Early, high-resolution maps (small cells) detect **small** objects, and later, low-resolution maps (big cells) detect **large** ones, each with default boxes of several shapes. So a small cat and a large cat in one image are both found in a single pass. → nb 04

13. Which detection architecture for medical imaging, and why?
The **R-CNN family** (Faster R-CNN, or **Mask R-CNN** for exact outlines). Medical tasks need **high accuracy and precise localization**, missing a tumour (a false negative) is unacceptable, and analysis is **offline**, so speed matters less. Example: finding small tumours in MRI scans.

14. Which detection architecture for autonomous driving, and why?
YOLO (or SSD). Cars need **real-time** detection (30+ fps) of pedestrians, vehicles and signs. YOLO's single pass is fast enough, and a slightly less precise box *now* beats a perfect one too late. SSD balances speed with multi-scale accuracy (distant small cars).

15. What challenge does semantic segmentation face compared to classification?
It needs both **context** (*what* is it?, which requires a large receptive field) **and precision** (*exactly where* is each boundary?, which requires fine spatial detail). Downsampling helps the first and destroys the second. Classification needs only one label per image, so it can discard spatial detail freely. Segmentation also needs pixel-level labels, which are expensive to annotate.

16. How does FCN differ from traditional CNNs in its output?
A traditional CNN ends with **fully connected layers** that output **one label for the whole image** (spatial layout lost). An FCN replaces them with **convolutional layers** and outputs a **dense pixel-wise prediction map**, a class score for every pixel, for inputs of any size. → nb 05

17. Why are FCN predictions often coarse?
The network predicts at a **low resolution** after repeated pooling (e.g. 1/32 of the input), then simply **upsamples** that small map. Fine detail lost in pooling can't be recovered, so boundaries are blurry and small objects are missed. → nb 05 (FCN 77% vs U-Net 99% accuracy on boundary pixels)

18. How do skip connections in U-Net improve segmentation accuracy?
They **copy high-resolution feature maps from each encoder level** and **concatenate** them with the decoder at the same resolution. The decoder then gets back the **fine spatial details** (edges, exact positions) that pooling lost, combined with the **high-level context** from deep layers, which gives sharp, accurate boundaries and preserves small structures. They also help gradients flow.

19. What is the role of the decoder in U-Net?
The **upsampling path**: it gradually **increases the resolution** back to the input size (transposed convolutions or upsampling), combines the encoder's context with fine details from the skip connections, and produces the **pixel-level prediction**. The encoder learns *what*; the decoder learns *where*.

20. Why is U-Net widely used in medical imaging?
1. **Precise boundaries**: tumour, organ and cell outlines are needed for measurement and treatment planning. 2. Skip connections keep **small structures** (cells, lesions, vessels). 3. It **works with small datasets** (medical labels are scarce and expensive), especially with augmentation. 4. It's simple and fast to train, and handles 2-D and 3-D scans.

Hard

1. Explain how dropout improves generalization, with an example.
During training, dropout randomly zeros a fraction *p* of neurons each pass, so the network **can't depend on any single neuron or fixed co-adapted group**. It must spread knowledge redundantly and learn features that work even with parts missing, which are **robust features** that transfer to unseen data. It's like training many "thinned" networks and averaging them at test time, when all neurons are used and outputs are scaled accordingly. *Example:* a layer of 10 neurons with p = 0.3: about 3 are dropped each pass. If one neuron had learned "background is grass → it's a cow", it's sometimes absent, so other neurons must learn real cow features (shape, head), and the model then works on a cow on a beach. Like a team where random members sit out practice, so everyone learns to contribute. → nb 01 (dropout narrowed the train/test gap)

2. Why did AlexNet's use of ReLU help reduce the vanishing gradient problem?
Backpropagation multiplies the gradient by each layer's activation derivative. **Sigmoid's derivative is at most 0.25** (and near 0 when saturated), and tanh's also approaches 0 for large inputs, so through many layers the product **shrinks exponentially** ($0.25^{10} \approx 10^{-6}$). **ReLU's derivative is exactly 1 for positive inputs**, so the gradient passes through active units **without shrinking**, and it never saturates for positive values. ReLU is also cheap (no exponentials). AlexNet's ReLU network reached its target error about **6× faster** than with tanh. (Caveat: "dead" ReLUs have zero gradient for negative inputs.) → nb 01, 02

3. How does compound scaling in EfficientNet systematically balance depth, width and resolution?
With one user-chosen coefficient **φ** (how much more compute you can afford): - depth d = **α^φ**, width w = **β^φ**, resolution r = **γ^φ**, - subject to **α · β² · γ² ≈ 2** (with α ≥ 1, β ≥ 1, γ ≥ 1).

FLOPs scale with $d \cdot w^2 \cdot r^2$ (width and resolution affect cost quadratically), so each +1 in φ about **doubles the computation**. α, β, γ are found **once** by a small grid search on the baseline B0 (α = 1.2, β = 1.1, γ = 1.15) and then reused to produce B1–B7. Every dimension grows in proportion: higher resolution comes with the extra depth (receptive field) and width (fine features) it needs. → nb 03 (φ table)

4. Why do residual connections allow training very deep networks without degradation?
- **Gradient flow:** out = F(x) + x gives ∂out/∂x = F′(x) + 1. The identity term gives an

uninterrupted gradient highway, so gradients don't vanish across 100+ layers. - **Easy identity:** if extra layers aren't needed, a block just learns F(x) ≈ 0 and passes x through. A deeper ResNet can therefore always do *at least as well* as a shallower one, which removes the **degradation problem** (plain deep nets having higher *training* error than shallower ones). - In practice, combined with BatchNorm, this enabled 50/101/152 layers.

→ nb 03: plain 22- and 52-layer nets stuck at 10% (chance); residual nets of the same depth trained to 66–76%.

5. Compare the trade-off between accuracy and speed in R-CNN, YOLO and SSD.
| | Accuracy | Speed | Why | |---|---|---|---| | **R-CNN** (and Fast/Faster) | **highest**, esp. small objects and cluttered scenes | **slowest** (original R-CNN: tens of seconds per image) | two stages; CNN per region proposal; careful per-region classification and box refinement | | **YOLO** | lower, struggles with **small, crowded** objects and precise localization | **fastest, real-time** (45+ fps in the original) | one pass, coarse grid, few boxes per cell | | **SSD** | competitive; good across object sizes | fast (near real-time) | one pass + **multi-scale** feature maps and default boxes |

Choose by application: accuracy-critical and offline (medical) → R-CNN family; real-time (driving, video) → YOLO; balanced or mobile → SSD.

6. How would you adapt YOLO for detecting very small objects?
- **Higher input resolution** and/or a **finer grid** (more, smaller cells). - **Multi-scale prediction heads with a Feature Pyramid Network** (as in YOLOv3+): detect small objects on high-resolution feature maps. - **More / smaller anchor boxes**, with anchor sizes tuned (k-means) to the small objects in the dataset. - **Tiling / slicing** large images into overlapping crops (drones, satellites), then merging detections. - **Augmentation** (mosaic, copy-paste of small objects, zoom) and **loss re-weighting** for small boxes; avoid excessive early downsampling.

→ nb 04 shows a one-object-per-cell grid losing two small shapes that share a cell.

7. Why are skip connections critical in both ResNet and U-Net, though used differently?
Both send information **around** layers that would otherwise lose it: - **ResNet:** short skips within a block, **added** (F(x) + x). Purpose: **gradient flow and an easy identity**, so very deep networks train without degradation. - **U-Net:** long skips from each **encoder** level to the matching **decoder** level, **concatenated**. Purpose: restore the **fine spatial detail** lost by downsampling, giving precise boundaries (they also help gradients).

Shared principle: let later layers access earlier information directly. ResNet uses it to make *depth* trainable; U-Net uses it to make *pixel precision* possible.

8. How does the encoder–decoder structure in U-Net ensure both context and precision?
- The **encoder** downsamples step by step: each level sees a wider area, building **context** (what the region is: tumour vs tissue). - The **decoder** upsamples back to full resolution to produce a per-pixel map. - At every level, **skip connections** hand the decoder the **high-resolution encoder features** (edges, exact positions), so it combines *what* (deep context) with *where* (shallow detail).

"Encoder compresses for context, decoder reconstructs for precision, skips bridge the two." Example: the encoder recognises that a region is a tumour, and the skips restore its exact boundary. → nb 05

9. Designing a segmentation model for satellite imagery: which architecture and why?
A **U-Net** (typically with an ImageNet-pretrained **ResNet/EfficientNet encoder**), because: - satellite targets (roads, rivers, field and building boundaries) are **thin structures** that need **pixel-level precision**, which U-Net's skip connections preserve, - the encoder's context separates look-alike classes (roof vs road, water vs shadow), - it trains well with limited labelled data and augmentation (rotations and flips are natural for overhead images).

Practicalities: handle huge images by **tiling** with overlap, use multispectral bands as extra input channels, and use Dice/IoU loss for class imbalance. If you must count or separate **individual** buildings or vehicles, use **Mask R-CNN** (instance segmentation) instead or as well.

10. How did the evolution LeNet → AlexNet → VGG → ResNet → EfficientNet shape the foundation for Vision Transformers?
- **LeNet:** learning features **end to end** from pixels. - **AlexNet: scale** (big data + GPUs) plus ReLU and dropout make large models trainable. ViTs depend on huge data and compute. - **VGG: depth** with simple, **repeated identical blocks**. A ViT is a stack of identical transformer blocks. - **ResNet: residual connections** (used in *every* transformer block) and normalization made very deep training reliable. - **EfficientNet:** systematic **scaling** of model size, data and resolution, which also guides how ViT sizes (Base, Large, Huge) are scaled.

ViT keeps this deep, residual, scaled-up recipe but replaces local convolutions with **global self-attention over patches**. The CNN lineage also motivated *why*: CNNs struggle with long-range dependencies, and ViT addresses that. Hybrid CNN + Transformer models continue the line. → nb 07

Mask R-CNN

1. What key limitation of traditional object detection does Mask R-CNN address?
Detectors output only **bounding boxes + labels**, which are rough rectangles. Many tasks need the **exact pixel-level shape of each individual object** (**instance segmentation**). Mask R-CNN adds a mask for every detected object.

2. How does Mask R-CNN extend Faster R-CNN?
It keeps Faster R-CNN (backbone + RPN + classification and box regression) and adds a **parallel mask branch**: a small FCN that predicts a **binary mask for each Region of Interest**. It also replaces **RoI Pooling with RoI Align** for pixel-accurate features.

3. What role does the Region Proposal Network (RPN) play?
It slides over the shared feature map and **proposes candidate object regions** (boxes + "objectness" scores) using anchor boxes. These proposals are what the later heads classify, refine and mask. It replaces slow external proposal methods (selective search) and shares computation with the backbone.

4. Why was RoI Align introduced, and how does it improve on RoI Pooling?
RoI Pooling **rounds (quantizes)** the proposal's fractional coordinates, and its sub-bins, to

whole feature-map cells. This misaligns the extracted features with the actual object, which is harmless for boxes but bad for **pixel-accurate masks** (one feature cell can be 16+ image pixels). **RoI Align removes all rounding**: it samples the feature map at the exact fractional positions with **bilinear interpolation**, so features line up with the object and mask accuracy improves noticeably. → nb 05 (numeric demo)

5. What are the three outputs produced for each detected object?
1. **Bounding box** (where). 2. **Class label** (what). 3. **Pixel-level mask** (its exact shape).

6. Why are pixel-level masks more useful than bounding boxes in medical imaging?
1. **Precise localization:** tumours and organs are irregular, and a box includes healthy tissue. 2. **Quantitative analysis:** masks give the true **area and volume** (e.g. tumour volume for chemotherapy dosage), while boxes overestimate. 3. **Clinical decisions:** surgery and radiotherapy planning and disease monitoring need exact borders and spread. 4. **Overlaps:** masks separate structures within structures (a tumour inside an organ).

7. Give two real-world applications of Mask R-CNN outside medical imaging.
Any two of: - **autonomous driving** (pedestrian, vehicle and lane shapes) - **crowd analysis** (separating overlapping people) - **robotics grasping** (exact object outline) - **photo/video editing** (cut out a person, background replacement) - **agriculture** (individual fruit or leaf counting) - **retail** (products on shelves) - **satellite imagery** (individual buildings)

8. How does Mask R-CNN handle overlapping objects in crowded scenes?
Each object is a separate **RoI proposal**, and each RoI gets **its own class and its own binary mask**, predicted inside its own box. Overlapping people become separate instances with separate masks, even where they touch. Masks are predicted **per class and independently** (no competition between classes), and NMS removes duplicate proposals of the same object. Semantic segmentation would merge them into one "person" blob. → nb 05 (touching-coins demo)

Sequence modeling for video (CNN + LSTM)

9. Why can't CNNs alone capture temporal dependencies in video?
A CNN processes **each frame independently** and has no memory of previous frames and no notion of order. It can recognise *what* is in a frame (a hand) but not *how* it changes (a wave). Shuffle the frames and a per-frame CNN gives the same answers. → nb 06 (order-blind models fail to tell left from right)

10. What specific problem do LSTMs solve compared to vanilla RNNs?
The **vanishing gradient** problem over long sequences, which gives RNNs only **short-term memory**. LSTMs add a **cell state** and **gates**, so information (and gradient) can be carried across many time steps: **long-term dependencies**. → nb 06 (at 50 steps: RNN 25% = chance, LSTM 99.8%)

11. Explain the input, forget and output gates.
Each gate is a sigmoid layer giving values from 0 to 1: - **Forget gate**: decides what **past information to discard** from the cell state (1 = keep, 0 = erase). - **Input gate**: decides what **new information to store** in the cell state. - **Output gate**: decides what part of the memory to **pass forward** as the hidden state / output now.

$c_t = f \odot c_{t-1} + i \odot \tilde{c}_t$; $h_t = o \odot \tanh(c_t)$.

12. In a CNN + LSTM pipeline, what does each component contribute?
- **CNN:** extracts **spatial features** from each frame (objects, shapes, edges, textures): the *what*. - **LSTM:** processes the **sequence** of per-frame feature vectors and models **temporal dependencies** (motion, order, speed, rhythm): the *how / when*. - **Output layer:** predicts the action or gesture class.

"Photographer (CNN) + storyteller (LSTM)."

13. Give an example of an application where CNN + LSTM is essential.
Gesture or sign-language recognition: the CNN detects the hand shape in each frame, and the LSTM recognises the movement sequence ("wave", "swipe", a sign). Others: action recognition in sports or surveillance ("throwing a ball"), video captioning, predicting pedestrian motion for self-driving cars.

14. How does sequence modeling enable gesture recognition compared to static image classification?
A gesture is defined by **movement over time**, while a single frame only shows a **pose**: "raising a hand" and "waving" can share the same frame, and "swipe left" vs "swipe right" contain the same frames in **reverse order**. A static classifier sees one frame and can't tell them apart. A sequence model (LSTM) remembers previous frames and recognises the **trajectory, order and speed** of the motion. → nb 06

RNNs vs LSTMs

15. What is the vanishing gradient problem, and why does it affect RNNs more than LSTMs?
During backpropagation (through time), the gradient is **multiplied by a factor at each step or layer**. If those factors are < 1 (tanh derivatives, small recurrent weights) it **shrinks exponentially**, so early inputs get almost no learning signal and the model "forgets" them. A vanilla RNN passes its memory through **tanh and the same weight matrix at every step**, so after 20–50 steps the gradient vanishes. An LSTM updates its cell state **additively** ($f \odot c + i \odot \tilde{c}$). When the forget gate is about 1, memory and gradient flow through almost unchanged (the "constant error carousel"). → nb 06 (gradient reaching step 1: RNN 4e-14 vs LSTM 4e-5)

16. Compare RNNs and LSTMs in terms of memory and best use cases.
| | RNN | LSTM | |---|---|---| | Memory | **short-term** (recent steps) | **long-term** (long dependencies) | | Gradient | vanishes over long sequences | gates largely solve it | | Cost | small, fast | about 4× parameters, slower | | Best for | short, simple sequences | long, complex sequences: video, paragraphs, speech | | Example | next word in a short sentence | context across a long paragraph or video |

17. Why are LSTMs better suited for variable-length sequences like video?
Both process one step at a time with shared weights, so any length is *possible*. But only the LSTM keeps relevant information over **long and varying spans**: the gates learn what to keep and what to forget regardless of clip length, so a 10-second gesture is handled as well as a 2-second one. An RNN's memory fades after a few steps, so longer clips lose their beginning.

18. Give an example of a task where RNNs might still be sufficient instead of LSTMs.
Short sequences where only recent context matters: **predicting the next character in short**

words, **smoothing or forecasting a sensor reading from the last few values**, **predicting the next note from the last few notes**, or a simple short-gesture detector over a handful of frames. RNNs are smaller and faster there. (GRUs are a lighter middle ground.)

Vision Transformers & Attention

19. What is the purpose of Query, Key and Value in attention?
Each token produces three learned vectors: - **Query** = *what I'm looking for*, - **Key** = *what I contain / can be matched on*, - **Value** = *the information I pass on*.

Attention compares each **Query with all Keys** (dot products) to get similarity scores, normalizes them with softmax into weights, and outputs the **weighted sum of the Values**: each token gathers information from the tokens most relevant to it. Attention(Q, K, V) = softmax($QK^T/\sqrt{d}$)·V. Example: "bank" attends to "river" or "money" to resolve its meaning. → nb 07

20. Why does splitting an image into patches help ViTs capture global context?
1. It turns the image into a **sequence of tokens** (like words), so the transformer's **self-attention** can relate **any patch to any other**, even far-apart ones, **from the first layer**. A CNN neuron only sees a small local window and needs many layers to connect distant regions. 2. It **reduces complexity**: attention cost grows with the square of the sequence length, and 196 patches instead of about 50,000 pixels makes it feasible. 3. It **balances local and global**: each patch embedding holds local detail, and attention across patches builds the global picture (e.g. stripes across a zebra's body).

CV Weeks 2–3 · Cheat Sheet

Building blocks

Term	One-liner
Conv layer	slides **learned** kernels over the image → feature maps · few parameters (shared weights) · keeps spatial layout · translation invariant
FC (dense) layer	every neuron sees every input · flattens (loses *where*) · huge parameter count · used at the end for the decision
ReLU	max(0, x): negatives → 0 · non-linear, cheap, gradient = 1 for x > 0 → less vanishing gradient
Sigmoid / tanh	slopes ≤ 0.25 / ≤ 1 and ~0 when saturated → gradients vanish in deep nets
Dropout	randomly zero a fraction *p* of neurons **during training only** → robust features, less overfitting · 10 neurons, p = 0.3 → ~3 dropped · survivors × 1/(1−p)
Max pooling 2×2	keep the max of each 2×2 block → halves H and W, no parameters
Training loop	forward → loss → `loss.backward()` (gradients) → `optimizer.step()`
Overfitting	great on training data, worse on new data (a train–test gap)
Vanishing gradient	the gradient is a product of many small factors → early layers stop learning

CNN timeline

Model	Year	Key facts
LeNet-5	1998	**Yann LeCun** · digits / **MNIST** · cheques & post · conv → pool → conv → pool → FC → FC · ~60 K params · proof of concept
AlexNet	2012	won **ImageNet** by a huge margin (top-5 error ~26% → **15.3%**) · **5 conv + 3 FC** · ~61 M params · **ReLU, dropout, GPU** · sparked the deep-learning revolution
VGG	2014	Oxford **Visual Geometry Group** · **16–19 layers** · only **3×3** filters · 2×3×3 = 5×5 view, 3×3×3 = 7×7 view with fewer params + more ReLUs · baseline · ~138 M params
ResNet	2015	solves **vanishing gradient / degradation** · **skip connections**: out = F(x) + x · learns the **residual** · gradient F′(x) + **1** · 50 / 101 / **152** layers · used in medical imaging
EfficientNet	2019	**compound scaling**: depth α^φ, width β^φ, resolution γ^φ, α·β²·γ² ≈ 2 (1.2, 1.1, 1.15) · B0 (5.3 M) beats ResNet-50 (25.6 M) · mobile apps
MobileNet	2017	depthwise-separable conv (per-channel K×K + 1×1 mix) → ~8–9× fewer weights · phones

Object detection = what + where (class + box for every object)

	R-CNN (2014)	YOLO (2016)	SSD (2016)
Stands for	Region-based CNN	You Only Look Once	Single Shot MultiBox Detector
How	~2,000 region proposals → CNN on **each** → classify + refine	**one pass**: S×S grid predicts boxes + classes (a regression problem)	one pass, predictions from **multi-scale feature maps** + default boxes
Speed / accuracy	slow / **most accurate**	**real-time** / weaker on small, crowded objects	fast / good across sizes
Use for	**medical** (accuracy > speed)	**driving**, video (speed > accuracy)	mobile, balanced
- **IoU** = overlap / union (≥ 0.5 = correct box) · **NMS** = keep the best box, drop overlapping duplicates.			
- Fast R-CNN: CNN once, crop the feature map · Faster R-CNN: **RPN** proposes regions.			
- Small objects in YOLO: higher resolution, finer grid, multi-scale heads (FPN), small anchors, tiling, augmentation.			

Segmentation = a label for every pixel

- **Semantic:** class per pixel (touching objects merge) · **Instance:** separate mask per object.
- Challenge: needs **context** (what) **and precision** (where).
- **FCN:** FC → conv layers · dense pixel map · any input size · **coarse** (predicts at low resolution, then upsamples).
- **U-Net (2015): encoder** (downsample → context) + **decoder** (upsample, transposed conv → precision) + **skip connections** (copy + concatenate encoder maps → fine detail) · standard in **medical imaging** (precise boundaries, small structures, small datasets) · also satellite imagery.
- **Mask R-CNN (2017):** Faster R-CNN + **mask branch** per RoI · **RPN** proposes regions · **RoI Align** (bilinear, no rounding) replaces RoI Pooling · outputs **box + class + mask** · handles overlapping objects (one mask per RoI) · masks give the true size and volume in medicine.
- Skip connections: ResNet **adds** (gradient flow, depth) · U-Net **concatenates** (spatial detail).

Sequences & video

- Video = **spatial** (what's in each frame → **CNN**) + **temporal** (how it changes → **RNN / LSTM**).
- **RNN:** hidden state $h_t = \tanh(W x_t + U h_{t-1})$ · **short-term** memory · vanishing gradients over long sequences.
- **LSTM:** cell state + gates · **forget** (discard), **input** (store), **output** (pass on) · $c_t = f \odot c_{t-1} + i \odot \tilde{c}$ · **long-term** memory · variable length.
- **CNN + LSTM:** CNN features per frame → LSTM over time → class (gesture, action, captioning, driving).
- RNN is still enough for short sequences (next character, last few sensor readings).

	RNN	LSTM
Memory	short-term	long-term
Problem	vanishing gradients	solves them
Best for	simple, short	complex, long (video, paragraphs)

Attention & Vision Transformers

- **$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d}) \cdot V$** · **Query** = what I look for · **Key** = what I offer · **Value** = what I pass on → output = weighted sum.
- **ViT (2020):** split into **patches** (e.g. 16×16 → 224^2 image = 196 tokens) → flatten + **linear embedding** + **position embedding** + **[CLS]** token → transformer encoder (attention + MLP, **residual** connections) → classify from [CLS].
- Patches → image becomes a sequence; any patch can attend to any other from layer 1 (**global context**); shorter sequence = affordable; local detail stays inside each patch.
- CNN = local, built-in locality assumption, data-efficient · ViT = global, needs **large datasets**, scales well, hybrid models.
- Uses: classification, medical imaging, video (temporal attention), vision + language.